PROJECT SETUP, EXECUTION & POWER BI REBUILD GUIDE

# Olist E-Commerce Revenue & Customer Retention Analytics

From a fresh Windows computer to a working, verified recreation of the PostgreSQL warehouse, the Python and SQL analyses and the four-page Power BI dashboard

| | |
|---|---|
| **Project** | E-commerce revenue and customer retention analytics on the Olist Brazilian marketplace dataset (2016–2018) |
| **Repository** | `github.com/MARCUS-00/ecommerce-retention-analytics` |
| **Author** | Manoj Kumar |
| **Reader** | A beginner Data Analyst who wants to install, run, understand and independently rebuild every part of the project |
| **Covers** | Windows setup · Docker and PostgreSQL · Python pipeline · SQL · notebooks · tests · Excel summary · Power BI model, all 43 DAX measures and all 44 report visuals · troubleshooting · exercises |
| **Authoritative report** | `dashboard/ecommerce_retention_dashboard_final.pbix` (four pages, Import mode) |
| **Edition** | October 2026 |

*This manual is an additional, hands-on companion to the Complete Project Documentation and the one-page executive memo. It does not replace them.*

# Contents

**How to use this guide** ..... 4
Recommended reading order ..... 4
Conventions ..... 4
Verification labels used in this guide ..... 4
**Part I — Understand the project** ..... 5
I.1 The project in plain English ..... 5
I.2 The project in technical terms ..... 6
I.3 Architecture: how the pieces connect ..... 7
I.4 Repository map ..... 8
I.5 Four different jobs, four different commands ..... 9
I.6 Populations and keys you must not mix ..... 9
I.7 Which Power BI file is the real one? ..... 10
**Part II — Set up a fresh Windows computer** ..... 11
II.1 How each instruction is laid out ..... 11
II.2 Windows Terminal and PowerShell ..... 11
II.3 Git for Windows and the repository ..... 13
II.4 Python 3.12 ..... 13
II.5 Virtual environment and Python packages ..... 14
II.6 Docker Desktop ..... 15
II.7 GNU Make, or the no-make route ..... 15
II.8 Kaggle account and the nine source files ..... 17
II.9 The .env settings file ..... 18
II.10 PostgreSQL in Docker ..... 19
II.11 Excel and other utilities ..... 20
II.12 Power BI Desktop and the PostgreSQL connector ..... 21
II.13 Final setup checklist ..... 21
**Part III — Run the entire project in order** ..... 22
III.1 Before you run anything ..... 22
III.2 The 18-step playbook ..... 23
III.3 First-time setup procedure ..... 28
III.4 Routine run after setup ..... 29
III.5 Restarting after shutting down Windows ..... 29
III.6 Rebuild the database from scratch (destructive) ..... 29
III.7 Fix a password or authentication mismatch ..... 30
III.8 Individual procedures ..... 30
III.9 The everyday workflow ..... 31
III.10 What normal output looks like ..... 32
**Part IV — SQL, Python and notebooks explained** ..... 33
IV.1 The data model ..... 33
IV.2 Staging and the Python entry points ..... 35
IV.3 Data-quality checks and the test suite ..... 37
IV.4 The transformation SQL ..... 38
IV.5 The 18 business queries ..... 39
IV.6 The four dashboard views ..... 41

IV.7 The four notebooks ... 42
IV.8 Concepts, explained with this project's data ... 44
IV.9 Historical outputs versus a fresh run ... 46
IV.10 Continuous integration and the CI fixtures ... 47
**Part V — Rebuild the Power BI report ... 48**
How to use Part V ... 48
V.A Prepare and verify the data before opening Power BI ... 48
V.B Connect Power BI Desktop to PostgreSQL ... 49
V.C Object inventory ... 52
V.D The data model and relationships ... 56
V.E DAX: concepts, calculated columns and all 43 measures ... 58
V.F Build the four report pages ... 85
V.G Slicers, filters and interactions ... 116
V.H The forecast route ... 117
V.I Formatting, accessibility and what is confirmed ... 118
V.J Save, refresh, verify and reopen ... 119
**Part VI — Findings, limitations and how to explain them ... 123**
VI.1 KPI trace: from raw data to dashboard to wording ... 123
VI.2 The seven findings ... 124
VI.3 The Mann–Whitney test, worded correctly ... 125
VI.4 Association is not causation ... 125
VI.5 Forecast evaluation ... 126
VI.6 Which population is each number about? ... 126
VI.7 Limitations to state up front ... 127
VI.8 Explaining the project aloud ... 128
**Part VII — Troubleshooting and maintenance ... 129**
VII.1 A method that works for every problem ... 129
VII.2 Docker and PostgreSQL ... 129
VII.3 SQL, the pipeline and the Makefile ... 130
VII.4 Windows, Python, Git and Kaggle ... 130
VII.5 Notebooks ... 131
VII.6 Tests, linting and compilation ... 131
VII.7 Power BI ... 132
VII.8 Incidents from the original build (historical record) ... 132
VII.9 Maintenance ... 133
VII.10 Extending the project safely ... 133
**Part VIII — Hands-on practice ... 135**
VIII.1 Exercises with hints and answer keys ... 135
VIII.2 A staged learning plan ... 138
VIII.3 “I can explain it unaided” checklist ... 138
**Appendix A — Command reference ... 140**
**Appendix B — What was verified, and how ... 141**
**Appendix C — Index of the 43 measures and 44 visuals ... 142**
**Appendix D — Official links ... 145**
**Glossary ... 146**

# How to use this guide

## Recommended reading order

| If you want to... | Read | Pages |
|---|---|---|
| Understand what the project is before touching anything | Part I | 5 |
| Set up a brand-new Windows PC | Part II, then Part III.2 (the 18-step playbook) | 11 |
| Run it again tomorrow | Part III.4 (routine run) and III.9 (everyday workflow) | 22 |
| Understand the SQL, Python and notebooks | Part IV | 33 |
| Recreate the Power BI report | Part V (highest priority), with the screenshots from the repository open beside it | 48 |
| Explain the results to others | Part VI | 123 |
| Fix something that broke | Part VII | 129 |
| Practise until you can do it unaided | Part VIII | 135 |

## Conventions

- **Terminal.** Every command is for **Windows PowerShell or PowerShell 7 in Windows Terminal** unless a label says otherwise. Part II.2 explains the differences from bash.
- **Working folder.** Unless stated, run commands from the repository root (the folder that contains `Makefile`, `src` and `sql`). In the owner's copy this is `D:\PROGRAMING\Data Analytics\ecommerce retention analytics\code`. On a fresh PC choose any location; a path without spaces is easier.
- **Placeholders** appear in angle brackets, for example `<your-password>`. Replace the whole thing, including the brackets. Passwords and tokens are never written in this guide.
- **Code labels and cards.** A small grey label above each code block names its language. Instruction cards carry seven fields: Where, Action, Purpose, Expected result, Verification, Troubleshooting, Next step. Destructive commands are boxed in red.

## Verification labels used in this guide

Not every instruction could be exercised on every platform, so a box states how it was checked, using four labels:

| Label | Meaning |
|---|---|
| **Executed** | The step was run end to end against the real Olist dataset in a Linux test environment (PostgreSQL 16.2, Python 3.12, the pinned `requirements.txt`). Results quoted as "executed" are real outputs. |
| **File-verified** | The fact was read directly from the final Power BI file (`.pbix`), the repository source, or the dashboard QA report. No guesswork is involved. |
| **Reviewed** | Checked by reading the code or the official documentation, but not run. Menu names in Power BI Desktop and Docker Desktop fall here: they follow Microsoft's and Docker's current documentation and can differ slightly between releases. |
| **Not verified** | Depends on something that could not be exercised: Docker Desktop on Windows, GNU Make on Windows, the Kaggle download, GitHub Actions, or a live Power BI Desktop refresh. A safe check is always given. Appendix B lists every such item. |

# Part I — Understand the project

This part explains what the project is, why it exists, how its pieces fit together and which of its files matter. It starts in plain English and becomes technical gradually.

## I.1 The project in plain English

Olist is a Brazilian online marketplace that connects small sellers with customers. The public dataset behind this project contains about 100,000 real, anonymised orders placed between 2016 and 2018, spread across nine related tables: customers, orders, order items, payments, reviews, products, sellers, locations and a category-name translation.

Imagine three managers preparing an annual plan. Each asks one question that the raw tables cannot answer on their own:

| Stakeholder | Question | Where the answer lives |
|---|---|---|
| Finance (CFO) | What drives revenue growth, and how concentrated is revenue? Concentration is risk. | Executive Overview and Sales Deep-Dive pages; SQL queries 01–06 |
| Marketing (CMO) | What share of customers ever come back, and which customer groups deserve spend? | Customers & Retention page; queries 12–15; notebook 02 |
| Operations | Do late deliveries hurt customer satisfaction, is the effect statistically real, and how many orders should we plan for next month? | Operations & Satisfaction page; queries 08–11; notebooks 03 and 04 |

The project turns the nine raw files into a trustworthy database, answers those questions with 18 SQL queries and four notebooks, checks the data quality automatically, and presents the answers on a four-page Power BI dashboard, an Excel summary and a one-page executive memo.

### What the project found

- **Customers rarely come back.** Only 2,801 of 93,358 customers (3.00%) ever placed a second delivered order. The marketplace is acquisition-driven, not retention-driven.
- **Late delivery goes with bad reviews.** Late orders have a median review score of 2 out of 5, with 54.07% rated one or two stars. On-time orders have a median of 5, with 9.22% rated one or two stars. A one-sided Mann–Whitney U test gives $p < 0.001$. This is an association, not proof of cause.
- **One in twelve orders is late.** 7,826 of 96,478 delivered orders (8.11%) arrived after the promised date.
- **Revenue is concentrated.** The top 5 of 74 categories earn 39.25% of delivered revenue; the top 10 of 2,970 sellers earn 12.93%; São Paulo state alone accounts for 37.42%.
- **Payments reconcile.** Item totals and payment totals differ on 0.30% of orders, and the gap is explained by vouchers, instalments and cancellations rather than by a pipeline error.
- **The simple forecast is good enough.** Over a six-fold walk-forward test, repeating last week (WAPE 0.2308) was not beaten by Holt–Winters (0.2310). The 28-day forecast is 6,660 orders.

Total delivered revenue (item price plus freight) is **R$15,419,773.75** over **96,478** delivered orders, an average order value of **R$159.83**.

> WHY THIS MATTERS
>
> Two limits to keep in mind from the start. The data is a frozen 2016–2018 snapshot from one marketplace, so nothing here is "live". And every finding is observational: the project shows what happened together, not what caused what.

## I.2 The project in technical terms

| Layer | Technology | Role in this project |
|---|---|---|
| Source data | Kaggle dataset `olistbr/brazilian-ecommerce`, nine CSV files | Downloaded once into `data/raw/` (never committed to Git) |
| Database | PostgreSQL 16 in a Docker container (`olist_postgres`) | Holds two schemas: `stg` (raw mirror) and `analytics` (clean star schema, dashboard views, forecast table) |
| Orchestration | Python 3.12 modules in `src/`, run with `python -m src.<module>`; GNU Make shortcuts | Download, load, validate, transform, analyse, build views, build Excel |
| Transformation | SQL files in `sql/01_schema`, `sql/02_transform` | Cast types, deduplicate, derive `delivery_days` and `is_late` |
| Analysis | SQL (18 queries) and four Jupyter notebooks (pandas, SciPy, statsmodels, matplotlib) | Answer the business questions, run the statistical test, produce the forecast |
| Quality | pytest (10 tests) driven by `src/validate.py` | Input and output data-quality gates |
| Presentation | Power BI Desktop (Import mode), DAX; Excel summary workbook; PDF/DOCX documents | Four-page dashboard, 43 measures, executive memo |
| Automation | GitHub Actions workflow `ci.yml` | Runs fixtures, pipeline and tests on every push (synthetic data, no Kaggle needed) |

Three design decisions shape every query in the project:

- **The person key is `customer_unique_id`, not `customer_id`.** In this dataset `customer_id` is created fresh for every order, so counting it would make every customer look like a one-time buyer. All retention, repeat-rate and RFM logic joins through `customer_unique_id`.
- **Revenue = item price + freight value, delivered orders only.** This definition is reconciled against payments (Finding 5) and used identically in SQL, Python and DAX.
- **Three populations are never mixed** (I.6): delivered orders, all placed orders, and all payment records.

## I.3 Architecture: how the pieces connect

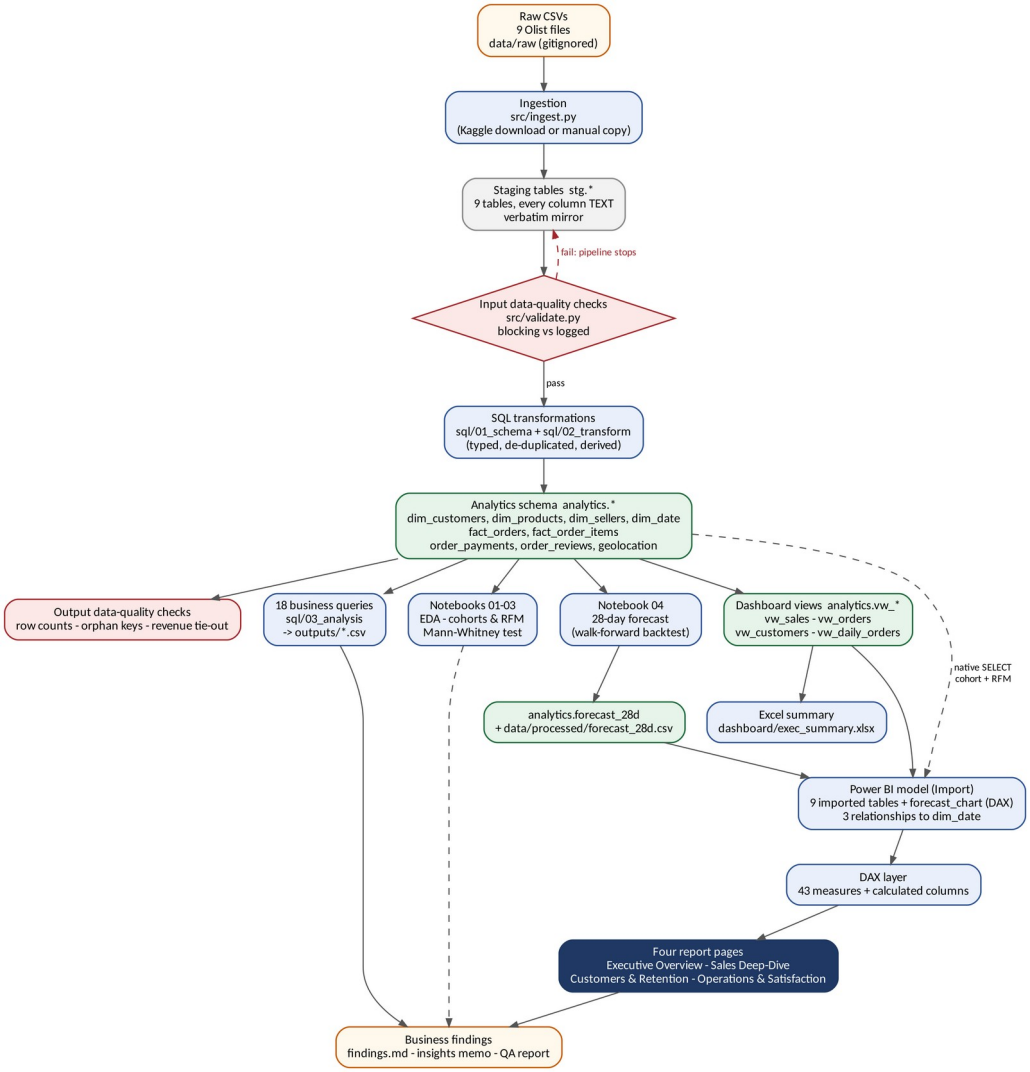



**Figure 1.** End-to-end architecture. Solid arrows are data flow; each box names the file or command that produces it.

Read the diagram from the top. The table below restates each stage with its command, so you can map the picture to the terminal.

| Stage | What happens | Command / file | Output |
|---|---|---|---|
| 1 Ingest | Download the nine CSVs if absent; load each verbatim as TEXT into `stg.*` | `make pipeline` → `src/ingest.py` | `stg.*` (9 tables, 1,000,163 rows in geolocation alone) |
| 2 Input checks | Row counts, primary keys, orphan keys, domain values, timestamp order, null policy, duplicate reviews | `src/validate.py` | Pass/fail plus a log of data anomalies (9 logged on the real data) |
| 3 Transform | Create the `analytics` schema and calendar, then typed tables with keys and derived columns | `sql/01_schema`, `sql/02_transform` via `src/transform.py` | 8 tables plus `dim_date` |
| 4 Output checks | Row reconciliation to staging, orphan keys, revenue tie-out | `src/validate.py` | Pass/fail |
| 5 Business queries | Run 18 saved SQL queries; save the first 200 rows of each | `make analysis` → `src/run_analysis.py` | `sql/03_analysis/outputs/*.csv` |
| 6 Notebooks | EDA, cohorts and RFM, statistical test, forecast | `make notebooks` | Executed `.ipynb` files; `analytics.forecast_28d`; `data/processed/forecast_28d.csv` |

| Stage | What happens | Command / file | Output |
|---|---|---|---|
| 7 Dashboard views | Four views that shape data for Power BI | `make views` → `src/build_dashboard_views.py` | `analytics.vw_sales`, `vw_orders`, `vw_customers`, `vw_daily_orders` |
| 8 Excel summary | Pivot-style workbook from `vw_sales` | `make excel` → `src/build_excel_summary.py` | `dashboard/exec_summary.xlsx` |
| 9 Power BI | Import nine tables, add a calendar relationship, 43 measures, four pages | Power BI Desktop | `dashboard/ecommerce_retention_dashboard_final.pbix` |

## I.4 Repository map

```
TEXT
ecommerce-retention-analytics/
|-- Makefile                    shortcuts for every command (needs GNU Make)
|-- docker-compose.yml          PostgreSQL 16 container, port, healthcheck, volume
|-- .env.example                template for the (git-ignored) .env settings file
|-- requirements.txt            pinned Python packages
|-- README.md  LICENSE  .gitignore
|-- .github/workflows/ci.yml    automatic test run on GitHub
|-- src/                        8 Python modules: utils, ingest, validate, transform,
|                               run_pipeline, run_analysis, build_dashboard_views,
|                               build_excel_summary
|-- sql/
|   |-- 01_schema/              analytics schema + generated dim_date
|   |-- 02_transform/           8 files: stg.* -> analytics.*
|   |-- 03_analysis/            18 business queries (+ outputs/ CSV samples)
|   `-- 04_dashboard_views/     4 Power BI feeding views
|-- notebooks/                  01_eda, 02_retention_rfm, 03_delivery_reviews,
|                               04_demand_forecast
|-- tests/                      test_input_quality, test_output_quality,
|                               fixtures/generate_fixtures.py
|-- data/
|   |-- raw/                    the 9 Kaggle CSVs (git-ignored; you supply them)
|   `-- processed/              forecast_28d.csv
|-- dashboard/                  final .pbix, 4 screenshots, QA report, DAX list,
|                               exec_summary.xlsx, archive/ (local only)
`-- docs/                       PLAYBOOK, architecture, data_dictionary, findings,
                                monitoring_policy, insights_memo (+ archived specs)
```

| Path | Read | Run | Edit | Notes |
|---|---|---|---|---|
| `README.md`, `docs/findings.md`, `docs/data_dictionary.md` | Yes | – | Rarely | Start here for context |
| `Makefile`, `docker-compose.yml` | Yes | Indirectly | Avoid | Changing the compose file can change database behaviour; see II.10 |
| `.env` | Open only in an editor, never share | – | Once | Holds the database password; ignored by Git |
| `src/*.py` | Yes | `make …` or `python -m src.<name>` | Avoid during a rebuild | Edit only when extending the project |
| `sql/01_schema`, `sql/02_transform` | Yes | Via `make pipeline` | Avoid | Every file drops and recreates its table |
| `sql/03_analysis/*.sql` | Yes | Via `make analysis`, or paste into any SQL client | Safe to experiment (copy first) | Read-only SELECTs |
| `sql/04_dashboard_views/*.sql` | Yes | Via `make views` | Avoid | Power BI imports these views |
| `notebooks/*.ipynb` | Yes | `make notebooks` or one by one | Avoid editing outputs by hand | Execution rewrites the file in place |
| `data/raw/` | – | – | Never edit | Re-download if damaged |

| Path | Read | Run | Edit | Notes |
|---|---|---|---|---|
| `dashboard/*.pbix` | Open in Power BI Desktop | Refresh | Save a copy first | The authoritative report |
| `dashboard/archive/` | No | – | – | Local, ignored by Git: older copies and PBIP exports |
| `docs/MASTER_DOC.md` | Background only | – | No | Archived planning document |

## I.5 Four different jobs, four different commands

Beginners often say "rerun the project" and mean four very different things. Knowing which one you need saves time and prevents accidental data loss.

| Job | What it does | Typical trigger | Runs | Changes or destroys |
|---|---|---|---|---|
| **Rebuild the pipeline** | Reloads raw CSVs into `stg`, validates, and recreates every `analytics` table | First setup; the SQL transformations changed; the database was lost | `make pipeline` | Drops and recreates all `analytics` tables **and silently drops the four dashboard views** (see III.1). Does not touch `data/raw`. |
| **Regenerate analytical results** | Re-run queries, notebooks and the Excel summary on the existing tables | You want fresh CSV/notebook/forecast outputs | `make analysis`, `make notebooks`, `make excel` | Overwrites `sql/03_analysis/outputs`, the `.ipynb` files, `analytics.forecast_28d`, `data/processed/forecast_28d.csv`, `dashboard/exec_summary.xlsx` |
| **Refresh Power BI** | Reloads the nine imported tables from PostgreSQL into the open report | Underlying data changed | Home → Refresh in Power BI Desktop | Only the report's in-memory data. Requires the database running **and the views present**. |
| **Recreate the report visuals** | Rebuilds pages, visuals and measures by hand | Learning; or the `.pbix` is lost | Part V of this guide | Nothing in the database; produces a new `.pbix` |

## I.6 Populations and keys you must not mix

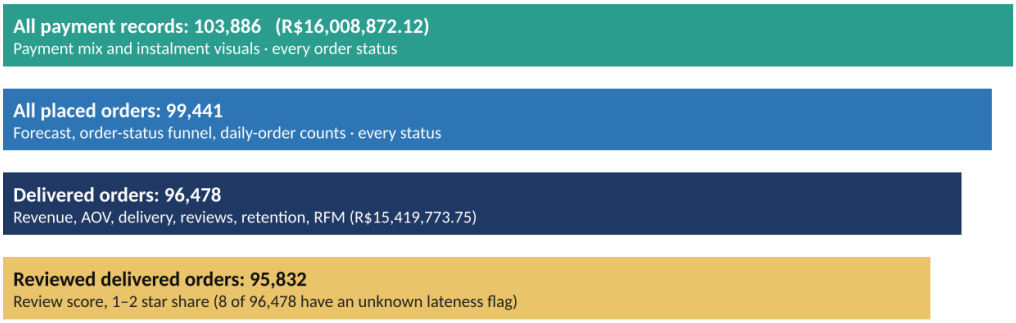



**Figure 2.** The four populations used in the project. Bar lengths are only indicative.

| Population | Size | Used for | Why |
|---|---|---|---|
| Delivered orders (`order_status = 'delivered'`) | 96,478 | Revenue, AOV, delivery metrics, reviews, retention, RFM, repeat rate | Revenue is only recognised when the order is delivered |

| Population | Size | Used for | Why |
|---|---|---|---|
| All placed orders | 99,441 | The 28-day forecast, the order-status funnel (query 07), the daily-order counts | Operations must plan capacity for every order regardless of eventual outcome |
| All payment records | 103,886 (R$16,008,872.12) | Payment-mix and instalment visuals | One order can have several payments, and cancelled orders still carry payment rows. These visuals will therefore **not** reconcile to delivered revenue; the dashboard says so on the page. |
| Reviewed delivered orders | 95,832 | Review score, 1–2★ share | Not every delivered order has a review (8 more have no lateness flag) |

> CAUTION
> Two numbers that look like they should match but do not: delivered revenue **R$15,419,773.75** and total payment value **R$16,008,872.12**. The difference is expected. It comes from the different populations above, not from an error. Query 16 reports the 303 orders whose item and payment totals differ by more than R$0.01.

## I.7 Which Power BI file is the real one?

The repository tracks exactly one report file: **`dashboard/ecommerce_retention_dashboard_final.pbix`**. Everything in this guide that says "the final report" refers to it.

- **Superseded copies and PBIP exports** (a PBIP is a folder-based project format) live in `dashboard/archive/`. That folder is listed in `.gitignore` except for its README, so it exists only on the owner's computer and is not on GitHub.
- **To tell them apart:** the final report is the only `.pbix` directly inside `dashboard/`, its name ends in `_final`, and `git ls-files dashboard` lists it. Anything in `archive/` is older and may contain different measures or relationships.
- **Do not open an archived file expecting it to match this guide.** The older planning document `docs/PLAYBOOK.md` section 8 still describes an early six-object model with a relationship to `forecast_28d`; the final report has 11 tables and exactly three relationships (Part V).

# Part II — Set up a fresh Windows computer

This part assumes a Windows computer on which none of the project's tools are installed. Follow the sections in order: each tool depends on the ones before it. Allow two to three hours for the first time, most of it waiting for downloads.

| Section | Tool | Needed for |
|---|---|---|
| II.2 | Windows Terminal and PowerShell | Every command in this guide |
| II.3 | Git for Windows | Getting the code |
| II.4 | Python 3.12 | Pipeline, tests, notebooks |
| II.5 | Virtual environment and packages | Isolated Python libraries |
| II.6 | Docker Desktop | Running PostgreSQL |
| II.7 | GNU Make (or the no-make route) | Short commands such as `make pipeline` |
| II.8 | Kaggle account and the data files | The nine source CSVs |
| II.9 | The `.env` settings file | Database name, user, password, port |
| II.10 | PostgreSQL container | The warehouse |
| II.11 | Excel or another spreadsheet app (optional) | Opening `exec_summary.xlsx` |
| II.12 | Power BI Desktop and the PostgreSQL connector | The dashboard |
| II.13 | Final setup checklist | Proof that everything works |

> NOTE
> **System requirements to check first.** Windows 10 22H2 (build 19045) or Windows 11 23H2 (build 22631) or newer; 64-bit processor with hardware virtualisation enabled in the BIOS/UEFI; 8 GB RAM minimum for Docker Desktop; about 5 GB of free disk for Docker images, the database volume (the finished database is about 290 MB), raw CSVs (121 MB) and Python packages. Power BI Desktop needs a screen of at least 1440 × 900 for comfortable report building. Check your Windows build with `winver`.

## II.1 How each instruction is laid out

Every action is shown as a card. Read the **Purpose** so you know why you are doing it, run the **Action**, compare with **Expected result**, prove it with **Verification**, and only then move to **Next step**. If something differs, go to **Troubleshooting** before continuing; problems compound if ignored.

## II.2 Windows Terminal and PowerShell

| STEP 2.1 · Open a PowerShell tab in Windows Terminal | |
|---|---|
| **Where** | Windows desktop. Windows 11 includes **Windows Terminal**; on Windows 10 install it from the Microsoft Store (publisher: Microsoft Corporation). |
| **Action** | Press the Windows key, type `Terminal`, press Enter. The default tab is Windows PowerShell (or PowerShell 7 if installed). Both work for this guide. Do **not** use Command Prompt (`cmd`) or Git Bash for the commands in this guide unless a step says so. |
| **Purpose** | All commands, paths and activation syntax in this guide are PowerShell syntax. Mixing shells is the most common cause of "command not found" and quoting errors. |

| | |
|---|---|
| **Expected result** | A window with a prompt such as `PS C:\Users\<you>>`. |
| **Verification** | `$PSVersionTable.PSVersion`<br>A version table prints. Windows PowerShell 5.1 or PowerShell 7.x are both fine. |
| **Troubleshooting** | If the tab says `C:\>` with no `PS`, you are in Command Prompt: open the tab drop-down and choose **Windows PowerShell**. |
| **Next step** | II.3 Git for Windows. |

## Shell differences that matter in this project

| Task | PowerShell (use this) | Bash / macOS / Linux (do not paste into PowerShell) |
|---|---|---|
| Activate the virtual environment | `.\.venv\Scripts\Activate.ps1` | `source .venv/bin/activate` |
| Python inside the venv | `.\.venv\Scripts\python.exe` | `.venv/bin/python` |
| Copy a file | `Copy-Item .env.example .env` | `cp .env.example .env` |
| Set an environment variable for one session | `$env:NAME = "value"` | `export NAME=value` |
| Show a file | `Get-Content file.txt` | `cat file.txt` |
| Folders with spaces | Wrap in quotes: `cd "D:\My Folder"` | Same, with forward slashes |
| Line continuation | A backtick at the end of the line | A backslash at the end of the line |
| Delete a folder | `Remove-Item -Recurse -Force .\folder` | `rm -rf folder` |

> **TIP**
> **You never have to activate the virtual environment.** Every command in this guide can call the venv's Python directly (`.\.venv\Scripts\python.exe -m …`). This avoids the PowerShell execution-policy error that blocks `Activate.ps1` on many new PCs. The Makefile does exactly the same: it calls `.venv/Scripts/python.exe` on Windows.

| **STEP 2.2 · (Only if you want to activate the venv) allow local scripts** | |
|---|---|
| **Where** | PowerShell. |
| **Action** | `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`<br>Answer `Y` when asked. |
| **Purpose** | Windows blocks unsigned scripts by default, so `Activate.ps1` fails with "running scripts is disabled on this system". `RemoteSigned` lets locally created scripts run for your user only. |
| **Expected result** | No output after you confirm. |
| **Verification** | `Get-ExecutionPolicy -Scope CurrentUser`<br>Prints `RemoteSigned`. |
| **Troubleshooting** | On a managed or work PC a group policy may override this. In that case skip activation and call `.\.venv\Scripts\python.exe` directly. |
| **Next step** | Optional. Continue with II.3. |

## II.3 Git for Windows and the repository

| STEP 3.1 · Install Git | |
|---|---|
| **Where** | Browser, then the downloaded installer. Official source: **git-scm.com/downloads/win** (the standalone installer, 64-bit). |
| **Action** | Run the installer and accept the defaults. The default adds `Git\cmd` to PATH, which is all this project needs. (Optional route: `winget search Git.Git` to confirm the package id, then `winget install --id Git.Git -e`.) |
| **Purpose** | Git downloads the project from GitHub and records your changes. |
| **Expected result** | The installer finishes without errors. |
| **Verification** | Open a **new** terminal tab (PATH changes only reach new tabs):<br>`git --version`<br>Prints `git version 2.x…`. |
| **Troubleshooting** | "git is not recognized": close every terminal and open a new one; if it persists, re-run the installer and keep "Git from the command line" selected. |
| **Next step** | STEP 3.2. |

| STEP 3.2 · Clone the repository | |
|---|---|
| **Where** | PowerShell, in the folder where you keep projects. |
| **Action** | `mkdir C:\Projects`<br>`cd C:\Projects`<br>`git clone https://github.com/MARCUS-00/ecommerce-retention-analytics.git`<br>`cd ecommerce-retention-analytics`<br>If a command in `mkdir` says the folder exists, that is fine. |
| **Purpose** | Creates a local copy of the code, SQL, notebooks, documents and the final `.pbix`. Choose a path **without spaces** if you can. The owner's copy lives under `D:\PROGRAMING\Data Analytics\ecommerce retention analytics\code`; spaces work but every path must then be quoted. |
| **Expected result** | A new folder `ecommerce-retention-analytics` containing `Makefile`, `src`, `sql`, `notebooks`, `dashboard`. |
| **Verification** | `git status`<br>`git log --oneline -3`<br>`dir`<br>`git status` says the working tree is clean; `dir` lists `Makefile`, `docker-compose.yml`, `requirements.txt`. |
| **Troubleshooting** | • "repository not found": check the URL for typos.<br>• Long-path errors: run `git config --global core.longpaths true` and clone again.<br>• Do **not** move the folder later while Docker or Power BI is using it. |
| **Next step** | II.4 Python. |

> NOTE
> The commands that follow assume your terminal is in the repository root. If you open a new tab, `cd` back first.

## II.4 Python 3.12

The pinned packages in `requirements.txt` were installed and tested on **Python 3.12** (the CI workflow also uses 3.12; the owner's Windows environment is Python 3.12.10). The README asks for 3.11 or 3.12; 3.12 is the version with evidence behind it. Do not use 3.13 or newer: the pinned `scipy==1.13.1` has no prebuilt

Windows package for them (checked against PyPI), so `pip install -r requirements.txt` would try to compile it and fail.

| STEP 4.1 · Install Python 3.12 | |
|---|---|
| **Where** | Browser: **python.org/downloads/windows/**, choose a 3.12.x "Windows installer (64-bit)". The Microsoft Store edition of Python 3.12 also works and is what the owner used; use one or the other, not both. |
| **Action** | Run the installer. On the first screen tick **Add python.exe to PATH**, then **Install Now**. |
| **Purpose** | Provides the interpreter that runs the pipeline, tests and notebooks. |
| **Expected result** | "Setup was successful". |
| **Verification** | In a **new** terminal:<br>`python --version`<br>`py -0p`<br>The first prints `Python 3.12.x`. The second lists installed interpreters (a `py` launcher appears with the python.org installer only). |
| **Troubleshooting** | • "Python was not found; run without arguments to install from the Microsoft Store": Windows is using its alias stub. Open **Settings → Apps → Advanced app settings → App execution aliases** and turn off the `python.exe` and `python3.exe` aliases, or reinstall with PATH ticked.<br>• Wrong version prints: another Python is earlier in PATH. Use `py -3.12 -m venv .venv` in STEP 5.1. |
| **Next step** | II.5 Virtual environment. |

## II.5 Virtual environment and Python packages

A **virtual environment** is a private folder (`.venv`) holding this project's copy of Python and its libraries, so they cannot clash with anything else on the PC. `make setup` does the three steps below; here they are by hand.

| STEP 5.1 · Create the virtual environment and install packages | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Action** | `python -m venv .venv`<br>`.\.venv\Scripts\python.exe -m pip install --upgrade pip`<br>`.\.venv\Scripts\python.exe -m pip install -r requirements.txt`<br>If `python` points at the wrong version, use `py -3.12 -m venv .venv` for the first line. |
| **Purpose** | Installs the exact pinned versions: pandas 2.2.3, SQLAlchemy 2.0.51, psycopg2-binary 2.9.12, SciPy 1.13.1, statsmodels 0.14.6, matplotlib 3.9.4, python-dotenv 1.2.2, pytest 8.4.2, jupyter 1.1.1, nbconvert 7.17.1, nbformat 5.10.4, kaggle 2.2.3 and openpyxl 3.1.5. |
| **Expected result** | A long download log ending in `Successfully installed …`. Takes several minutes. |
| **Verification** | `.\.venv\Scripts\python.exe --version`<br>`.\.venv\Scripts\python.exe -m pip check`<br>`.\.venv\Scripts\python.exe -c "import pandas, sqlalchemy, psycopg2, scipy, statsmodels, openpyxl; print(pandas.__version__)"`<br>Version `3.12.x`; `No broken requirements found.`; and `2.2.3`. |
| **Troubleshooting** | • Build errors mentioning a compiler: you are probably on a Python version without prebuilt wheels. Recreate the venv with 3.12.<br>• Timeouts: retry; behind a proxy set `HTTPS_PROXY`.<br>• To start over, delete only the `.venv` folder (safe: it contains no project data) and repeat the step. |
| **Next step** | II.6 Docker Desktop. |

VERIFICATION STATUS

**Executed.** On Linux with Python 3.12.3, the pinned requirements installed cleanly, `pip check` reported "No broken requirements found", `python -m kaggle --version` printed `Kaggle CLI 2.2.3` and `python -m compileall src tests` passed. The Windows installer steps are Reviewed against python.org.

## II.6 Docker Desktop

**Docker** runs software in lightweight, isolated "containers". Here it runs PostgreSQL so that you install no database server directly on Windows. Key terms: an **image** is the downloadable template (`postgres:16`); a **container** is a running copy of it (`olist_postgres`); a **volume** is storage that survives when the container is removed (`olist_pg_data`, holding the database files); a **port mapping** connects a Windows port to a port inside the container; a **healthcheck** is a command Docker runs repeatedly (`pg_isready`) to decide whether the database is truly ready.

| STEP 6.1 · Install and start Docker Desktop | |
|---|---|
| **Where** | Browser: **docs.docker.com/desktop/setup/install/windows-install/** (official). Download "Docker Desktop for Windows". |
| **Action** | Run the installer and keep the **WSL 2** backend option. Restart if asked. Start **Docker Desktop** from the Start menu and accept the licence terms. Wait until the status at the bottom-left of the window shows the engine as running. |
| **Purpose** | Provides `docker` and `docker compose`, which `docker-compose.yml` uses to run PostgreSQL 16. |
| **Expected result** | Docker Desktop shows "Engine running". The whale icon in the system tray stops animating. |
| **Verification** | `docker --version`<br>`docker compose version`<br>`docker run --rm hello-world`<br>The last command downloads a tiny test image, prints "Hello from Docker!", and removes the container. |
| **Troubleshooting** | • "Virtualization is disabled": enable Intel VT-x / AMD-V (SVM) in the BIOS/UEFI.<br>• WSL errors: in PowerShell run `wsl --update`, then restart Docker Desktop. Docker's documented minimum is WSL 2.1.5.<br>• "error during connect … docker_engine": Docker Desktop is not running yet; start it and wait.<br>• Low RAM: close other apps; Docker Desktop recommends at least 8 GB in total. |
| **Next step** | II.7 GNU Make. |

VERIFICATION STATUS

**Not verified.** Docker Desktop installation and the commands above were not executed on Windows. They follow Docker's official Windows installation page. The `docker compose up --wait` behaviour (waiting for the healthcheck, implying detached mode) was confirmed in Docker's CLI reference.

## II.7 GNU Make, or the no-make route

The Makefile gives short names to long commands. **GNU Make is not part of Windows and does not come with Git for Windows.** You have two options. Option A installs Make. Option B skips Make entirely using the equivalents in the table below; the project works identically either way.

| STEP 7.1 · Option A: install Make with Scoop or Chocolatey | |
|---|---|
| **Where** | Scoop: a normal PowerShell tab (**scoop.sh**, official). Chocolatey: an **elevated** PowerShell ("Run as administrator"; **chocolatey.org**). |
| **Action** | Scoop route:<br>`Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`<br>`Invoke-RestMethod -Uri https://get.scoop.sh \| Invoke-Expression`<br>`scoop install make`<br>Chocolatey route (after installing Chocolatey from its official site):<br>`choco install make -y` |
| **Purpose** | Lets you type `make pipeline` instead of the full Python command. |
| **Expected result** | The installer reports success and adds `make` to PATH (GNU Make 4.4.1 is the Chocolatey package version). |
| **Verification** | Open a **new** terminal:<br>`make --version`<br>Prints `GNU Make 4.x`. |
| **Troubleshooting** | • "make is not recognized": new terminal needed, or PATH was not updated.<br>• If a `make` target fails with a path or shell error that the equivalent Python command does not, use Option B. The Makefile's `clean` target in particular uses Unix tools (`rm`, `find`) that Windows lacks; never use it (see the warning in III.1).<br>• Scoop and Chocolatey download scripts from the internet; install them only from their official sites and read what they ask before approving. |
| **Next step** | II.8 Kaggle. |

## Option B: the no-make equivalents

Run each of these from the repository root. They are the exact commands from the Makefile, with `.venv/Scripts/python.exe` written in Windows form. Use whichever you prefer; the rest of this guide shows both.

| Make target | PowerShell equivalent | What it does |
|---|---|---|
| `make setup` | `python -m venv .venv` then `.\.venv\Scripts\python.exe -m pip install --upgrade pip` then `… -m pip install -r requirements.txt` | Create the venv and install packages |
| `make db-up` | `docker compose up -d --wait` | Start PostgreSQL and wait until healthy |
| `make db-down` | `docker compose down` | Stop and remove the container; **keeps** the data volume |
| `make fixtures` | `.\.venv\Scripts\python.exe tests/fixtures/generate_fixtures.py data/raw` | Write synthetic CSVs into `data/raw` (**overwrites real data**) |
| `make pipeline` | `.\.venv\Scripts\python.exe -m src.run_pipeline` | Ingest, validate, transform, validate |
| `make analysis` | `.\.venv\Scripts\python.exe -m src.run_analysis` | Run the 18 queries and save CSV samples |
| `make views` | `.\.venv\Scripts\python.exe -m src.build_dashboard_views` | Create the four Power BI views |
| `make excel` | `.\.venv\Scripts\python.exe -m src.build_excel_summary` | Write `dashboard/exec_summary.xlsx` |
| `make test` | `.\.venv\Scripts\python.exe -m pytest -q` | Run the 10 data-quality tests |
| `make notebooks` | Four commands: `.\.venv\Scripts\python.exe -m jupyter nbconvert --to notebook --execute --inplace notebooks/01_eda.ipynb`, and the same for `02_retention_rfm`, `03_delivery_reviews`, `04_demand_forecast` | Execute notebooks and overwrite them with fresh outputs |
| `make clean` | **No safe Windows equivalent is given.** It runs `docker compose down -v`, which deletes the database volume, then `rm`/`find` commands. | Destructive; see III.1 |

VERIFICATION STATUS

**Executed / Not verified.** Every `python -m ...` and `pytest` equivalent above was executed on Linux (same commands, with `/` paths). GNU Make itself, and its behaviour under Windows `cmd/sh`, was not run on Windows. The Makefile selects `.venv/Scripts/python.exe` on Windows through `ifeq ($(OS),Windows_NT)` (File-verified).

## II.8 Kaggle account and the nine source files

The data comes from the public Kaggle dataset **Brazilian E-Commerce Public Dataset by Olist** (slug `olistbr/brazilian-ecommerce`). The pipeline can download it for you, or you can place the files yourself. The loader skips the download whenever all nine CSVs are already in `data/raw/`, so the manual route is fully supported.

| STEP 8.1 · Option A: let the pipeline download with the Kaggle CLI | |
|---|---|
| **Where** | Browser (kaggle.com) and PowerShell. |
| **Action** | 1. Create or sign in to a free Kaggle account.<br>2. Open your profile **Settings** and, in the **API** section, create an API token. Kaggle's current CLI documentation describes several credential methods; this project uses the long-established file method: the download is a file named `kaggle.json`.<br>3. Put the file here:<br>`mkdir "$env:USERPROFILE\.kaggle" -Force`<br>`Move-Item "$HOME\Downloads\kaggle.json" "$env:USERPROFILE\.kaggle\kaggle.json"`<br>4. Keep it private. Never paste its contents into a chat, a screenshot or a Git commit. `.gitignore` already excludes a `kaggle.json` placed in the repository. |
| **Purpose** | `src/ingest.py` runs `python -m kaggle datasets download -d olistbr/brazilian-ecommerce -p data/raw --unzip` when files are missing. The CLI needs credentials to authenticate. |
| **Expected result** | The token file exists in your user profile folder. |
| **Verification** | `Test-Path "$env:USERPROFILE\.kaggle\kaggle.json"`<br>`.\.venv\Scripts\python.exe -m kaggle --version`<br>`True`, then `Kaggle CLI 2.2.3`. The real test is the first `make pipeline` (III.2). |
| **Troubleshooting** | • 401/403 errors: the token is wrong, expired or was regenerated. Create a new token and replace the file.<br>• If your Kaggle settings page offers only the newer token types, read the Kaggle CLI documentation at github.com/Kaggle/kaggle-cli (docs/README.md): it lists the supported alternatives (`kaggle auth login`, the `KAGGLE_API_TOKEN` variable, or `~/.kaggle/access_token`). If the CLI authenticates, the pipeline will work, because it only shells out to the CLI.<br>• If you are blocked, use Option B. |
| **Next step** | II.9 `.env`. |

| STEP 8.2 · Option B: place the files manually (always works) | |
|---|---|
| **Where** | Browser and File Explorer. |
| **Action** | Open `https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce`, click **Download**, unzip, and copy the nine `.csv` files into `data\raw\` inside the repository (create the folder if it does not exist):<br>`mkdir data\raw -Force` |
| **Purpose** | Avoids Kaggle API credentials completely. `data/raw/` is git-ignored, so the files will never be committed. |

| | |
|---|---|
| **Expected result** | Nine files with exactly these names are in `data\raw`. |
| **Verification** | `Get-ChildItem data\raw \| Select-Object Name, Length`<br>Compare with the table below. |
| **Troubleshooting** | File names must match exactly (`olist_customers_dataset.csv`, …, `product_category_name_translation.csv`). A missing or renamed file makes the loader try to download and fail without credentials. |
| **Next step** | II.9 `.env`. |

| File in `data\raw` | Bytes (owner's copy) | Staging table | Rows loaded |
|---|---|---|---|
| `olist_customers_dataset.csv` | 9,033,957 | `stg.customers` | 99,441 |
| `olist_geolocation_dataset.csv` | 61,273,883 | `stg.geolocation` | 1,000,163 |
| `olist_order_items_dataset.csv` | 15,438,671 | `stg.order_items` | 112,650 |
| `olist_order_payments_dataset.csv` | 5,777,138 | `stg.order_payments` | 103,886 |
| `olist_order_reviews_dataset.csv` | 14,451,670 | `stg.order_reviews` | 99,224 |
| `olist_orders_dataset.csv` | 17,654,914 | `stg.orders` | 99,441 |
| `olist_products_dataset.csv` | 2,379,446 | `stg.products` | 32,951 |
| `olist_sellers_dataset.csv` | 174,703 | `stg.sellers` | 3,095 |
| `product_category_name_translation.csv` | 2,613 | `stg.product_category_translation` | 71 |

If your byte sizes differ, you may have a newer or damaged copy of the dataset; the row counts after `make pipeline` (III.2, step 7) are the definitive check.

## II.9 The .env settings file

| **STEP 9.1 · Create `.env` from the template** | |
|---|---|
| **Where** | PowerShell in the repository root, then Notepad or any editor. |
| **Action** | `Copy-Item .env.example .env`<br>`notepad .env`<br>The file has five lines:<br>`DB_HOST=localhost`<br>`DB_PORT=5432`<br>`DB_NAME=olist`<br>`DB_USER=analyst`<br>`DB_PASSWORD=<choose-a-password>`<br>Replace the password. **Use only letters, digits and underscores.** The Python code builds a connection URL from these values, so characters such as `@ : / # %` break it. |
| **Purpose** | One file configures both Docker (it creates the database user and password) and Python (it connects with them). Power BI will ask for the same user and password separately, and uses port 5432 on `localhost`. |

| | |
|---|---|
| **Expected result** | A saved `.env` file next to `Makefile`. |
| **Verification** | `git status --short`<br>`.env` must **not** appear in the list (it is git-ignored). Never run `type .env` or screenshot it while sharing your screen. |
| **Troubleshooting** | • Keep `DB_PORT=5432` unless another PostgreSQL already uses that port. If you change it, the Power BI server box must use the same port (`localhost:<port>`).<br>• Docker uses `DB_PASSWORD` **only the first time it creates the volume.** Changing the password later does not change the stored one and produces "password authentication failed". See III.7 for the safe fix.<br>• Do not commit `.env` and do not paste it anywhere. |
| **Next step** | II.10 Start PostgreSQL. |

## II.10 PostgreSQL in Docker

`docker-compose.yml` defines one service. Its important settings, all File-verified:

| Setting | Value | Meaning |
|---|---|---|
| Image | `postgres:16` | Official PostgreSQL 16 |
| Container name | `olist_postgres` | The name you use in `docker exec` / `docker logs` |
| Environment | `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB` from `.env` | Creates the role and the database `olist` on first start. The role `analyst` is the superuser of this private database, so no extra permissions are needed. |
| `POSTGRES_INITDB_ARGS` | `--auth-host=md5` | Makes the database accept **md5** password authentication for network connections instead of the default scram-sha-256. Power BI Desktop's PostgreSQL driver rejected valid credentials under scram-sha-256 during the original build. It applies only when the volume is created. |
| Port | `${DB_PORT}:5432` | Windows port `DB_PORT` forwards to PostgreSQL's 5432 inside the container |
| Volume | `olist_pg_data` → `/var/lib/postgresql/data` | The database files. Docker prefixes the Compose project name, so the real name is probably `code_olist_pg_data` for the owner's folder; check with `docker volume ls`. |
| Healthcheck | `pg_isready -U ${DB_USER} -d ${DB_NAME}` every 5 s, up to 10 retries | `docker compose up --wait` returns only when this passes |

| **STEP 10.1 · Start the database** | |
|---|---|
| **Where** | PowerShell, repository root. Docker Desktop must be running. |
| **Action** | `docker compose up -d --wait`<br>(`make db-up` runs the same command.) The first run downloads the `postgres:16` image. |
| **Purpose** | Creates the container and the data volume, initialises the database and waits until it is healthy. |

| | |
|---|---|
| **Expected result** | Lines such as `Container olist_postgres  Started` and `Container olist_postgres  Healthy`, then the prompt returns. |
| **Verification** | `docker compose ps`<br>`docker exec olist_postgres pg_isready -U analyst -d olist`<br>`.\.venv\Scripts\python.exe -c "from src.utils import get_engine; from sqlalchemy import text; c=get_engine().connect(); print(c.execute(text('show server_version')).scalar())"`<br>`docker compose ps` shows `healthy`; `pg_isready` prints `accepting connections`; Python prints a `16.x` version. The last command is an independent check that your `.env` credentials work from Python. It prints no secrets. |
| **Troubleshooting** | • `port is already allocated`: another program uses 5432. Change `DB_PORT` in `.env` (for example 5433), run `docker compose down`, then start again, and use that port in Power BI.<br>• `unhealthy`: run `docker logs olist_postgres` and read the last lines.<br>• `password authentication failed`: `.env` password differs from the one the volume was created with (III.7).<br>• Python cannot connect but `pg_isready` works: check `DB_HOST/DB_PORT` in `.env`. |
| **Next step** | II.11 (optional), then II.12 Power BI. |

VERIFICATION STATUS

**Executed / Not verified.** The same SQL, Python connection code and healthcheck logic were executed against a local PostgreSQL 16.2 server on Linux. The Docker container itself, the md5 setting inside it and the volume name were not run. In a separate test cluster, `initdb --auth-host=md5` was confirmed to set `password_encryption` to md5.

DESTRUCTIVE COMMAND

**`docker compose down -v` and `make clean` delete the `olist_pg_data` volume.** The whole database is lost and must be rebuilt (III.6). `docker compose down` **without** `-v` is safe: it only removes the container and keeps the volume. Never add `-v` unless you mean to start from nothing.

## II.11 Excel and other utilities

| Tool | Needed? | Why / how |
|---|---|---|
| Microsoft Excel (or LibreOffice Calc, Google Sheets) | Optional | Only to **open** `dashboard/exec_summary.xlsx`. The file is **generated** by `openpyxl`, which `pip` already installed; Excel is not needed to produce it. |
| Jupyter | Installed by `requirements.txt` | Notebooks are executed by `nbconvert`. To explore interactively: `.\.venv\Scripts\python.exe -m jupyter lab`. |
| A SQL client (optional) | No | Everything can be inspected with `docker exec -it olist_postgres psql -U analyst -d olist`. A graphical client such as DBeaver or pgAdmin also works against `localhost:5432`. |
| A code editor (optional) | No | Notepad is enough; Visual Studio Code makes reading SQL and Python easier. |
| Linters | None configured | The repository has no linter configuration. A compile check is available and safe: `.\.venv\Scripts\python.exe -m compileall -q src tests`. |

## II.12 Power BI Desktop and the PostgreSQL connector

| STEP 12.1 · Install Power BI Desktop | |
|---|---|
| **Where** | Official sources only: the Microsoft Store (`https://aka.ms/pbidesktopstore`) or the Microsoft Download Center (`https://www.microsoft.com/download/details.aspx?id=58494`, the 64-bit installer). |
| **Action** | Choose **one** source and install. Do not install both, as Microsoft warns the two editions conflict. Windows only; Power BI Desktop does not run on macOS or Linux. |
| **Purpose** | Opens and edits the final `.pbix` and lets you build the report from scratch. |
| **Expected result** | Power BI Desktop starts and shows the start screen. |
| **Verification** | Start the app, choose **Get data → More... → PostgreSQL database**. The connector is built in (it bundles the Npgsql driver), so **no separate driver install is required**. |
| **Troubleshooting** | • Missing WebView2: install the Microsoft Edge WebView2 Runtime when the installer requests it.<br>• Cramped layout: report pages here are 1280 px wide; the canvas scales (Fit to page), but a 1440 × 900 screen or larger is easier to work on. |
| **Next step** | II.13 Final checklist. |

VERIFICATION STATUS

**Reviewed.** The install sources and requirements come from Microsoft's Power BI Desktop and Power Query PostgreSQL-connector documentation. A copy of the final `.pbix` was opened in Power BI Desktop to view its four pages, but no refresh against the Docker database was performed. The Power BI instructions in Part V are checked against the final `.pbix` file, that viewing session, the screenshots and the QA report.

## II.13 Final setup checklist

| # | Check | Command or action | Expected |
|---|---|---|---|
| 1 | PowerShell tab open in the repo root | `Get-Location` | Path ends with the repository folder |
| 2 | Git | `git --version` | `git version 2.x` |
| 3 | Python | `python --version` | `Python 3.12.x` |
| 4 | Virtual environment | `.\.venv\Scripts\python.exe --version` | `Python 3.12.x` |
| 5 | Packages consistent | `.\.venv\Scripts\python.exe -m pip check` | `No broken requirements found.` |
| 6 | Docker engine running | `docker version` shows both Client and Server | Server section present |
| 7 | Make (or no-make route chosen) | `make --version` | `GNU Make 4.x`, or skip |
| 8 | Source CSVs present | `(Get-ChildItem data\raw -Filter *.csv).Count` | `9` |
| 9 | `.env` exists and is ignored | `Test-Path .env` and `git status --short` | `True`; `.env` not listed |
| 10 | Database healthy | `docker compose ps` | `healthy` |
| 11 | Python can connect | The one-line connection test in STEP 10.1 | Prints `16.x` |
| 12 | Power BI Desktop starts | Open the app | Start screen; PostgreSQL connector listed |

When all twelve pass, continue to Part III. Nothing has been loaded into the database yet: it is empty.

# Part III — Run the entire project in order

Part II left you with tools but an empty database. This part runs the project from the first command to the last, explains what each command reads and writes, and then gives short procedures for the situations you will meet repeatedly: coming back after a restart, rebuilding the database, re-running one notebook, and so on.

## III.1 Before you run anything

### The dependency chain

Each stage needs the one before it. If you skip a stage the next one fails or, worse, uses stale data.

```text
Docker engine -> PostgreSQL container (healthy)
  -> data/raw CSVs -> stg.* (ingest) -> input checks
    -> analytics.* tables (transform) -> output checks
      -> 18 queries (analysis)    -> sql/03_analysis/outputs/*.csv
      -> notebooks 01-03          -> executed .ipynb files
      -> notebook 04 (forecast)   -> analytics.forecast_28d + data/processed/forecast_28d.csv
      -> dashboard views          -> analytics.vw_*   (needed by Excel and Power BI)
        -> Excel summary          -> dashboard/exec_summary.xlsx
        -> Power BI model         -> the .pbix
```

### What each command reads, writes and destroys

| Command | Reads | Writes | Destroys / overwrites | Safe to repeat? |
|---|---|---|---|---|
| `docker compose up -d --wait` | `docker-compose.yml`, `.env` | Container, volume (first time) | Nothing | Yes |
| `docker compose down` | – | – | Removes the container only; **keeps** the data volume | Yes |
| `make pipeline` | `data/raw/*.csv` | `stg.*` (all 9 tables), all `analytics` tables | Replaces every `stg` and `analytics` table. **Silently drops the four `analytics.vw_*` views** (cascade). Leaves `analytics.forecast_28d` untouched. | Yes (idempotent). **Run `make views` afterwards.** |
| `make test` | `stg.*`, `analytics.*` | Nothing | Nothing (SELECT only) | Yes, any time |
| `make analysis` | `analytics.*` | `sql/03_analysis/outputs/*.csv` (first 200 rows each) | Overwrites those CSVs | Yes |
| `make notebooks` | `analytics.*` | The four `.ipynb` files; **notebook 04** also writes `analytics.forecast_28d` and `data/processed/forecast_28d.csv` | Overwrites notebooks (their stored outputs), the forecast table (dropped and recreated) and the CSV | Yes, but results can shift slightly (IV.12) |
| `make views` | `analytics.*` | `analytics.vw_sales`, `vw_orders`, `vw_customers`, `vw_daily_orders` | Drops and recreates those four views | Yes |
| `make excel` | `analytics.vw_sales` | `dashboard/exec_summary.xlsx` | Overwrites the workbook. **Fails if the views are missing.** | Yes |

> **DESTRUCTIVE COMMAND**
>
> **`make clean`: do not run it on Windows.** It executes `docker compose down -v` (deletes the database volume and every table), then `rm -rf .pytest_cache` and a `find … -exec rm -rf` command. Those last two are Unix commands that are untested on Windows. Use `docker compose down` to stop the database and delete `.pytest_cache` manually if you care. To deliberately start from an empty database, follow III.6.

DESTRUCTIVE COMMAND

**`make fixtures` overwrites your real CSVs.** It writes 9 tiny synthetic files (100 orders) into `data/raw/`, replacing the real Olist files of the same names. If you then run `make pipeline`, the database is filled with fake data. It exists for continuous integration. If you want to experiment with it, use a separate copy of the repository and a separate database (III.8.6).

CAUTION

**Changing the database password later.** The password in `.env` only takes effect when the Docker volume is created. If you edit it afterwards, Python and Power BI will fail with "password authentication failed" until you either revert the edit or follow III.7.

## III.2 The 18-step playbook

This is the complete, ordered route from verified setup to a saved, inspected report. Steps 1–4 are checks; steps 5–8 are what `make pipeline` does in one go, shown separately here so each stage can be seen and verified; steps 15–18 move into Power BI (detailed in Part V). All commands run from the repository root in PowerShell. Where a Make target exists it is shown beside the direct command.

| PLAYBOOK STEP 1 · Verify prerequisites and configuration | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | Run the twelve checks in II.13. At minimum:<br>`.\.venv\Scripts\python.exe --version`<br>`docker version`<br>`Test-Path .env` |
| **Needs** | Everything in Part II. |
| **Produces** | Nothing; confirms readiness. |
| **Success** | Python 3.12.x; Docker prints Client and Server sections; `True`. |
| **If it fails** | Return to the failing section of Part II or to Part VII. |

| PLAYBOOK STEP 2 · Start and validate Docker | |
|---|---|
| **Where** | Windows Start menu, then PowerShell. |
| **Command** | Start **Docker Desktop** and wait for "Engine running", then:<br>`docker info --format "{{.ServerVersion}}"` |
| **Needs** | Docker Desktop installed; virtualisation enabled. |
| **Produces** | A running Docker engine. |
| **Success** | A version number prints (no connection error). |
| **If it fails** | "error during connect": Docker Desktop is still starting or not running; wait and retry. See II.6. |

| PLAYBOOK STEP 3 · Start PostgreSQL and verify readiness | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `docker compose up -d --wait  # or: make db-up`<br>`docker compose ps`<br>`docker exec olist_postgres pg_isready -U analyst -d olist` |
| **Needs** | Step 2; a valid `.env`. |
| **Produces** | Container `olist_postgres`, volume `olist_pg_data`, empty database `olist`. |
| **Success** | `Healthy`; `accepting connections`. |
| **If it fails** | Port clash, authentication or unhealthy container: II.10 and Part VII.2. |

| PLAYBOOK STEP 4 · Obtain and validate the source data | |
|---|---|
| **Where** | PowerShell and File Explorer. |
| **Command** | Either place the nine CSVs manually (STEP 8.2) or rely on the Kaggle download that step 5 triggers (STEP 8.1). Then:<br>`(Get-ChildItem data\raw -Filter *.csv).Count` |
| **Needs** | Kaggle credentials (Option A) or the manual download (Option B). |
| **Produces** | Nine CSVs in `data/raw/` (121 MB in total). |
| **Success** | `9`, with the file sizes from II.8. |
| **If it fails** | Missing/renamed file: re-extract the zip. Kaggle 401/403: regenerate the token or use Option B. |

| PLAYBOOK STEP 5 · Ingest the CSVs into staging | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m src.ingest`<br>`make pipeline` runs this stage as its first action. Staging tables are replaced each time (`if_exists="replace"`). |
| **Needs** | Steps 3 and 4. Reads `data/raw/*.csv` and `.env`. |
| **Produces** | Schema `stg` with nine **all-TEXT** tables, a verbatim mirror of the CSVs. Nothing is typed or cleaned here. |
| **Success** | Nine `Loaded stg.<table>: N rows` lines (99,441 / 1,000,163 / 112,650 / 103,886 / 99,224 / 99,441 / 32,951 / 3,095 / 71), then `Ingest complete`. The log may start with `All 9 source CSVs already present … skipping download`. |
| **If it fails** | `RuntimeError: Kaggle download … failed`: see II.8. `password authentication failed`: III.7. `connection refused`: step 3 is not healthy. |

| PLAYBOOK STEP 6 · Validate staging (input data quality) | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m pytest -q tests/test_input_quality.py` |
| **Needs** | Step 5. The tests call `run_input_checks`, which only runs SELECT statements. |
| **Produces** | Nothing in the database. The full check report (including logged anomalies) is printed by `make pipeline`. |
| **Success** | `6 passed`. |
| **If it fails** | A blocking failure names the table and rule (empty table, duplicate key, orphan key, invalid domain value). Re-download the data. Never "fix" it by editing staging by hand. |

| PLAYBOOK STEP 7 · Run the SQL transformations | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m src.transform`<br>This executes `sql/01_schema/*.sql` then `sql/02_transform/*.sql` in filename order, one transaction per file. (`make pipeline` instead runs steps 5, 6, 7 and 8 together and **stops** at the first blocking failure.) |
| **Needs** | Steps 5–6 passed. |
| **Produces** | Schema `analytics` with `dim_date`, `dim_customers`, `dim_products`, `dim_sellers`, `fact_orders`, `fact_order_items`, `order_payments`, `order_reviews`, `geolocation`. |
| **Success** | Ten `Executed <file>.sql (n statements)` lines and `Transform complete`. |
| **If it fails** | A SQL error names the file. Common cause: staging is empty or incomplete; rerun step 5. Do not edit SQL to silence an error; see Part VII.3. |

| PLAYBOOK STEP 8 · Populate and validate the analytics schema | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m pytest -q tests/test_output_quality.py`<br>Then compare counts:<br>`docker exec olist_postgres psql -U analyst -d olist -c "SELECT 'dim_customers' t, count(*) FROM analytics.dim_customers UNION ALL SELECT 'dim_products', count(*) FROM analytics.dim_products UNION ALL SELECT 'dim_sellers', count(*) FROM analytics.dim_sellers UNION ALL SELECT 'fact_orders', count(*) FROM analytics.fact_orders UNION ALL SELECT 'fact_order_items', count(*) FROM analytics.fact_order_items UNION ALL SELECT 'order_payments', count(*) FROM analytics.order_payments UNION ALL SELECT 'order_reviews', count(*) FROM analytics.order_reviews UNION ALL SELECT 'geolocation', count(*) FROM analytics.geolocation UNION ALL SELECT 'dim_date', count(*) FROM analytics.dim_date;"` |
| **Needs** | Step 7. |
| **Produces** | Nothing new. Confirms row reconciliation to staging, no orphan keys, and an exact revenue tie-out. |
| **Success** | `4 passed`, and these counts: dim_customers 99,441 · dim_products 32,951 · dim_sellers 3,095 · fact_orders 99,441 · fact_order_items 112,650 · order_payments 103,886 · order_reviews 98,673 · geolocation 19,015 · dim_date 1,096. |
| **If it fails** | A count that differs from staging points to a join or filter problem in a transform file. Compare with the staging counts from step 5. |

NOTE

**Shortcut.** `make pipeline` (or `.\.venv\Scripts\python.exe -m src.run_pipeline`) runs steps 5–8 as one command. It printed `Input validation: PASSED (0 blocking failures, 9 logged anomalies)`, ten transform lines, `Output validation: PASSED (0 blocking failures, 1 logged anomalies)` and `Pipeline complete` in 33.7 seconds on the test machine. The `WARNING … LOGGED [...]` lines are **expected**: they record known data oddities (for example 1,359 orders handed to the carrier before approval, 547 orders with several reviews). They are not failures.

| PLAYBOOK STEP 9 · Run the SQL business analyses | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m src.run_analysis  # or: make analysis` |
| **Needs** | Step 8. Reads every `sql/03_analysis/*.sql`. |
| **Produces** | One CSV per query in `sql/03_analysis/outputs/` (18 files; at most the first 200 rows of each). |
| **Success** | 18 log lines, one per query, e.g. `01_monthly_revenue_mom_growth: 23 rows`, `03_category_pareto: 74 rows`, `14_rfm_customer_scores: 93358 rows (saved first 200)`. |
| **If it fails** | The failing query is named. Check that step 8 produced all tables. |

| PLAYBOOK STEP 10 · Run notebooks 01 and 02 (EDA, cohorts and RFM) | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m jupyter nbconvert --to notebook --execute --inplace notebooks/01_eda.ipynb`<br>`.\.venv\Scripts\python.exe -m jupyter nbconvert --to notebook --execute --inplace notebooks/02_retention_rfm.ipynb` |
| **Needs** | Step 8. They read `analytics.*` and import `src.utils`. |
| **Produces** | Notebooks rewritten with fresh outputs. No database changes. |
| **Success** | `[NbConvertApp] Writing … bytes to notebooks/….ipynb`, exit code 0; each took 4–6 seconds on the test machine. |
| **If it fails** | `No module named src`: run from the repository root. `Kernel` errors: step 1's pip check. See Part VII.5. |

| PLAYBOOK STEP 11 · Run notebooks 03 and 04 (statistical test and forecast) | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m jupyter nbconvert --to notebook --execute --inplace notebooks/03_delivery_reviews.ipynb`<br>`.\.venv\Scripts\python.exe -m jupyter nbconvert --to notebook --execute --inplace notebooks/04_demand_forecast.ipynb` |
| **Needs** | Step 8. Notebook 04 **creates `analytics.forecast_28d`** and `data/processed/forecast_28d.csv`. |
| **Produces** | Mann–Whitney result; backtest table; the 28-day forecast. |
| **Success** | Notebook 03: `n_late=7661, n_on_time=88163`, U = 150706850.5, and a printed p-value of `0.000e+00` (a floating-point underflow, reported as **p < 0.001**). Notebook 04: seasonal naive mean WAPE 0.2308, Holt–Winters 0.2310, verdict seasonal_naive; forecast total 6,660 for 2018-08-23 to 2018-09-19. Check: `SELECT count(*), round(sum(forecast)) FROM analytics.forecast_28d;` gives 28 and 6660. |
| **If it fails** | Notebook 04 asserts there are no missing days in the stable window; if that assertion fails, the raw data differs from the dataset used here. See IV.9 for why fold-level numbers can differ in the third decimal. |

| PLAYBOOK STEP 12 · Run tests, compilation and (optional) linting | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m pytest -q  # or: make test`<br>`.\.venv\Scripts\python.exe -m compileall -q src tests` |
| **Needs** | Step 8. |
| **Produces** | Nothing in the database. `compileall` writes `__pycache__` folders (git-ignored). |
| **Success** | `10 passed` (6 input, 4 output). `compileall` prints nothing. |
| **If it fails** | See Part VII.6. The repository configures **no linter**; none is required. |

| PLAYBOOK STEP 13 · Build the Power BI feeding views | |
|---|---|
| **Where** | PowerShell, repository root. |
| **Command** | `.\.venv\Scripts\python.exe -m src.build_dashboard_views  # or: make views` |
| **Needs** | Step 8. **Must be rerun after every `make pipeline`.** |
| **Produces** | `analytics.vw_sales` (110,197 rows), `vw_orders` (96,478), `vw_customers` (93,358), `vw_daily_orders` (634). |
| **Success** | `Executed 0x_vw_….sql` lines and `Dashboard views built`. |
| **If it fails** | Relation does not exist: a base table is missing; rerun steps 7–8. |

| PLAYBOOK STEP 14 · Generate and inspect the Excel summary | |
|---|---|
| **Where** | PowerShell, then Excel. |
| **Command** | `.\.venv\Scripts\python.exe -m src.build_excel_summary  # or: make excel` |
| **Needs** | Step 13 (reads `analytics.vw_sales`). |
| **Produces** | `dashboard/exec_summary.xlsx` with sheets **Raw Extract**, **Category x Month Pivot**, **State Pivot**. |
| **Success** | Log: 110,197 extract rows (sheet capped at 5000), 74 categories x 23 months, 27 states. The bottom-right cell of the Category x Month Pivot sheet (grand total) is 15,419,773.75. |
| **If it fails** | `relation "analytics.vw_sales" does not exist`: run step 13. Remember the **Raw Extract sheet is only the first 5,000 rows of an unordered query**, so its rows can differ between runs; the two pivot sheets are exact. |

| PLAYBOOK STEP 15 · Connect Power BI and configure the model | |
|---|---|
| **Where** | Power BI Desktop. Part V.A–V.D. |
| **Command** | Open the final `.pbix`, or create a new report and import the nine objects with the SET-prefixed native queries listed in V.B. Then set the three relationships, mark `dim_date` as the date table and create the calculated columns and the calculated table (V.D–V.E). |
| **Needs** | Steps 3, 11 and 13 complete. Database running. |
| **Produces** | The data model with 11 tables. |
| **Success** | Row counts in V.B.5 match; three relationships in Model view. |
| **If it fails** | Error 53100, authentication, or an empty table: V.B and Part VII.7. |

| PLAYBOOK STEP 16 · Build or recreate the report pages | |
|---|---|
| **Where** | Power BI Desktop. Part V.E–V.I. |
| **Command** | Create the 43 measures in `_Measures` (V.E), then follow the visual recipes page by page (V.F). |
| **Needs** | Step 15. |
| **Produces** | Four pages: Executive Overview, Sales Deep-Dive, Customers & Retention, Operations & Satisfaction. |
| **Success** | Each page matches the repository screenshot and the layout blueprint. |
| **If it fails** | Use the per-visual "Common errors" in V.F. |

| PLAYBOOK STEP 17 · Validate metrics, slicers and visuals | |
|---|---|
| **Where** | Power BI Desktop. Part V.G and V.J. |
| **Command** | With no slicer applied compare every KPI card with the reference values in V.J. Then test each slicer (V.G). |
| **Needs** | Step 16. |
| **Produces** | A signed-off report. |
| **Success** | Revenue R$15,419,773.75 · orders 96,478 · AOV R$159.83 · latest-month MoM −4.1% · customers 93,358 · repeat 3.00% · late 8.11% · median delivery 10 days · forecast total 6,660. |
| **If it fails** | V.J.3 lists the commonest "looks right but is wrong" defects. |

| PLAYBOOK STEP 18 · Save, reopen and inspect the final report | |
|---|---|
| **Where** | Power BI Desktop and File Explorer. |
| **Command** | **File → Save as** a **new** file name for a rebuilt report (never overwrite the authoritative `.pbix`). Close Power BI, reopen the saved file from disk, and click through all four pages. |
| **Needs** | Step 17. |
| **Produces** | A saved `.pbix`. |
| **Success** | Reopens without errors; values still correct; slicers reset to "all". |
| **If it fails** | Locked file: close other copies of Power BI or the preview in Explorer, then Save As again. |

VERIFICATION STATUS

**Executed.** Steps 3–14 were executed in a Linux test environment (PostgreSQL 16.2, Python 3.12.3) with the pinned requirements and the real dataset: ingest, input tests (6 passed), transform, output tests (4 passed), analysis, notebooks 01–04 (all exit 0), views, tests (10 passed) and compileall. The step-by-step route (steps 5–8 separately, in a separate database) and the all-in-one `src.run_pipeline` both succeeded. Docker, Make, Windows paths, the Kaggle download, Excel summary inspection in Excel and steps 15–18 (Power BI Desktop) were not executed.

## III.3 First-time setup procedure

1. Complete **Part II** (tools, repository, `.venv`, Docker, `.env`, data files).
2. Run playbook steps 3 → 14 in order (or `make db-up`, `make pipeline`, `make test`, `make analysis`, `make notebooks`, `make views`, `make excel`).
3. Open the final `.pbix` and refresh (III.8.4), or rebuild it (Part V).
4. Confirm the reference values in V.J.

TIP

**Make route in one block.** `make db-up` → `make pipeline` → `make test` → `make analysis` → `make notebooks` → `make views` → `make excel`. Always `make views` after `make pipeline`, otherwise `make excel` and the Power BI refresh fail.

## III.4 Routine run after setup

When nothing about the code or data has changed and you simply want everything running again:

1. Start Docker Desktop and wait for "Engine running".
2. `docker compose up -d --wait`
3. Check the data is still there: `docker exec olist_postgres psql -U analyst -d olist -c "SELECT count(*) FROM analytics.vw_sales;"` returns 110197.
4. Open the `.pbix`; use **Refresh** if you want to reload.

You do **not** need to rerun the pipeline: the data lives in the Docker volume, not in the container.

## III.5 Restarting after shutting down Windows

| Situation | What happened to the database | What to do |
|---|---|---|
| Windows shut down or restarted; Docker Desktop was running | The container stopped; the volume is intact | Start Docker Desktop, then `docker compose up -d --wait`. Data is unchanged. |
| Docker Desktop set to start at login | The container may already be running (Compose does not set a restart policy, so it may not) | `docker compose ps`; if absent or exited, run `docker compose up -d --wait`. |
| You ran `docker compose down` (no `-v`) | Container removed, volume kept | `docker compose up -d --wait`; the existing data reappears. |
| You ran `docker compose down -v` or `make clean` | Volume deleted: database empty | Rebuild with III.6. |
| Power BI opens but shows old numbers | Imported data is a stored snapshot | Start the container before **Refresh**; the snapshot itself does not need the database. |

To prove the data survived: `docker exec olist_postgres psql -U analyst -d olist -Atc "SELECT count(*) FROM analytics.fact_orders;"` should print `99441`.

## III.6 Rebuild the database from scratch (destructive)

DESTRUCTIVE COMMAND

**This deletes the whole database.** Everything in the `olist_pg_data` volume is destroyed, including `analytics.forecast_28d` and the four views. The raw CSVs in `data/raw/` and your Git files are **not** touched. Do this only when the database is corrupted or you want a clean-room test. If you only need fresh tables, just rerun `make pipeline`; it is already idempotent and much safer.

1. Stop everything that uses the database (close Power BI's Refresh, notebooks).
2. `docker compose down -v` (removes the container **and** the volume).
3. `docker volume ls` and confirm no `…olist_pg_data` volume remains.
4. `docker compose up -d --wait` (creates a new empty database using the current `.env`).
5. `make pipeline`, then `make test`, then `make views`.
6. `make notebooks` (recreates `analytics.forecast_28d`), then `make analysis` and `make excel`.
7. In Power BI: **Refresh**. If credentials are requested, enter the same user and password as `.env`.

Expect about 35 seconds for the pipeline itself on a modern machine (33.7 s measured on Linux; the playbook's estimate of 3–8 minutes for Windows with Docker is unverified).

## III.7 Fix a password or authentication mismatch

Symptom: `password authentication failed for user "analyst"` from Python or Power BI. Cause: the password in `.env` differs from the one stored in the volume when it was first created. Two safe options:

- **Keep the data (preferred).** Put the original password back in `.env`. If you have forgotten it, set a new one inside the database: `docker exec olist_postgres psql -U analyst -d olist -c "ALTER USER analyst PASSWORD '<new-password>';"`, then use the same new password in `.env` and in Power BI. This connects through the container's local trust, not over the network.
- **Start over.** Follow III.6; the new volume takes the password currently in `.env`.

## III.8 Individual procedures

### III.8.1 Rerun one notebook

Prerequisite: the analytics tables exist (steps 5–8). Replace the file name as needed:

```
POWERSHELL
.\.venv\Scripts\python.exe -m jupyter nbconvert --to notebook --execute --inplace
      notebooks/03_delivery_reviews.ipynb
```

The file is rewritten in place. If a cell fails, execution stops at that cell and reports the error. To keep the committed notebooks exactly as published, run on a copy (`Copy-Item notebooks\03_delivery_reviews.ipynb $env:TEMP\`) with `--output-dir`, or afterwards discard the change with `git restore notebooks`. To look at a notebook interactively, run `.\.venv\Scripts\python.exe -m jupyter lab`.

### III.8.2 Rerun one SQL analysis

Run all 18 with `make analysis`. To run one query by hand, pipe the file into `psql` inside the container:

```
POWERSHELL
Get-Content sql\03_analysis\13_repeat_purchase_rate.sql -Raw | docker exec -i
      olist_postgres psql -U analyst -d olist
```

Expected: `total_customers 93358, repeat_customers 2801, repeat_pct 3.00`. Queries are read-only SELECTs, so experimenting is safe.

### III.8.3 Rebuild the SQL views

`make views` (or `.\.venv\Scripts\python.exe -m src.build_dashboard_views`). Each file is `DROP VIEW IF EXISTS … ; CREATE VIEW …`, so it is safe to repeat. Check with:

```
POWERSHELL
docker exec olist_postgres psql -U analyst -d olist -c "\dv analytics.*"
```

Four views must be listed. Rebuild them any time `make pipeline` ran, because the pipeline's `DROP TABLE … CASCADE` removes them.

### III.8.4 Regenerate Excel

Run `make views` first if you have run the pipeline, then `make excel`. Close `exec_summary.xlsx` in Excel before running (an open file is locked on Windows and the write fails).

### III.8.5 Refresh imported Power BI data

1. Start the database and verify the four views and `forecast_28d` exist (`\dv analytics.*` and `SELECT count(*) FROM analytics.forecast_28d;` → 28).

2. Open the `.pbix`. On the **Home** ribbon choose **Refresh**.
3. If prompted, choose **Database** authentication and enter the `.env` user and password yourself. Never type them into a shared screen.
4. Wait for the refresh to finish; all nine imported tables reload. The calculated table `forecast_chart` and all measures recompute automatically.
5. Verify against V.J. Do **not** save over the authoritative file unless you intend to update it; use **Save as** for experiments.

Notes: the PostgreSQL error `53100` (could not resize shared memory segment) is prevented by the `SET max_parallel_workers_per_gather = 0;` prefix already present in the report's queries; see V.B and Part VII.7.

### III.8.6 Run tests without destroying existing data

- **Safest:** `make test` on the live database. The ten tests only execute SELECT statements and write nothing.
- **CI-style experiment with synthetic data:** use a **second copy of the repository** (not your working copy) so `data/raw` is not overwritten, and a **second database**: `docker exec olist_postgres createdb -U analyst olist_test`. In the copy, set `DB_NAME=olist_test` in its own `.env`, then `make fixtures`, `make pipeline`, `make test`. This is exactly what the GitHub Actions workflow does with fresh containers.
- **Never** run `make fixtures` in your real working copy.

> VERIFICATION STATUS
>
> **Executed.** The second-database procedure was executed: a second database was created, the repository copy generated the fixtures, ran the pipeline and passed all 10 tests, and the original database kept its 99,441 orders. `createdb` through `docker exec` was not run.

### III.8.7 Rebuild the dashboard independently of the pipeline

Rebuilding visuals needs only the database objects, not the pipeline. If the nine objects exist (V.A), go straight to Part V. If you only need to restore the report, copy the authoritative `.pbix` from Git (`git checkout -- dashboard/ecommerce_retention_dashboard_final.pbix` restores the committed version).

## III.9 The everyday workflow

| When | Do this |
|---|---|
| Start of a work session | Docker Desktop → `docker compose up -d --wait` → `docker compose ps` shows healthy |
| Explore data | `docker exec -it olist_postgres psql -U analyst -d olist`, or run a query file (III.8.2) |
| Changed SQL or Python | `make pipeline` → `make test` → **`make views`** → `make analysis` |
| Need fresh notebook outputs or forecast | `make notebooks` (then re-check the forecast total, 28 rows) |
| Need the Excel workbook | `make excel` |
| Open the dashboard | Open the `.pbix` → **Refresh** only if the database changed |
| Before committing | `git status` (no `.env`, no `data/raw`), `make test` |
| End of session | `docker compose down` (keeps data). **Never** `-v`. |

## III.10 What normal output looks like

| Command | Measured time (Linux test machine) | Key success line |
|---|---|---|
| `make pipeline` | 33.7 s | `Pipeline complete` |
| `make test` | 2.6 s | `10 passed` |
| `make analysis` | a few seconds | 18 lines `NN_name: N rows` |
| Each notebook | 4–6 s | exit code 0 |
| `make views` | about 2 s | `Dashboard views built` |

Timings on your Windows PC with Docker will differ. Treat them as a sense of scale, not a requirement.

# Part IV — SQL, Python and notebooks explained

This part explains what the code actually does, in the order the data flows through it. Read it with the repository open: every file name below is real.

## IV.1 The data model

The database has two schemas. **`stg`** is a raw mirror of the nine CSVs with every column stored as text, so nothing is lost or silently changed on the way in. **`analytics`** is a typed, keyed, deduplicated star schema built from it.

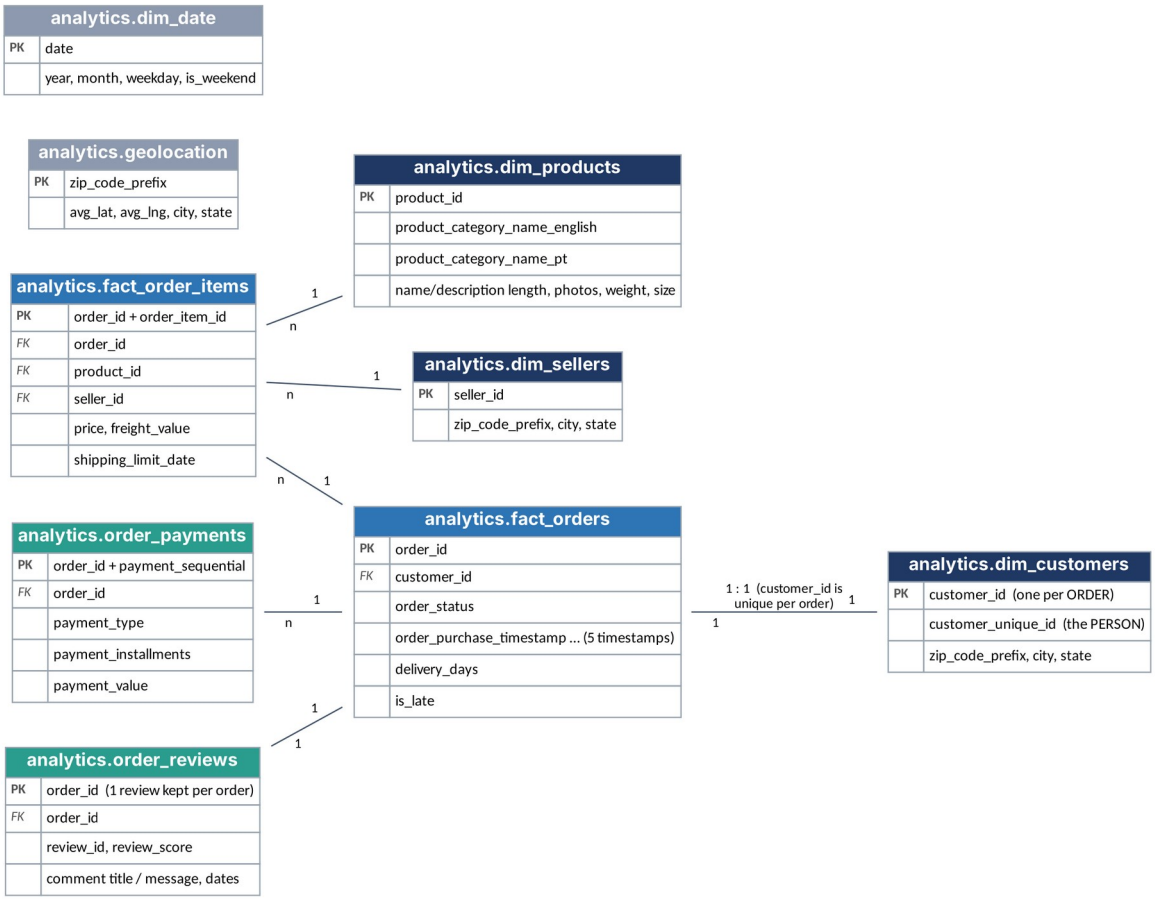



**Figure 3.** The `analytics` schema. Lines show the foreign keys declared in `sql/02_transform`. `geolocation` and `dim_date` stand alone: nothing joins to them in SQL. Power BI relates `dim_date` to the three order-date views (Part V.D).

| Table | Grain (one row per…) | Key | Rows | Important decisions |
|---|---|---|---|---|
| `dim_customers` | Order-scoped `customer_id` | PK `customer_id` | 99,441 | `customer_unique_id` is the person. One person can own many `customer_id` rows. |
| `dim_products` | Product | PK `product_id` | 32,951 | English category via the translation table; falls back to the Portuguese name when untranslated; stays NULL when the product has no category at all. |
| `dim_sellers` | Seller | PK `seller_id` | 3,095 | Straight copy with typing. |
| `fact_orders` | Order | PK `order_id`; FK `customer_id` | 99,441 | Five timestamps cast to `timestamp`; derives `delivery_days` and `is_late`. |
| `fact_order_items` | Order line | PK (`order_id`, `order_item_id`); FKs to orders, products, sellers | 112,650 | `price` and `freight_value` as `numeric`; revenue = their sum. |

| Table | Grain (one row per…) | Key | Rows | Important decisions |
|---|---|---|---|---|
| `order_payments` | Payment record | PK (`order_id`, `payment_sequential`) | 103,886 | No deduplication: several payments per order are legitimate. |
| `order_reviews` | Order (after dedup) | PK `order_id` | 98,673 | Source has 547 orders with several reviews; the latest by `review_answer_timestamp` is kept. |
| `geolocation` | Zip-code prefix | PK `zip_code_prefix` | 19,015 | Averaged latitude/longitude, most common city and state. |
| `dim_date` | Calendar day | PK `date` | 1,096 | Generated with `generate_series` from 2016-01-01 to 2018-12-31; columns year, month, weekday, is_weekend. |

## Date and time handling

- Timestamps are cast with `::timestamp` (no time zone). Nothing is converted between zones.
- `delivery_days` is `EXTRACT(DAY FROM (delivered − purchased))`: whole days only, so 9 days 23 hours counts as 9. It is NULL when the order was never delivered.
- `is_late` is `delivered_customer_date > estimated_delivery_date`, a timestamp comparison. It is NULL (not false) when there is no delivery date, because "late" does not apply to an undelivered order. This is the origin of the 8 "Unknown" orders in the dashboard and the `BLANK()/FALSE()` edge case in V.E.
- The dashboard views add `order_date` (a pure `DATE`) next to the timestamp. Power BI's date relationship must match DATE to DATE; relating to a timestamp with a time part silently matches nothing.

## How the SQL objects feed Power BI

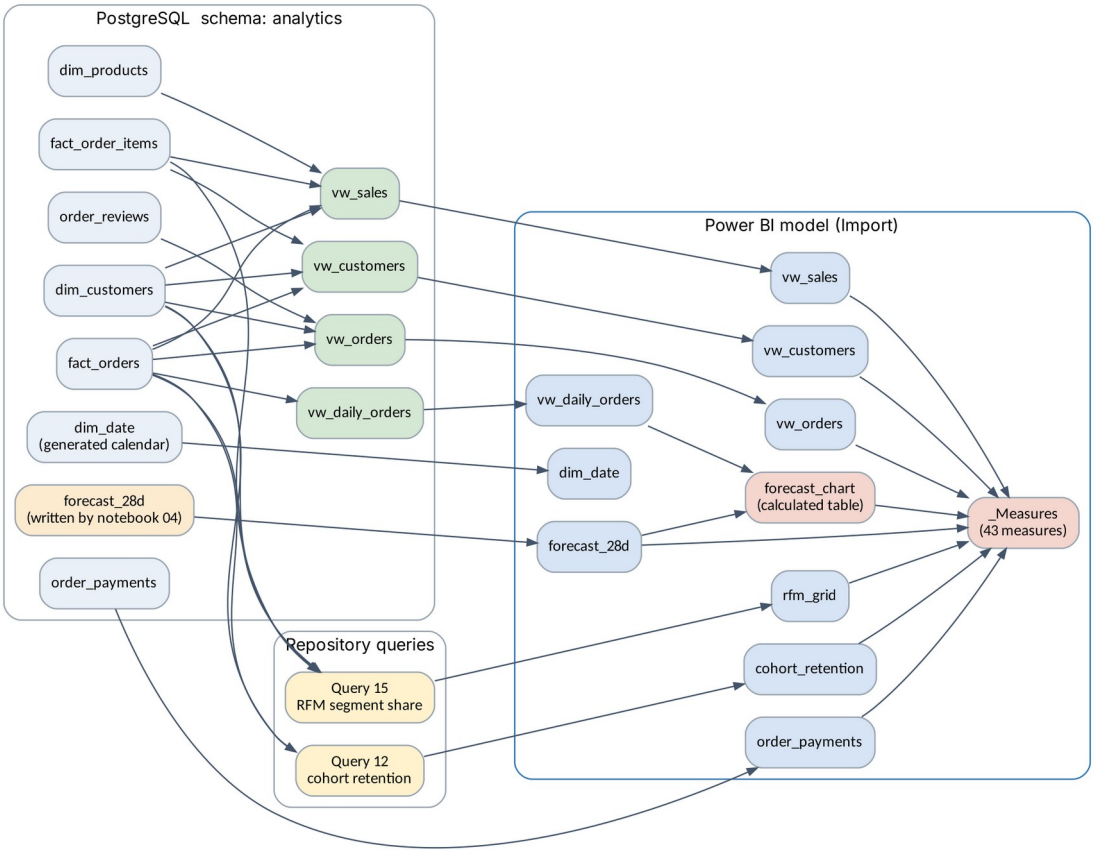



**Figure 4.** Lineage from `analytics` tables and repository queries to the Power BI tables and the `_Measures` table.

| Power BI table | Source in PostgreSQL | Rows | Used for |
|---|---|---|---|
| `vw_sales` | `analytics.vw_sales` | 110,197 | Revenue, orders, categories, states, sellers (pages 1–2) |
| `vw_orders` | `analytics.vw_orders` | 96,478 | Delivery and review metrics (page 4) |
| `vw_customers` | `analytics.vw_customers` | 93,358 | Customer, repeat, one-time counts (page 3) |
| `vw_daily_orders` | `analytics.vw_daily_orders` | 634 | Actual daily orders behind the forecast chart |
| `dim_date` | `analytics.dim_date` | 1,096 | Calendar, date slicer, month axis |
| `forecast_28d` | `analytics.forecast_28d` (written by notebook 04) | 28 | Forecast values and bounds |
| `order_payments` | `analytics.order_payments` | 103,886 | Payment mix and instalments |
| `cohort_retention` | Query 12 (native query) | 219 | Cohort heat-map |
| `rfm_grid` | Query 15 (native query) | 15 | RFM segment shares |

## IV.2 Staging and the Python entry points

Eight small modules in `src/` do the work. They all connect through the same helper and read the same `.env`. Run any module with `python -m src.<name>` from the repository root (not by double-clicking the file) so that `import src…` works.

| src/utils.py · shared helpers | |
|---|---|
| **Purpose** | Provides `get_engine()` (builds a SQLAlchemy engine for `postgresql+psycopg2` from the five `DB_*` variables, loading `.env` through python-dotenv) and `get_logger(name)` (consistent timestamped log format). |
| **Inputs** | `.env` or real environment variables. A variable already set in your shell **wins** over `.env`. |
| **Outputs** | An engine and loggers. Nothing is written. |
| **Verify** | The one-line connection test in STEP 10.1. |
| **Fails when** | `KeyError: 'DB_USER'` (a variable is missing from `.env`); a password containing `@ : / # %` corrupts the URL. |

| src/ingest.py · download and load staging | |
|---|---|
| **Purpose** | `download_dataset(dest)` skips if all nine expected CSVs exist, else runs `python -m kaggle datasets download -d olistbr/brazilian-ecommerce -p data/raw --unzip`. `load_staging(engine, raw_dir)` creates schema `stg` and loads each CSV with `pandas.read_csv(dtype=str)` and `to_sql(if_exists="replace", chunksize=10000)`. |
| **Inputs** | `data/raw/*.csv`; Kaggle credentials only if files are missing. |
| **Outputs** | Nine `stg.<table>` tables, all TEXT. Returns a dict of row counts. |
| **Run / verify** | `python -m src.ingest` → nine `Loaded stg.… rows` lines. Spot check: `SELECT count(*) FROM stg.orders;` = 99441. |
| **Fails when** | Kaggle authentication (a `RuntimeError` with no guessing of mirrors); a missing CSV; the database is not reachable. |

| src/validate.py · data-quality gates | |
|---|---|
| **Purpose** | `run_input_checks` examines `stg.*`: row counts (non-empty), primary-key uniqueness, referential integrity (6 foreign keys), domain values (review score 1–5, no negative price, freight or payment), temporal order, null rates, duplicate reviews. `run_output_checks` examines `analytics.*`: row reconciliation to staging, orphan keys, and a revenue tie-out. Each check is **blocking** (fails the pipeline) or **logged** (recorded as a known anomaly). |
| **Inputs** | `stg.*` or `analytics.*`. |
| **Outputs** | A `Report` object and log lines. Nothing is written to the database. |
| **Run / verify** | Through pytest (IV.3) or `make pipeline`. Real data: input validation passed with 0 blocking failures and 9 logged anomalies; output validation passed with 0 blocking failures and 1 logged anomaly. |
| **Fails when** | Blocking checks find empty tables, duplicate keys, orphan keys or out-of-range values; a malformed staging column produces a cast error that is converted into a blocking failure rather than a crash. |

| src/transform.py · run SQL files | |
|---|---|
| **Purpose** | `run_sql_dir(engine, path)` executes every `*.sql` file in a folder in filename order, **one transaction per file**. It strips `--` comments, splits on `;` and executes each statement. |
| **Inputs** | `sql/01_schema`, `sql/02_transform` (and, from `build_dashboard_views`, `sql/04_dashboard_views`). |
| **Outputs** | `analytics` tables or views. Log line per file: `Executed <file> (n statements)`. |
| **Limitation** | The splitter is a plain string split. It cannot cope with a semicolon or `--` inside a string literal, or with `$$` function bodies. All current SQL is safe; add a real SQL parser before adding such SQL. |
| **Fails when** | Staging tables are missing or have unexpected columns. |

| src/run_pipeline.py · the orchestrator | |
|---|---|
| **Purpose** | Runs: `download_dataset` → `load_staging` → `run_input_checks` (stop with exit code 1 on a blocking failure) → `run_sql_dir` ×2 → `run_output_checks` (exit 1 on failure) → "Pipeline complete". |
| **Run / verify** | `make pipeline`. Echo the exit code with `$LASTEXITCODE` in PowerShell: `0` means success. |
| **Downstream** | Everything: analysis, notebooks, views, Excel, tests, Power BI. |
| **Fails when** | Any of the above; it **does not** rebuild the dashboard views (run `make views`). |

| src/run_analysis.py · the 18 saved queries | |
|---|---|
| **Purpose** | Reads every `sql/03_analysis/*.sql`, runs it with `pandas.read_sql`, writes `outputs/<name>.csv` with the first 200 rows, and logs the full row count. |
| **Outputs** | 18 CSV files. Overwrites existing ones. |
| **Verify** | The 18 log lines in III.2 step 9. |

| src/build_dashboard_views.py · Power BI views | |
|---|---|
| **Purpose** | Runs `run_sql_dir` on `sql/04_dashboard_views`. Kept separate from the pipeline because views are a BI concern, not a quality gate. |
| **Outputs** | Four views (IV.6). |
| **Fails when** | Run before the pipeline (base tables missing). |

| src/build_excel_summary.py · Excel workbook | |
|---|---|
| **Purpose** | Reads `order_id`, category, state, `order_date`, `line_revenue` from `analytics.vw_sales`; fills missing categories with `unknown`; builds a month label; writes three sheets to `dashboard/exec_summary.xlsx` with `openpyxl`: **Raw Extract** (first 5,000 rows only), **Category x Month Pivot** (revenue, with totals) and **State Pivot** (revenue and distinct orders, sorted descending). |
| **Verify** | Pivot grand total 15,419,773.75; 74 categories, 23 months, 27 states. |
| **Known limitation** | "Raw Extract" is the first 5,000 rows of a query with no `ORDER BY`, so its rows may differ run to run. It is a sample; the pivots are exact. The script was left unchanged. |

| tests/fixtures/generate_fixtures.py · synthetic data for CI | |
|---|---|
| **Purpose** | Writes nine tiny, schema-faithful CSVs (100 orders) with a fixed random seed and three planted defects (one duplicate review, one payment mismatch, one temporal violation) so the validation suite is shown to work. |
| **Danger** | Writes to the folder you give it, overwriting same-named files. Used by CI and by `make fixtures`. See III.1. |

## IV.3 Data-quality checks and the test suite

| Check group | Layer | Type | Result on the real data |
|---|---|---|---|
| Row counts (9 staging tables non-empty) | Input | Blocking | Pass |
| Primary keys unique (orders, order_items, customers, products, sellers) | Input | Blocking | Pass |
| Referential integrity (6 foreign keys) | Input | Blocking | Pass |
| Domain values (score 1–5; price, freight, payment ≥ 0) | Input | Blocking | Pass |
| Temporal sanity (3 rules) | Input | Logged | 1,359 · 23 · 6 violations |
| Null policy (4 columns above threshold) | Input | Logged | `order_approved_at` 0.16%, `order_delivered_customer_date` 2.98%, review title 88.34%, review message 58.70% |
| Review duplicates | Input | Logged | 789 repeated `review_id`s; 547 orders with several reviews |
| Row reconciliation (analytics vs staging) | Output | Blocking | Pass (reviews 99,224 → 98,673 and geolocation 1,000,163 → 19,015 are intended reductions) |
| Orphan keys in `analytics` | Output | Blocking | Pass |
| Revenue tie-out (items in `analytics` equal items in staging) | Output | Blocking + logged | Exact tie-out (R$15,843,553.24 both sides, all statuses); 303 orders where items and payments differ |

`pytest` wraps these checks in ten tests. Every one is read-only.

| Test file | Tests | Asserts |
|---|---|---|
| `tests/test_input_quality.py` | 6 | Staging tables populated; primary keys unique; referential integrity clean; domain values valid; no blocking failures; at least five anomalies were logged |
| `tests/test_output_quality.py` | 4 | Row reconciliation; no orphan keys; revenue ties out to staging; no blocking failures |

> **NOTE**
> The input tests require that staging exists, the output tests that `analytics` exists. Running `pytest` against an empty database fails by design.

## IV.4 The transformation SQL

Every file follows the same idempotent pattern: `DROP TABLE IF EXISTS … CASCADE;` then `CREATE TABLE … AS SELECT …`, then keys and indexes. That is why `make pipeline` can always be rerun, and also why it silently removes any view built on those tables.

| File | Creates | What it does |
|---|---|---|
| `01_schema/01_create_schema.sql` | schema `analytics` | `CREATE SCHEMA IF NOT EXISTS analytics;` |
| `01_schema/02_dim_date.sql` | `dim_date` | Calendar from `generate_series('2016-01-01', '2018-12-31', '1 day')`; weekday text; `is_weekend` from ISO day of week 6–7; PK on `date` |
| `02_transform/01_dim_customers.sql` | `dim_customers` | Copy of `stg.customers`; PK `customer_id` |
| `02_transform/02_dim_products.sql` | `dim_products` | `COALESCE(english, portuguese)` category; numeric casts; **note the source misspelling `product_name_lenght`**, renamed to `product_name_length` |
| `02_transform/03_dim_sellers.sql` | `dim_sellers` | Copy; PK `seller_id` |
| `02_transform/04_fact_orders.sql` | `fact_orders` | Timestamps cast; `delivery_days`; `is_late`; FK to `dim_customers`; index on purchase timestamp |
| `02_transform/05_fact_order_items.sql` | `fact_order_items` | Casts; composite PK; three FKs; two indexes |
| `02_transform/06_order_payments.sql` | `order_payments` | Casts; composite PK; FK to orders; **no dedup** |
| `02_transform/07_order_reviews.sql` | `order_reviews` | `DISTINCT ON (order_id)` ordered by newest `review_answer_timestamp`, then `review_id` (a deterministic tie-break) |
| `02_transform/08_geolocation.sql` | `geolocation` | Group by zip prefix: `AVG(lat)`, `AVG(lng)`, `MODE()` of city and state |

```sql
-- sql/02_transform/04_fact_orders.sql (core)
CREATE TABLE analytics.fact_orders AS
SELECT order_id, customer_id, order_status,
       order_purchase_timestamp::timestamp AS order_purchase_timestamp,
       -- ... four more timestamps cast the same way ...
       EXTRACT(DAY FROM (order_delivered_customer_date::timestamp
                         - order_purchase_timestamp::timestamp))::int AS delivery_days,
       (order_delivered_customer_date::timestamp
        > order_estimated_delivery_date::timestamp) AS is_late
FROM stg.orders;
```

```sql
-- sql/02_transform/07_order_reviews.sql (core): keep ONE review per order
CREATE TABLE analytics.order_reviews AS
SELECT DISTINCT ON (order_id)
       review_id, order_id, review_score::int AS review_score, ...
FROM stg.order_reviews
ORDER BY order_id, review_answer_timestamp::timestamp DESC, review_id;
```

How to execute and verify the transformations: `python -m src.transform`, then the count query in III.2 step 8. To see a derived column working, compare an on-time and a late order directly:

```sql
SELECT order_id, order_estimated_delivery_date, order_delivered_customer_date,
       delivery_days, is_late
FROM analytics.fact_orders
WHERE order_status = 'delivered' ORDER BY is_late DESC, delivery_days DESC LIMIT 5;
```

## IV.5 The 18 business queries

Each file in `sql/03_analysis/` starts with a one-line `-- QUESTION:` comment, runs read-only and returns a small table. To run one by hand, use III.8.2. The table summarises all 18; the sections after it explain the harder ones clause by clause.

Files: `01_monthly_revenue_mom_growth.sql`, `02_aov_trend.sql`, `03_category_pareto.sql`, `04_revenue_by_state.sql`, `05_payment_mix_installments.sql`, `06_seller_concentration.sql`, `07_order_status_funnel.sql`, `08_delivery_days_distribution.sql`, `09_late_pct_by_month.sql`, `10_late_pct_by_state.sql`, `11_late_vs_review_score.sql`, `12_cohort_retention_matrix.sql`, `13_repeat_purchase_rate.sql`, `14_rfm_customer_scores.sql`, `15_rfm_segment_share.sql`, `16_items_vs_payments_reconciliation.sql`, `17_duplicate_review_count.sql`, `18_temporal_violation_count.sql`.

| # | Question | Population | Key technique | Result on the real data | Used by |
|---|---|---|---|---|---|
| 01 | Monthly revenue and month-over-month growth | Delivered | `date_trunc`, CTE, `LAG` | 23 months with sales (no November 2016); latest month Aug 2018 R$985,491.64, −4.1% (SQL prints −4.1) | Page 1 |
| 02 | Average order value by month | Delivered | Two-level aggregation (per order, then per month) | 23 rows; AOV about R$143–R$176 | Page 2 |
| 03 | Category Pareto | Delivered | `COALESCE`, window `SUM`, cumulative share | 74 rows; top 5 = 39.25% | Page 1 |
| 04 | Revenue by state | Delivered | Join to customers, window share | 27 states; SP 37.42%, RJ 13.33%, MG 11.80% | Page 1 |
| 05 | Payment mix and instalments | **All** payment rows | `GROUP BY`, `COUNT(*) OVER ()` | credit_card 73.92%, boleto 19.04%, voucher 5.56%, debit_card 1.47%, not_defined 3 rows | Page 2 |
| 06 | Seller concentration | Delivered | `ROW_NUMBER`, cumulative share, `LIMIT 10` | Top 10 = 12.93%; #1 seller 1.60% | Page 2 |
| 07 | Order-status funnel | **All** orders | `COUNT(*) OVER ()` | delivered 97.02%; shipped 1.11%; canceled 0.63% | Context |
| 08 | Delivery-time distribution | Delivered with a date | Group by `delivery_days` | 146 distinct values; mode 7 days (7,651 orders) | Page 4 |
| 09 | Late % by month | Delivered | `COUNT(*) FILTER (WHERE is_late)` | Nov 2017 14.31%; Feb 2018 15.99%; Mar 2018 21.36% | Page 4 |
| 10 | Late % by state | Delivered, states ≥ 30 orders | `HAVING`, `FILTER` | Highest AL 23.93%, MA 19.67%, PI 15.97% | Context |
| 11 | Do late orders score lower? | Reviewed delivered | `PERCENTILE_CONT`, `FILTER` | On time: 88,163 · 4.29 · median 5 · 9.22% 1–2★; Late: 7,661 · 2.57 · median 2 · 54.07% | Page 4 |
| 12 | Cohort retention (long form) | Delivered | Two CTEs, `age()`, `COUNT(DISTINCT)` | 219 cohort/offset rows; 23 cohorts | Page 3 |
| 13 | Repeat-purchase rate | Delivered | `COUNT(*) FILTER` | 93,358 customers; 2,801 repeat; 3.00% | Page 3 |
| 14 | RFM scores per customer | Delivered | `NTILE(5)`, `CASE` | 93,358 rows | Per-customer reference |
| 15 | RFM segment grid | Delivered | `NTILE`, `SUM(SUM()) OVER ()` | 15 rows (5 recency × 3 frequency) | Page 3; notebook 02 |
| 16 | Item totals vs payment totals | All orders with both | `ABS()` tolerance 0.01 | 303 mismatching orders | Finding 5 |
| 17 | Duplicate reviews before dedup | Staging | `HAVING COUNT(*) > 1` | 547 orders | Validation |
| 18 | Impossible timestamp orders | All orders | `FILTER` ×3 | 1,359 · 23 · 6 | Validation |

### Query 01 (monthly revenue and growth)

```sql
WITH monthly AS (
  SELECT date_trunc('month', o.order_purchase_timestamp)::date AS month,
         SUM(oi.price + oi.freight_value) AS revenue
  FROM analytics.fact_orders o
  JOIN analytics.fact_order_items oi USING (order_id)
  WHERE o.order_status = 'delivered'
  GROUP BY 1)
SELECT month, revenue,
  ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
        / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS mom_growth_pct
FROM monthly ORDER BY month;
```

- **Join and filter.** Order items carry the money; orders carry the status. `USING (order_id)` joins them, and `WHERE order_status = 'delivered'` applies the revenue rule.
- **`date_trunc('month', …)::date`** collapses every timestamp to the first day of its month.
- **`LAG(revenue) OVER (ORDER BY month)`** reads the previous **row**, not the previous calendar month. There is no row for November 2016, so December 2016 is compared with October 2016. The DAX measure `Revenue MoM %` instead uses the calendar prior month (V.E), which gives a blank for that month. Both are defensible; know which one you are looking at.
- **`NULLIF(…, 0)`** turns a zero denominator into NULL so the division yields NULL rather than an error.
- **Interpretation.** Revenue grows through 2017, reaches R$1,153,364.20 in November 2017 (Black Friday) and plateaus around R$1.0–1.1 million per month in 2018. The first months have tiny bases, so their percentages (e.g. +32,306.7% in October 2016) are meaningless extremes.

### Query 03 (category Pareto) and the cumulative share

```sql
SELECT category, revenue,
  ROUND(100.0 * revenue / SUM(revenue) OVER (), 2)                    AS revenue_share_pct,
  ROUND(100.0 * SUM(revenue) OVER (ORDER BY revenue DESC)
        / SUM(revenue) OVER (), 2)                                    AS
     cumulative_share_pct
FROM category_revenue ORDER BY revenue DESC;
```

`SUM(revenue) OVER ()` is the grand total repeated on every row (the denominator). Adding `ORDER BY revenue DESC` inside `OVER` makes the sum **running**: each row adds itself and everything larger. The top 5 categories (health_beauty 9.16%, watches_gifts 8.20%, bed_bath_table 7.95%, sports_leisure 7.25%, computers_accessories 6.70%) therefore reach a cumulative 39.25%. `COALESCE(…, 'unknown')` keeps products without any category in the total instead of dropping their revenue.

### Query 11 (late versus on-time reviews)

```sql
SELECT o.is_late, COUNT(*) AS reviewed_orders,
  ROUND(AVG(r.review_score), 2) AS avg_review_score,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY r.review_score) AS median_review_score,
  ROUND(100.0 * COUNT(*) FILTER (WHERE r.review_score <= 2) / COUNT(*), 2) AS pct_1_2_star
FROM analytics.fact_orders o
JOIN analytics.order_reviews r USING (order_id)
WHERE o.order_status = 'delivered' AND o.is_late IS NOT NULL
GROUP BY o.is_late ORDER BY o.is_late;
```

The inner join to reviews keeps only reviewed orders (95,824 with a known lateness flag; `AND o.is_late IS NOT NULL` excludes the unknown ones). `PERCENTILE_CONT(0.5)` is the median. `COUNT(*) FILTER (WHERE …)` is a conditional count. Because the join is inner, `review_score` is never NULL here, so `<= 2` is safe in SQL; the DAX equivalent needs care (V.E, `Pct 1-2 Star`).

### Query 12 (cohort retention)

```sql
WITH first_purchase AS (
  SELECT c.customer_unique_id,
         MIN(date_trunc('month', o.order_purchase_timestamp)) AS cohort_month
  FROM analytics.fact_orders o
  JOIN analytics.dim_customers c USING (customer_id)
  WHERE o.order_status = 'delivered'
  GROUP BY 1),
activity AS (
  SELECT DISTINCT c.customer_unique_id,
         date_trunc('month', o.order_purchase_timestamp) AS activity_month
  FROM analytics.fact_orders o
  JOIN analytics.dim_customers c USING (customer_id)
  WHERE o.order_status = 'delivered')
SELECT f.cohort_month::date AS cohort,
       (EXTRACT(YEAR FROM age(a.activity_month, f.cohort_month)) * 12
        + EXTRACT(MONTH FROM age(a.activity_month, f.cohort_month)))::int AS month_offset,
       COUNT(DISTINCT a.customer_unique_id) AS active_customers
FROM first_purchase f
JOIN activity a USING (customer_unique_id)
GROUP BY 1, 2 ORDER BY 1, 2;
```

Explained in IV.8.1 with a worked example. The output is **long format**: one row per (cohort, months-since-first-purchase) with the number of distinct people active in that month. Month 0 is the cohort size. Power BI imports exactly this query as `cohort_retention` (219 rows).

### Query 15 (RFM segment grid)

```sql
WITH rfm AS (
  SELECT c.customer_unique_id,
         MAX(o.order_purchase_timestamp)::date AS last_order,
         COUNT(DISTINCT o.order_id) AS frequency,
         SUM(oi.price + oi.freight_value) AS monetary
  FROM analytics.fact_orders o
  JOIN analytics.dim_customers c USING (customer_id)
  JOIN analytics.fact_order_items oi USING (order_id)
  WHERE o.order_status = 'delivered'
  GROUP BY 1),
scored AS (
  SELECT customer_unique_id,
         NTILE(5) OVER (ORDER BY last_order) AS r_score,
         CASE WHEN frequency = 1 THEN 1 WHEN frequency = 2 THEN 2 ELSE 3 END AS f_bucket,
         monetary
  FROM rfm)
SELECT r_score, f_bucket, COUNT(*) AS customers,
       ROUND(SUM(monetary), 2) AS segment_revenue,
       ROUND(100.0 * SUM(monetary) / SUM(SUM(monetary)) OVER (), 2) AS revenue_share_pct
FROM scored GROUP BY r_score, f_bucket
ORDER BY r_score DESC, f_bucket DESC;
```

Power BI imports exactly this query as `rfm_grid` (15 rows); the segment names are added there by a DAX calculated column (V.E.2). Details in IV.8.2.

### Queries 16, 17 and 18 (reconciliation and anomaly counts)

Query 16 sums `price + freight_value` per order and `payment_value` per order, joins them and keeps rows whose absolute difference exceeds R$0.01: 303 orders (0.30% of 99,441). Query 17 counts orders with more than one review in **staging** (547), i.e. before deduplication. Query 18 counts the three impossible-timestamp patterns (1,359 · 23 · 6). These three are quality evidence rather than business answers; the validation suite logs the same numbers.

## IV.6 The four dashboard views

Views move complexity out of DAX: filters and joins are done once in SQL so measures stay simple sums and counts. Each begins `DROP VIEW IF EXISTS …;` and is rebuilt by `make views`. The files are

`sql/04_dashboard_views/01_vw_sales.sql`, `02_vw_orders.sql`, `03_vw_customers.sql` and `04_vw_daily_orders.sql`.

```sql
-- analytics.vw_sales : delivered item lines (110,197 rows)
SELECT oi.order_id, oi.order_item_id, oi.product_id, oi.seller_id,
       p.product_category_name_english, c.customer_state, c.customer_unique_id,
       o.order_purchase_timestamp, o.order_purchase_timestamp::date AS order_date,
       (oi.price + oi.freight_value) AS line_revenue
FROM analytics.fact_order_items oi
JOIN analytics.fact_orders o     USING (order_id)
JOIN analytics.dim_products p    USING (product_id)
JOIN analytics.dim_customers c   USING (customer_id)
WHERE o.order_status = 'delivered';
```

```sql
-- analytics.vw_orders : delivered orders + review (96,478 rows)
SELECT o.order_id, o.customer_id, c.customer_state, o.order_purchase_timestamp,
       o.order_purchase_timestamp::date AS order_date, o.delivery_days, o.is_late,
       o.order_estimated_delivery_date, o.order_delivered_customer_date, r.review_score
FROM analytics.fact_orders o
JOIN analytics.dim_customers c USING (customer_id)
LEFT JOIN analytics.order_reviews r USING (order_id)     -- LEFT: not every order has a review
WHERE o.order_status = 'delivered';
```

```sql
-- analytics.vw_customers : one row per PERSON (93,358 rows)
SELECT c.customer_unique_id, COUNT(DISTINCT o.order_id) AS delivered_orders,
       SUM(oi.price + oi.freight_value) AS lifetime_revenue,
       MIN(o.order_purchase_timestamp) AS first_order,
       MAX(o.order_purchase_timestamp) AS last_order,
       (COUNT(DISTINCT o.order_id) >= 2) AS is_repeat
FROM analytics.fact_orders o
JOIN analytics.dim_customers c   USING (customer_id)
JOIN analytics.fact_order_items oi USING (order_id)
WHERE o.order_status = 'delivered'
GROUP BY c.customer_unique_id;
```

```sql
-- analytics.vw_daily_orders : ALL placed orders per day (634 rows)
SELECT order_purchase_timestamp::date AS order_date, COUNT(*) AS orders
FROM analytics.fact_orders GROUP BY 1;
```

| View | Grain | Why built this way |
|---|---|---|
| `vw_sales` | Order line | Revenue is per line (price + freight), and category and seller live at line level. 110,197 lines belong to 96,478 delivered orders. |
| `vw_orders` | Order | Delivery and review metrics are per order. `LEFT JOIN` keeps orders without a review so delivery counts are not understated. |
| `vw_customers` | Person | Pre-computes `is_repeat` so DAX is a plain `DISTINCTCOUNT`. Uses `customer_unique_id`. |
| `vw_daily_orders` | Day | All statuses (the forecast population). Only days with at least one order appear: 634 rows between 2016-09-04 and 2018-10-17. |

> CAUTION
> Because `vw_orders` joins reviews to orders and `vw_sales` joins orders to items, **never relate these tables to each other in Power BI** and never add a measure that mixes them; the grains differ.

## IV.7 The four notebooks

Notebooks live in `notebooks/` and import `src.utils` via `sys.path.insert(0, "..")`, so they work from any kernel working directory. They read `analytics.*`; only notebook 04 writes to the database. Execute them with `nbconvert` (III.2 steps 10–11) or interactively with `jupyter lab`.

| Notebook 01 · EDA and the stable window | |
|---|---|
| **Purpose** | Profiles the data and derives the **stable window** `2017-01-05` to `2018-08-22` that the forecast uses. Early 2016 is a sparse ramp with zero-order days through 2017-01-04; from late August 2018 volume collapses to a trickle. |
| **Inputs / outputs** | Reads `fact_orders`, products, customers. Prints/plots: daily orders, null timestamps by status, category counts, state counts, delivery days; asserts "0 gap days" in the window. No database writes. |
| **Verify** | Printed: `Stable window 2017-01-05 to 2018-08-22: 0 gap days confirmed.` |
| **Downstream** | Notebook 04 reuses the window constants. |

| Notebook 02 · Retention cohorts and RFM | |
|---|---|
| **Purpose** | Runs query 12, pivots it into a cohort × month-offset retention matrix and heat-map, then reads the RFM grid of query 15 (the same SQL the dashboard imports) and labels segments in business language. |
| **Key logic** | `retention_pct = active / month-0 size`; `label_segment` maps (recency, frequency) to **Champions**, **At-Risk Repeat**, **Recent One-Time**, **Lapsed One-Time** (IV.8.2). |
| **Outputs** | Printed tables and charts; no database writes. |
| **Verify** | Segment table: Lapsed One-Time 54,419 customers (58.29%, 55.88% of revenue), Recent One-Time 36,138 (38.71%, 38.51%), At-Risk Repeat 1,597 (1.71%, 3.09%), Champions 1,204 (1.29%, 2.52%), the same as the dashboard. |

| Notebook 03 · Delivery lateness versus review score | |
|---|---|
| **Purpose** | Tests whether late orders get lower reviews. H0: the two score distributions do not differ. H1 (one-sided): late scores are stochastically **lower** (`scipy.stats.mannwhitneyu(late, on_time, alternative="less")`). |
| **Inputs** | 95,824 reviewed delivered orders with a known `is_late`. |
| **Outputs** | n = 7,661 late and 88,163 on-time; U = 150,706,850.5; printed p-value `0.000e+00`; summary table; plain-language conclusion. |
| **Important** | The printed `0.000e+00` is **floating-point underflow, not a probability of zero**. Report it as **p < 0.001**. See IV.8.3. |

| Notebook 04 · 28-day demand forecast | |
|---|---|
| **Purpose** | Forecasts daily **all-placed-orders** volume for the next 28 days and compares two methods with a walk-forward backtest. |
| **Steps** | Load 595 daily counts (2017-01-05 to 2018-08-22) and assert no gaps → show Black Friday 2017-11-24 (1,176 orders, 6.0× the prior two-week average of 197/day) lies inside every training window → backtest 6 folds × 28 days → choose the winner by mean WAPE → compute daily bounds → write `analytics.forecast_28d` and `data/processed/forecast_28d.csv`. |
| **Outputs** | Table `analytics.forecast_28d(date PK, forecast, lower, upper)`, 28 rows; CSV of the same. |
| **Verify** | Seasonal naive wins (0.2308 vs 0.2310); total 6,660 orders for 2018-08-23 to 2018-09-19; summed bounds 3,584.9 and 9,943. |
| **Fails when** | The no-gaps assertion; a missing `analytics.fact_orders`; permissions to drop and recreate the table. |

| Notebook | Reads | Writes | Depends on |
|---|---|---|---|
| 01_eda | `analytics.*` | Nothing | Pipeline |
| 02_retention_rfm | `analytics.*` | Nothing | Pipeline |

| Notebook | Reads | Writes | Depends on |
|---|---|---|---|
| 03_delivery_reviews | `analytics.fact_orders`, `order_reviews` | Nothing | Pipeline |
| 04_demand_forecast | `analytics.fact_orders` | `analytics.forecast_28d`, `data/processed/forecast_28d.csv` | Pipeline; its output is needed by Power BI |

## IV.8 Concepts, explained with this project's data

### IV.8.1 Cohort retention

A **cohort** is the group of customers who made their first purchase in the same month. **Retention at month *n*** is the share of that cohort who bought again *n* months later. Worked example with five invented customers:

| Person | Purchases (delivered) | Cohort (first month) |
|---|---|---|
| A | Jan, Mar | Jan |
| B | Jan | Jan |
| C | Feb, Mar | Feb |
| D | Feb | Feb |
| E | Feb, Feb (two orders) | Feb |

| Cohort | Month 0 | Month 1 | Month 2 |
|---|---|---|---|
| Jan | 2 (A, B) = 100% | 0 = 0% | 1 (A) = 50% |
| Feb | 3 (C, D, E) = 100% | 1 (C) = 33% | – |

Query 12 does exactly this. `first_purchase` finds each person's earliest month. `activity` lists each person's **distinct** active months (so person E's two February orders count once). Joining them on `customer_unique_id` and computing the whole-month difference with `age()` gives the offset. `COUNT(DISTINCT …)` then counts people per (cohort, offset). Person E shows why the person key matters: counting `customer_id` would treat every order as a new customer.

On the real data month 0 is 100% by definition, and month 1 is tiny: weighted across all cohorts except the newest (2018-08, which has not had time to return), **421 of 87,214 customers (0.48%)** bought again in the following month. That near-empty heat-map is the finding. Do not "fix" it.

### IV.8.2 RFM segmentation

**R**ecency, **F**requency and **M**onetary value describe each customer. In this project:

- **Recency score** `r_score = NTILE(5) OVER (ORDER BY last_order)`: five equal-size groups by date of last order; **5 = most recent**. Customers with identical dates are split across bucket edges arbitrarily, so group sizes and revenue in a bucket can differ slightly between engines. The dashboard footnote warns of this.
- **Frequency bucket** `f_bucket`: 1 order, 2 orders, or 3+ orders. Not a quintile, because 97% of customers bought once and `NTILE` would split identical values arbitrarily.
- **Monetary**: lifetime revenue. Query 14 also scores it as `m_score = NTILE(5)`; the dashboard grid uses revenue totals rather than an M score.

- **Segments** (calculated column `rfm_grid[segment]`): frequency ≥ 2 and recency ≥ 4 = **Champions (repeat, recent)**; frequency ≥ 2 and recency ≤ 3 = **At-Risk Repeat**; frequency 1 and recency ≥ 4 = **Recent One-Time (acquisition pool)**; everything else = **Lapsed One-Time**.

| Segment | Customers | Share of customers | Revenue (R$) | Share of revenue |
|---|---|---|---|---|
| Lapsed One-Time | 54,419 | 58.29% | 8,617,203.27 | 55.88% |
| Recent One-Time (acquisition pool) | 36,138 | 38.71% | 5,938,383.02 | 38.51% |
| At-Risk Repeat | 1,597 | 1.71% | 475,825.98 | 3.09% |
| Champions | 1,204 | 1.29% | 388,361.48 | 2.52% |
| **Total** | **93,358** | **100%** | **15,419,773.75** | **100%** |

NOTE

**Ties on the boundary dates.** Each recency quintile ends on a date that it shares with the next quintile (2017-08-12, 2017-12-04, 2018-03-05 and 2018-05-29), and `NTILE` splits the customers on such a date in an order PostgreSQL does not define. A different scoring method or query plan can therefore move a few customers: earlier versions of notebook 02, which used `pandas.qcut`, reported 1,201 Champions and 2.51% of revenue (Lapsed One-Time 55.89%). Notebook 02 now reads query 15, the query the dashboard imports, so the notebook, `make analysis`, Power BI and the documents all show the table above. Only the second decimal was ever affected. The dashboard footnote says scores run "1–5 on recency, frequency and spend"; precisely, recency uses five bands, frequency three buckets, and spend enters as revenue.

### IV.8.3 The Mann–Whitney U test

Review scores are **ordinal** (1 to 5) and heavily skewed (most reviews are 5), so a t-test, which assumes roughly normal numbers, is the wrong tool. Mann–Whitney U compares the two groups using **ranks**: pool all 95,824 scores, rank them (ties share the average rank) and ask whether late orders' ranks tend to sit lower.

- **Hypotheses.** H0: late and on-time scores come from the same distribution. H1 (one-sided): late scores are stochastically smaller.
- **U** counts, over all late/on-time pairs, how often the late score is higher (ties count half). Here U = 150,706,850.5 out of 7,661 × 88,163 = 675,416,743 pairs, i.e. 22.3%. The complement, **77.7%**, is the probability that a random on-time order outscores a random late one (ties counted half). This "common-language" effect size, or equivalently a rank-biserial correlation of about 0.55, is what matters.
- **The p-value.** With samples this large the normal approximation of U gives a tie-corrected z of roughly −91. A p-value that small underflows a 64-bit float, so SciPy prints `0.000e+00`. The true value is positive but unrepresentable. Always write **p < 0.001**, never "p = 0".
- **Significance is not importance.** With ~96,000 observations almost any difference is "significant". The effect size (median 2 versus 5; 54.07% versus 9.22% giving one or two stars) shows it is also large.
- **Association, not causation.** Late deliveries correlate with distance, particular sellers and freight cost, which could themselves affect satisfaction. The test cannot separate these.

### IV.8.4 The forecast

The target is the **daily count of all placed orders**. Two methods are compared:

- **Seasonal naive**: predict that each future day equals the same weekday last week. In code, `np.tile(last_week, horizon // 7)` repeats the final seven observed days four times. It needs no fitting.
- **Holt–Winters** (`ExponentialSmoothing(trend="add", seasonal="add", seasonal_periods=7)`): additive level, trend and a weekly seasonal pattern, fitted by optimisation.

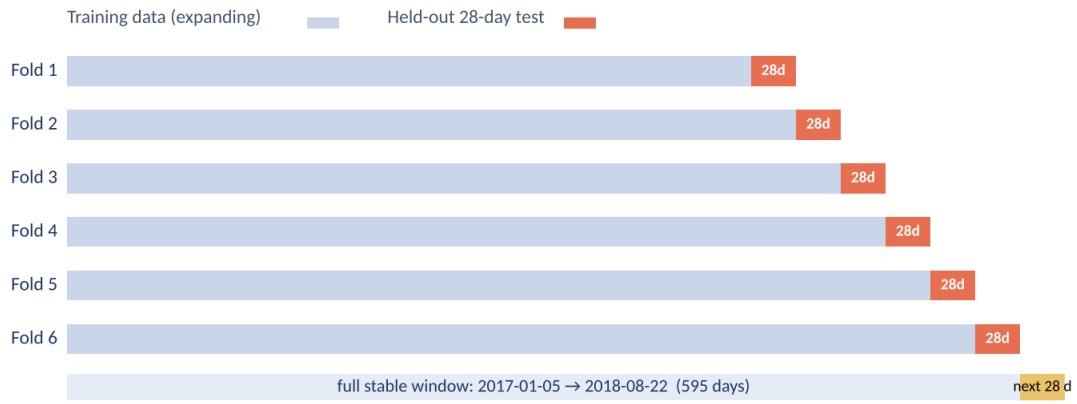



**Figure 5.** Walk-forward backtest: six expanding training windows, each tested on the next 28 days. The final fit uses the whole 595-day window.

- **Walk-forward backtest.** For fold *k* the model sees only data before the test block, forecasts the next 28 days and is scored. Training grows each time (expanding window). This mimics real use and avoids "peeking" at the future. Test blocks: 2018-03-08 → 2018-04-04 … 2018-07-26 → 2018-08-22.
- **WAPE** (weighted absolute percentage error) = Σ|actual − forecast| ÷ Σ|actual|. Lower is better; it is stable when some days are small.
- **Bias** = mean(forecast − actual). Positive means over-forecasting.
- **Result.** Mean WAPE 0.2308 (seasonal naive, bias −10.14) versus 0.2310 (Holt–Winters, bias about +2.4): a tie. The acceptance rule says the model must **beat** the baseline to be reported, so the simpler baseline is the recommendation.
- **Bounds.** The 168 pooled backtest errors (6 × 28) give the 2.5th and 97.5th percentiles, roughly −110 and +117 orders per day; added to the forecast, with the lower bound floored at zero. These are **daily** bounds. Summing 28 daily bounds (3,584.9 and 9,943) is **not** a valid 28-day prediction interval: it assumes the daily errors are perfectly correlated. Across the six folds the 28-day totals actually missed by between −2,363 and +731 orders, and six folds are too few to build a proper total interval.
- **Black Friday.** 2017-11-24 had 1,176 orders against about 197 per day. It sits in the training data of every fold (the first test block starts 104 days later), so it cannot distort the reported accuracy. It is not removed.

## IV.9 Historical outputs versus a fresh run

The `.ipynb` files in the repository contain **stored outputs** from the run that produced the published figures. Running a notebook again overwrites them with new outputs. Never edit a stored output to "make it match"; if numbers differ, understand why.

| Item | Stored (historical) value | Fresh run (executed) | Verdict |
|---|---|---|---|
| Notebook 01 stable window | 2017-01-05 to 2018-08-22, 0 gap days | Identical | Same |
| Notebook 02 segment shares, 3.0% repeat rate | As in IV.8.2 (after the switch to query 15) | Identical | Same |
| Notebook 03 U, n, medians, 1–2★ shares | U 150,706,850.5; 7,661 / 88,163; 54.07% / 9.22% | Identical | Same |
| Notebook 04 seasonal-naive fold WAPE and bias | e.g. fold 1: 0.154745, +26.11 | Identical | Same |
| Notebook 04 Holt–Winters fold values | fold 1 WAPE 0.167941; fold 3 bias +34.78; mean bias +2.44 | fold 1 0.168123; fold 3 +34.66; mean bias +2.46 | **Differ in the 3rd–4th decimal** |
| Notebook 04 mean WAPE (4 dp) | 0.2308 / 0.2310 | 0.2308 / 0.2310 | Same; verdict unchanged |
| `data/processed/forecast_28d.csv` | 6,660 total; bounds 3,584.9 / 9,943 | Byte-identical file | Same |

The Holt–Winters difference comes from the numerical optimiser inside statsmodels, which can converge to marginally different parameters on a different operating system, CPU or library build, even with identical pinned package versions. It does not change any conclusion because the baseline forecast is deterministic and wins either way. The fresh run above was on Linux with Python 3.12.3. On Windows the same small differences may appear, or not; this was not tested.

VERIFICATION STATUS

**Safe check after any rerun.** Notebook 04: mean WAPE prints 0.2308 and 0.2310 (4 decimals), the verdict says `seasonal_naive wins`, the total is 6,660 and `analytics.forecast_28d` has 28 rows. If these hold, small fold-level differences do not matter. If the forecast total changes, the data or the seasonal-naive logic changed and needs investigation.

## IV.10 Continuous integration and the CI fixtures

`.github/workflows/ci.yml` runs on every push and pull request to `main`: it starts a `postgres:16` service, installs `requirements.txt` on Python 3.12, generates synthetic CSVs with `generate_fixtures.py`, runs `python -m src.run_pipeline` and `python -m pytest -q`. It needs **no Kaggle credentials** because the CSVs are synthetic. It proves the code and the validation logic work; it says nothing about the real dataset.

VERIFICATION STATUS

**Not verified.** The workflow was not run on GitHub. The same sequence (fixtures → pipeline → 10 tests) was executed locally against a separate database and passed. A push to GitHub could not be made in the working environment because no credentials were available, so the local commits are ahead of `origin/main`.

# Part V — Rebuild the Power BI report

This is the part of the guide to work through slowly. It takes you from a running database to the finished four-page report: connecting, shaping the model, writing all 43 DAX measures, building every visual, wiring the slicers, and proving that the numbers are right. Everything is taken from the final report file, so where this part gives a name, a formula, a size or a colour, it is the one stored in the file.

> NOTE
> **Which file is the real one?** `dashboard/ecommerce_retention_dashboard_final.pbix` is the authoritative report. The other files under `dashboard/archive/` (kept locally, not in Git) are earlier versions and project exports. **Open the authoritative file only to read or to refresh it. Do all experiments in a copy made with File → Save as.**

## How to use Part V

| Route | What you do | When to choose it |
|---|---|---|
| **1. Inspect and refresh** | Open the final `.pbix`, enter the database password when asked, press **Refresh**, compare the numbers with V.J. | You only want to see the report working, or to update it after re-running the pipeline. |
| **2. Rebuild from nothing** | Follow V.A to V.J in order: new blank report → connect → model → measures → visuals. | You want to learn Power BI properly, or the report file is lost. |
| **3. Rebuild one piece** | Jump to the measure card (V.E) or the visual recipe (V.F) you need. | You are fixing or extending one visual. |

How the labels work: **File-verified** means the item was read from the final `.pbix` itself (model tables, formulas, formats, visual types, field wells, positions, stored formatting properties). **Reviewed** means it comes from Microsoft documentation or from the repository documents and screenshots. A menu path in the Power BI window can move between releases; the report was created with the Power BI Desktop release 2026.09, and if a menu has moved, use the search box at the top of the window.

> VERIFICATION STATUS
> **Verification basis.** The model and the visuals were extracted from the final `.pbix` and every number in this part was recomputed from the PostgreSQL warehouse built by the pipeline. A copy of the final `.pbix` was also opened in Power BI Desktop and all four pages were viewed without a refresh: the cards, the RFM bars and the page-4 table show the reference values in V.J.2. No refresh against the Docker database was performed here; the owner's own build and refresh are recorded in `dashboard/BUILD_QA_REPORT.md`.

## V.A Prepare and verify the data before opening Power BI

Power BI only reads what PostgreSQL holds. A report built on a half-finished database does not fail loudly; it shows blanks or small wrong numbers. Spend two minutes on the checks below first.

| # | Check | Command (PowerShell, repository root) | Expected |
|---|---|---|---|
| 1 | Container is running and healthy | `docker compose ps` | `olist_postgres` … `Up … (healthy)`, port `5432` |
| 2 | Database answers | `docker exec olist_postgres pg_isready -U analyst -d olist` | `accepting connections` |
| 3 | The four dashboard views exist | `docker exec olist_postgres psql -U analyst -d olist -Atc "SELECT count(*) FROM information_schema.views WHERE table_schema='analytics' AND table_name LIKE 'vw_%';"` | `4`. If `0`, run `make views` (the pipeline drops them). |
| 4 | Forecast table is filled | `docker exec olist_postgres psql -U analyst -d olist -Atc "SELECT count(*), round(sum(forecast)::numeric,0) FROM analytics.forecast_28d;"` | `28\|6660`. If the table is missing, run notebook 04. |

| # | Check | Command (PowerShell, repository root) | Expected |
|---|---|---|---|
| 5 | Dashboard views have the expected rows | `docker exec olist_postgres psql -U analyst -d olist -Atc "SELECT (SELECT count(*) FROM analytics.vw_sales),(SELECT count(*) FROM analytics.vw_orders),(SELECT count(*) FROM analytics.vw_customers),(SELECT count(*) FROM analytics.vw_daily_orders);"` | `110197\|96478\|93358\|634` |
| 6 | Revenue reference value | `docker exec olist_postgres psql -U analyst -d olist -Atc "SELECT round(sum(line_revenue),2) FROM analytics.vw_sales;"` | `15419773.75` |

CAUTION

**Never run `make pipeline` between checks 3 and Power BI without running `make views` afterwards.** The pipeline rebuilds the `analytics` tables with `DROP … CASCADE`, which silently drops the four `vw_*` views. Power BI then fails with "relation analytics.vw_sales does not exist".

Also decide the port now. The report's queries use `localhost:5432`. If `DB_PORT` in your `.env` is not 5432, either change it back or edit the server box in every query (V.B).

# V.B Connect Power BI Desktop to PostgreSQL

## V.B.1 What the connection looks like

- **Storage mode:** Import. Power BI copies the data into the report file; the database is only needed when you refresh.
- **Server and database:** `localhost:5432` and `olist`. The user name and password are **not stored in the report**. You type them once into Power BI's credential dialog (Database tab); Windows keeps them for your account, not in the `.pbix`.
- **Every table query is a native query.** All nine start with `SET max_parallel_workers_per_gather = 0;`. In the original build, loading `vw_sales` from the Docker container failed with PostgreSQL error `53100` ("could not resize shared memory segment"): Docker gives the container a 64 MB shared-memory area, and parallel query workers need more. Turning parallel workers off for that session avoids the error. The setting lasts only for the query's connection and changes nothing in the database.
- **A permanent alternative** (recorded in the QA report, not applied here): add `shm_size: 1gb` to the PostgreSQL service in `docker-compose.yml` and recreate the container. The data volume survives container re-creation (III.5). The `SET` prefix needs no change to Docker, which is why the report uses it.
- **Query folding is switched off** (`EnableFolding = false`) because the SQL is sent as written; Power BI does not rewrite it.

VERIFICATION STATUS

**File-verified.** The nine queries below are copied from the final `.pbix`. The setting "Parallel loading of tables = One" is described in the QA report but is not stored in the model metadata, so it could not be confirmed from the file. It is not needed to reproduce the report; the SET prefix is what cured error 53100.

## V.B.2 Create the new report file

| STEP B.2 · Start a copy to work in | |
|---|---|
| **Where** | Power BI Desktop. |
| **Action** | Start Power BI Desktop → **Blank report** → **File → Save as** → choose a folder outside the repository's `dashboard` folder (for example `D:\work\olist_rebuild.pbix`). |
| **Purpose** | Saving first means a crash during the long steps does not lose work, and guarantees you cannot overwrite the authoritative file. |

| | |
|---|---|
| **Expected result** | The title bar shows your new file name. |
| **Verification** | Title bar is not `ecommerce_retention_dashboard_final`. |
| **Troubleshooting** | If you opened the final file by mistake, use **Save as** immediately; do not press Ctrl+S first. |
| **Next step** | B.3. |

## V.B.3 Add the nine tables

There are two ways to get a table in. The Navigator route (Get data → PostgreSQL database → tick tables) is the quickest, but it does not add the `SET` prefix, so it may hit error 53100 on `vw_sales`. The route below reproduces the report file exactly.

| **STEP B.3 · Add the first query (vw_customers)** | |
|---|---|
| **Where** | Power BI Desktop, Home ribbon. |
| **Action** | 1. **Home → Get data → Blank query.** The Power Query Editor opens.<br>2. In the Power Query Editor choose **Home → Advanced Editor**.<br>3. Delete everything in the box and paste the M code of the first query from V.B.4 (`vw_customers`).<br>4. Click **Done**.<br>5. If a yellow bar says credentials are needed, click **Edit Credentials** → **Database** tab → type the **user name and password from your `.env`** → **Connect**. Type them yourself; do not paste them into notes, chat or screenshots.<br>6. If Power BI shows a "Native Database Query" approval dialog, review the `SET …; SELECT …` text and click **Run**.<br>7. In the Query Settings pane on the right, rename the query to exactly `vw_customers`. |
| **Purpose** | Loads one table into the model. The table name matters: every DAX formula in V.E refers to it by name. |
| **Expected result** | A table of data appears in the editor preview with the columns listed in V.C. |
| **Verification** | The preview shows `customer_unique_id`, `delivered_orders`, `lifetime_revenue`, `first_order`, `last_order`, `is_repeat`. |
| **Troubleshooting** | • "We couldn't authenticate with the credentials": the password differs from the one the database volume was created with (III.7).<br>• "relation … does not exist": run `make views` (V.A, check 3).<br>• Error 53100: confirm the `SET` prefix is in the query text.<br>• Connection refused: Docker is not running or the container is stopped (V.A, checks 1–2). |
| **Next step** | Repeat steps 1–7 for the other eight queries, using a new **Blank query** each time, then B.5. |

> **TIP**
> **Shortcut for the other eight.** In the Queries pane right-click `vw_customers` → **Duplicate**, open the copy's Advanced Editor, change only the table name after `FROM analytics.` and rename the query. The two native-query tables (`cohort_retention`, `rfm_grid`) are different and must be pasted in full.

## V.B.4 The nine queries (Power Query M)

The seven object-based tables share one pattern. Replace `<name>` by `vw_customers`, `vw_daily_orders`, `vw_orders`, `vw_sales`, `dim_date`, `forecast_28d` or `order_payments`; the query name in Power BI is the same word.

```
POWER QUERY M
let
    Source = Value.NativeQuery(
        PostgreSQL.Database("localhost:5432", "olist"),
        "SET max_parallel_workers_per_gather = 0; SELECT * FROM analytics.<name>",
        null, [EnableFolding = false])
in
    Source
```

Line by line: `Value.NativeQuery` sends text to the database; its first argument is the connection (`PostgreSQL.Database("server", "database")`); the second is the SQL; `null` means no query parameters; `[EnableFolding = false]` tells Power Query not to try to translate later steps back into SQL. The query returns the whole table, which Import mode stores.

`cohort_retention` and `rfm_grid` are not views. Their SQL is the repository queries 12 and 15 (IV.5), embedded in the report, followed by an explicit type step because PostgreSQL returns `numeric` columns that Power Query would otherwise leave untyped.

**cohort_retention** (repository query 12):

```
POWER QUERY M
let
    Source = Value.NativeQuery(
        PostgreSQL.Database("localhost:5432", "olist"),
        "WITH first_purchase AS ( SELECT c.customer_unique_id, MIN(date_trunc('month',
      o.order_purchase_timestamp)) AS cohort_month FROM analytics.fact_orders o JOIN
      analytics.dim_customers c USING (customer_id) WHERE o.order_status = 'delivered' GROUP BY 1 ),
      activity AS ( SELECT DISTINCT c.customer_unique_id, date_trunc('month',
      o.order_purchase_timestamp) AS activity_month FROM analytics.fact_orders o JOIN
      analytics.dim_customers c USING (customer_id) WHERE o.order_status = 'delivered' ) SELECT
      f.cohort_month::date AS cohort, (EXTRACT(YEAR FROM age(a.activity_month, f.cohort_month)) * 12 +
      EXTRACT(MONTH FROM age(a.activity_month, f.cohort_month)))::int AS month_offset, COUNT(DISTINCT
      a.customer_unique_id) AS active_customers FROM first_purchase f JOIN activity a USING
      (customer_unique_id) GROUP BY 1, 2 ORDER BY 1, 2",
        null, [EnableFolding = false]),
    Typed = Table.TransformColumnTypes(Source, {{"cohort", type date}, {"month_offset", Int64.Type},
      {"active_customers", Int64.Type}})
in
    Typed
```

**rfm_grid** (repository query 15):

```
POWER QUERY M
let
    Source = Value.NativeQuery(
        PostgreSQL.Database("localhost:5432", "olist"),
        "WITH rfm AS ( SELECT c.customer_unique_id, MAX(o.order_purchase_timestamp)::date AS
      last_order, COUNT(DISTINCT o.order_id) AS frequency, SUM(oi.price + oi.freight_value) AS
monetary
      FROM analytics.fact_orders o JOIN analytics.dim_customers c USING (customer_id) JOIN
      analytics.fact_order_items oi USING (order_id) WHERE o.order_status = 'delivered' GROUP BY 1 ),
      scored AS ( SELECT customer_unique_id, NTILE(5) OVER (ORDER BY last_order) AS r_score, CASE WHEN
      frequency = 1 THEN 1 WHEN frequency = 2 THEN 2 ELSE 3 END AS f_bucket, monetary FROM rfm )
SELECT
      r_score, f_bucket, COUNT(*) AS customers, ROUND(SUM(monetary), 2) AS segment_revenue,
ROUND(100.0
      * SUM(monetary) / SUM(SUM(monetary)) OVER (), 2) AS revenue_share_pct FROM scored GROUP BY
      r_score, f_bucket ORDER BY r_score DESC, f_bucket DESC",
        null, [EnableFolding = false]),
    Typed = Table.TransformColumnTypes(Source, {{"r_score", Int64.Type}, {"f_bucket", Int64.Type},
      {"customers", Int64.Type}, {"segment_revenue", type number}, {"revenue_share_pct", type
number}})
in
    Typed
```

WHY THIS MATTERS

**Why the SQL is inside the report.** Query 12 and 15 produce shapes that no dashboard view exposes (a long cohort table and a 15-cell RFM grid). Embedding the SQL keeps the report self-contained, but it means the logic exists twice: in `sql/03_analysis` and in the report. If you change the repository query, change the embedded copy as well.

### V.B.5 Close, apply and check the row counts

| STEP B.5 · Close & Apply and confirm the load | |
|---|---|
| **Where** | Power Query Editor → Home → **Close & Apply**. |
| **Action** | Click Close & Apply and wait for the loading dialog to finish. Then open **Data view** (table icon on the left rail), click each table in the Data pane and read the row count in the status bar at the bottom ("Table: … (N rows)"). |
| **Purpose** | Confirms the whole warehouse reached the model before any modelling starts. |
| **Expected result** | The counts in the table below. |
| **Verification** | Every count matches. A count of 0 or a missing table means the query failed. |
| **Troubleshooting** | Loading takes the longest for `vw_sales` (110,197 rows) and `order_payments` (103,886). If it fails once with 53100, check the `SET` prefix and refresh again. |
| **Next step** | V.C. |

| Power BI table | Rows (expected) | Source |
|---|---|---|
| `vw_customers` | 93,358 | view |
| `vw_daily_orders` | 634 | view (2016-09-04 to 2018-10-17) |
| `vw_orders` | 96,478 | view |
| `vw_sales` | 110,197 | view |
| `dim_date` | 1,096 | table (2016-01-01 to 2018-12-31) |
| `forecast_28d` | 28 | table written by notebook 04 |
| `order_payments` | 103,886 | table |
| `cohort_retention` | 219 | query 12 |
| `rfm_grid` | 15 | query 15 |

`forecast_chart` (118 rows) and `_Measures` (1 row) are created later in DAX, not loaded.

## V.C Object inventory

The final model has **11 tables, 3 relationships, 8 calculated columns, 2 calculated tables and 43 measures**. Use this inventory as the checklist for a rebuild: when your model matches it, the measures and visuals will work unchanged.

| Table | Kind | Rows | One row per | Used for |
|---|---|---|---|---|
| _Measures | Calculated table | 1 | One placeholder row | Holds all 43 measures |
| cohort_retention | Import (native query 12) | 219 | (cohort, month_offset) | Cohort heat-map and month-1 retention |
| dim_date | Import | 1,096 | Calendar day | Marked date table; month axis, date slicer |
| forecast_28d | Import | 28 | Forecast day | Forecast cards |
| forecast_chart | Calculated table | 118 | Day (90 actual + 28 forecast) | Forecast line chart |
| order_payments | Import | 103,886 | Payment record | Payment mix and instalments |
| rfm_grid | Import (native query 15) | 15 | (r_score, f_bucket) | RFM segment bars |
| vw_customers | Import | 93,358 | Person | Customer counts and repeat rate |

| Table | Kind | Rows | One row per | Used for |
|---|---|---|---|---|
| vw_daily_orders | Import | 634 | Calendar day with orders | Actual orders behind the forecast chart |
| vw_orders | Import | 96,478 | Delivered order | Delivery and review measures |
| vw_sales | Import | 110,197 | Delivered order line | Revenue, orders, categories, states, sellers |

### _Measures

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| `Dummy` | Whole number | – | – | Placeholder column (value 1) that makes the table exist. Nothing references it. |

### cohort_retention

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| `cohort` | Date | – | `mmm yy` | First-purchase month of the cohort (row of the heat-map). |
| `month_offset` | Whole number | – | `0` | Months since the first purchase (0 to 20). Column of the heat-map. |
| `active_customers` | Whole number | – | `#,0` | Distinct people from that cohort who bought in that month. Summed by measures. |

### dim_date

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| `date` | Date | – | `Long Date` | Primary key; the table is marked as the date table on this column. |
| `year` | Whole number | – | `0` | Calendar year (2016–2018). |
| `month` | Whole number | – | `0` | Month number 1–12. |
| `weekday` | Text | – | – | Day name. |
| `is_weekend` | True/False | – | `"TRUE";"TRUE";"FALSE"` | Weekend flag. |
| `month_start` | Date (calculated) | – | `mmm yy` | First day of the month; the month axis of every monthly chart (V.E.2). |

### forecast_28d

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| `date` | Date | – | `Long Date` | The 28 forecast days (2018-08-23 to 2018-09-19). |
| `forecast` | Decimal | – | `#,0.0` | Forecast orders per day. |
| `lower` | Decimal | – | `#,0.0` | Daily lower 95% bound. |
| `upper` | Decimal | – | `#,0.0` | Daily upper 95% bound. |

### forecast_chart

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| `date` | Date | – | – | Calculated table: 90 actual days then 28 forecast days (118 rows). |
| `actual` | Decimal | – | – | Actual orders; blank on forecast days. |

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| forecast | Decimal | – | – | Forecast; blank on actual days. |
| lower | Decimal | – | – | Lower bound; blank on actual days. |
| upper | Decimal | – | – | Upper bound; blank on actual days. |

### order_payments

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| order_id | Text | Hidden | – | Order identifier (not used by any measure). |
| payment_sequential | Whole number | Hidden | 0 | Sequence of payments within an order. |
| payment_type | Text | – | – | credit_card, boleto, voucher, debit_card, not_defined. |
| payment_installments | Whole number | – | 0 | Number of instalments. |
| payment_value | Decimal | – | #,0.00 | Amount of the payment record (R$). |
| installments_bucket | Text (calculated) | – | – | "0", "1" … "10", "11+" label (V.E.2). |
| installments_sort | Whole number (calculated) | Hidden | 0 | 0–11 sort key for the bucket label. |

### rfm_grid

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| r_score | Whole number | – | 0 | Recency score 1 (least recent) to 5 (most recent). |
| f_bucket | Whole number | – | 0 | Frequency bucket: 1, 2, or 3 (3 = three or more delivered orders). |
| customers | Whole number | – | #,0 | People in the (r_score, f_bucket) cell. |
| segment_revenue | Decimal | – | \R\$#,0.00 | Revenue of those people. |
| revenue_share_pct | Decimal | – | 0.00 | Revenue share per cell, computed in SQL. |
| segment | Text (calculated) | – | – | Named segment from the score pair (V.E.2). |

### vw_customers

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| customer_unique_id | Text | Hidden | – | The person. |
| delivered_orders | Whole number | – | 0 | Delivered orders for that person. |
| lifetime_revenue | Decimal | – | \R\$#,0.00 | Total revenue for that person. |
| first_order | Date/time | Hidden | General Date | First order timestamp. |
| last_order | Date/time | Hidden | General Date | Last order timestamp. |
| is_repeat | True/False | – | "TRUE";"TRUE";"FALSE" | True when delivered_orders ≥ 2. |
| customer_type | Text (calculated) | – | – | "Repeat (2+ delivered orders)" or "One-time (1 delivered order)". |

### vw_daily_orders

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| order_date | Date | – | Long Date | Calendar day. |

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| orders | Whole number | – | 0 | Orders placed that day (all placed orders, not only delivered). |

### vw_orders

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| order_id | Text | Hidden | – | Order identifier. |
| customer_id | Text | Hidden | – | Order-scoped customer id. |
| customer_state | Text | – | – | Two-letter state code. |
| order_purchase_timestamp | Date/time | Hidden | General Date | Purchase timestamp. |
| order_date | Date | – | Long Date | Purchase date; relationship column. |
| delivery_days | Whole number | – | 0 | Whole days from purchase to delivery; blank if not delivered. |
| is_late | True/False | – | "TRUE";"TRUE";"FALSE" | Delivered after the estimated date; blank if undelivered. |
| order_estimated_delivery_date | Date/time | Hidden | General Date | Promised date. |
| order_delivered_customer_date | Date/time | Hidden | General Date | Actual delivery. |
| review_score | Whole number | – | 0 | 1–5; blank when the order has no review. |
| delivery_status | Text (calculated) | – | – | Late / On time / Unknown (no delivery date). |
| delivery_days_capped | Whole number (calculated) | – | 0 | delivery_days capped at 60 for the histogram. |

### vw_sales

| Column | Type | Hidden | Format | Meaning |
|---|---|---|---|---|
| order_id | Text | Hidden | – | Order identifier. |
| order_item_id | Whole number | Hidden | 0 | Line number within the order. |
| product_id | Text | Hidden | – | Product identifier. |
| seller_id | Text | Hidden | – | Seller identifier. |
| product_category_name_english | Text | – | – | English category; blank for 1,537 lines. |
| customer_state | Text | – | – | Two-letter state code. |
| customer_unique_id | Text | Hidden | – | The person. |
| order_purchase_timestamp | Date/time | Hidden | General Date | Purchase timestamp. |
| order_date | Date | – | Long Date | Purchase date; relationship column. |
| line_revenue | Decimal | Hidden | \R\$#,0.00 | price + freight_value for the line; summed by Total Revenue. |
| seller_label | Text (calculated) | – | – | "Seller " + first 8 characters of seller_id. |

NOTE

**Hidden** means the column is hidden from Report view (right-click the column → Hide in report view). Identifier and timestamp columns are hidden so that report builders see only fields that make sense on a chart. `vw_sales[line_revenue]` is hidden precisely so that nobody drags it onto a chart and gets an implicit sum instead of the `Total Revenue` measure. Nine numeric columns keep the *Sum* default summarisation (`active_customers`, `forecast`, `lower`, `upper`, `payment_value`, `customers`, `segment_revenue`, `orders`, `line_revenue`); every other column is set to *Don't summarize*.

VERIFICATION STATUS

**File-verified.** Table list, column names and types, hidden flags, formats, summarisation and row counts come from the `.pbix` data model. Whether the placeholder column `Dummy` in `_Measures` is hidden was not captured, and nothing depends on it.

## V.D The data model and relationships

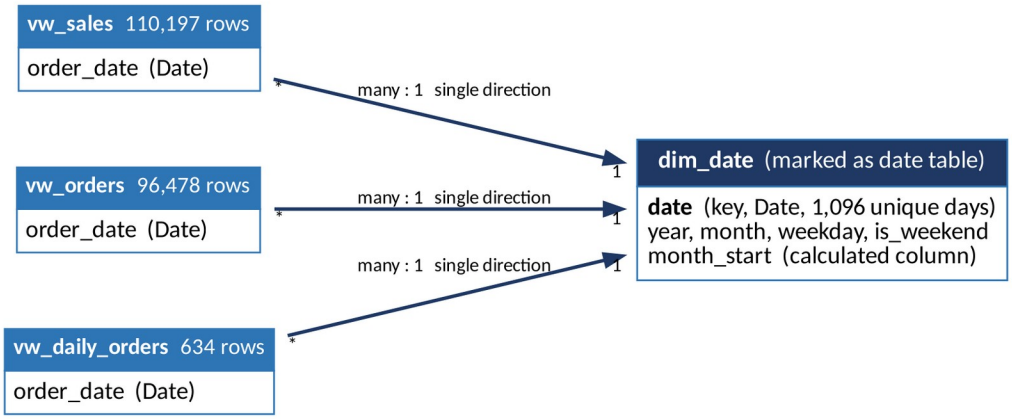



**Figure 6.** The three relationships in the Power BI model. Arrows point from the date table ("one" side) to the tables it filters ("many" side).

| From (many) | To (one) | Cardinality | Cross-filter | Active | Assume referential integrity |
|---|---|---|---|---|---|
| `vw_sales[order_date]` | `dim_date[date]` | Many to one (*:1) | Single (dim_date → vw_sales) | Yes | Off |
| `vw_orders[order_date]` | `dim_date[date]` | Many to one (*:1) | Single | Yes | Off |
| `vw_daily_orders[order_date]` | `dim_date[date]` | Many to one (*:1) | Single | Yes | Off |

Single direction means a filter on the date table narrows the order tables, but a filter on an order table does **not** narrow the date table. That is why a state slicer on `vw_sales[customer_state]` still lets every month appear on a monthly axis (months without sales simply show no bar).

### The tables that are deliberately not related

| Table | Why it has no relationship | Consequence in the report |
|---|---|---|
| `order_payments` | Payments include every order status, while the date table drives delivered sales. Relating them would silently mix two populations. | The date, state and category slicers do **not** filter the two payment visuals; a text box on page 2 says so. |
| `cohort_retention`, `rfm_grid` | They are pre-aggregated by cohort month or by score; they carry no order date, state or category. | Page 3 has no slicers; every figure is all-time. |
| `vw_customers` | One row per person across all time, so a date slice would be meaningless. | Customer counts ignore the date slicer. |

| Table | Why it has no relationship | Consequence in the report |
|---|---|---|
| `forecast_28d`, `forecast_chart` | Future dates; the forecast concerns all placed orders, not delivered sales. | Forecast visuals ignore the slicers. |
| `vw_orders` ↔ `vw_sales` | Different grain (order vs order line). Each has its own state column and its own slicer. | Page 4 uses `vw_orders[customer_state]`; pages 1–2 use `vw_sales[customer_state]`. |

| STEP D.1 · Create the relationships | |
|---|---|
| **Where** | Power BI Desktop → **Model view** (the diagram icon on the left rail). |
| **Action** | Drag `order_date` from `vw_sales` onto `date` in `dim_date`. Double-click the new line and check: Cardinality **Many to one (*:1)**, Cross filter direction **Single**, **Make this relationship active** ticked, **Assume referential integrity** unticked → **OK**. Repeat for `vw_orders[order_date]` and `vw_daily_orders[order_date]`. |
| **Purpose** | Lets one date slicer and one month axis control all three order tables. |
| **Expected result** | Three solid lines from `dim_date` (the "1" end) to the three tables (the "*" end). Power BI may offer to detect relationships automatically; do not accept extra ones. |
| **Verification** | Home → **Manage relationships** lists exactly three rows. No other relationship exists. |
| **Troubleshooting** | • Dashed line: the relationship is inactive; tick "Make this relationship active".<br>• "Many-to-many" offered: the date column in `dim_date` contains duplicates or the other column is not a date.<br>• The line points the wrong way: you dragged from the one side; delete and redo. |
| **Next step** | D.2. |

| STEP D.2 · Mark dim_date as the date table and create month_start | |
|---|---|
| **Where** | Report or Data view. |
| **Action** | 1. Select the `dim_date` table → right-click it in the Data pane → **Mark as date table** → **Mark as date table** → choose the `date` column → OK. (The same command is on the **Table tools** ribbon.)<br>2. Select `dim_date` → **Table tools → New column** and enter the formula `month_start = DATE ( dim_date[year], dim_date[month], 1 )`. Set its format to `mmm yy` (Column tools → Format → Custom).<br>3. **File → Options and settings → Options → CURRENT FILE → Data Load → Time intelligence** → untick **Auto date/time**. |
| **Purpose** | DAX time-intelligence functions such as `DATEADD` need a marked, contiguous date table. Marking it also removes the hidden automatic date tables Power BI would otherwise create for each date column, and turning Auto date/time off keeps the model free of them. |
| **Expected result** | A small calendar icon appears on the `date` column. The `month_start` column shows values like "Sep 16". |
| **Verification** | Right-click `dim_date` again: the menu offers **Mark as date table** with a tick or the table shows the calendar icon; Options → Data Load shows Auto date/time unticked. |
| **Troubleshooting** | Power BI refuses to mark the table when the key column has gaps, duplicates or blanks. `dim_date` is generated with `generate_series`, so it has none; if you replaced it with your own calendar, regenerate it from the SQL. |
| **Next step** | D.3. |

VERIFICATION STATUS

**Reviewed.** Mark-as-date-table requirements (a Date or Date/Time column that is unique, has no nulls and is contiguous) and the removal of automatic date tables come from Microsoft Learn. The marked state and `date` as key are File-verified: `dim_date[date]` is flagged as the key with the Time category.

| STEP D.3 · Hide technical columns, set summarisation and sorting | |
|---|---|
| **Where** | Report view or Data view, Data pane. |
| **Action** | 1. Hide the columns marked **Hidden** in V.C (right-click → **Hide in report view**).<br>2. For `order_payments[installments_bucket]` choose **Column tools → Sort by column → installments_sort**. (The text labels "1" … "10", "11+" would otherwise sort alphabetically: "1, 10, 11+, 2, …".)<br>3. For every numeric column that is not one of the nine *Sum* columns listed in V.C, set **Column tools → Summarization → Don't summarize**.<br>4. Apply the formats in V.C (Column tools → Format). |
| **Purpose** | Keeps the field list clean and the axis in logical order. |
| **Expected result** | The installment chart orders its columns 0, 1, 2 … 10, 11+. |
| **Verification** | In Report view, expand `vw_sales`: you see `customer_state`, `order_date`, `product_category_name_english`, `seller_label` only. |
| **Troubleshooting** | Sort-by-column requires a one-to-one mapping between label and sort key; if Power BI reports an error, check that `installments_sort` was computed from the same bucket logic (V.E.2). |
| **Next step** | V.E. |

VERIFICATION STATUS

**Reviewed.** The hidden helper column `installments_sort` and the hidden-column list are File-verified. The stored sort-by-column assignment itself is not exposed by the available extraction; the helper column exists for exactly this purpose, and the sort order can be confirmed visually on the installment chart.

### Model verification checklist

| # | Check | How | Expected |
|---|---|---|---|
| 1 | Table count | Model view | 11 tables |
| 2 | Relationship count | Manage relationships | 3, all many-to-one, single, active |
| 3 | Date table | Table tools | `dim_date` marked; key `date` |
| 4 | Auto date/time | Options → Current file → Data Load | Off |
| 5 | Calculated columns | Data view | 8 (V.E.2) |
| 6 | Calculated tables | Data view | `_Measures`, `forecast_chart` |
| 7 | Measures | Fields pane | 43 in `_Measures`, in five display folders |
| 8 | Hidden columns | Report view field list | No ids or timestamps visible |
| 9 | Locale | File → Options → Regional Settings | English (United States); the stored model culture is en-US |
| 10 | Row counts | V.B.5 | All match |

# V.E DAX: concepts, calculated columns and all 43 measures

## V.E.1 The DAX you need, taught with this project

**DAX** (Data Analysis Expressions) is the formula language of Power BI. Four ideas explain almost every formula in this report.

| Idea | Plain meaning | Where you see it here |
|---|---|---|
| **Calculated column** | A formula evaluated once per row when the data is loaded; the result is stored in the model like any other column. | `delivery_status`, `month_start`, `segment` (V.E.2) |
| **Measure** | A formula evaluated at report time, once per visual cell, for whatever is filtered at that moment. Nothing is stored. | All 43 measures in `_Measures` (V.E.3) |
| **Filter context** | The set of filters acting on a cell: the slicers, the row and column of the visual, and filters built inside the formula. A measure is "what is the answer **given these filters**?" | A bar for "SP" in a revenue chart: the context is `customer_state = SP`. |
| **Row context** | While a calculated column or an iterator (`SUMX`, `FILTER`) walks a table, it sits on one row at a time. A row context does not filter other tables until it is turned into a filter. | `FILTER ( ALL ( category ), … )` inside the cumulative share |
| **Context transition** | Calling a measure from inside a row context makes `CALCULATE` convert the current row into filters. That is why [Total Revenue] inside `SUMX` gives each category's own revenue. | `Category Cumulative Share %`, `Top 5 Category Share %` |

| Function | What it does in one line |
|---|---|
| `CALCULATE ( expr, filters … )` | Evaluates `expr` after changing the filter context with the filters given. A filter on a column **replaces** existing filters on that column. |
| `FILTER ( table, condition )` | Returns the rows of a table for which the condition is true; an iterator. |
| `ALL ( col )` / `REMOVEFILTERS ( col )` | Ignore filters on a column or table. `ALL` can also be a table source; `REMOVEFILTERS` can only appear as a `CALCULATE` filter. |
| `DIVIDE ( a, b )` | a ÷ b, with a blank result instead of an error when b is zero or blank. |
| `DISTINCTCOUNT` / `COUNTROWS` | Count different values of a column / count rows of a table. |
| `MEDIAN`, `AVERAGE`, `SUM` | Aggregations; `AVERAGE` and `MEDIAN` ignore blanks. |
| `VAR … RETURN` | Name an intermediate result once and reuse it; variables are evaluated in the context where they are defined. |
| `RANKX`, `TOPN`, `SUMX` | Rank a list; pick the top N rows of a table; sum an expression row by row. |
| `DATEADD`, `EDATE` | Shift a set of dates (needs a marked date table) / shift one date by whole months. |
| `IF`, `SWITCH ( TRUE (), … )` | Conditional logic; `SWITCH ( TRUE (), … )` is a readable chain of tests. |

## The BLANK()=FALSE() edge case

> CAUTION
>
> In DAX, **a blank compares equal to 0, to an empty string and to FALSE** in simple comparisons. `BLANK() = FALSE ()` is **TRUE**. In this model `vw_orders[is_late]` is blank for the 8 delivered orders that have no delivery date. A label such as `IF ( is_late = FALSE (), "On time", "Late" )` therefore calls those 8 orders "On time". Likewise `review_score <= 2` is TRUE for a blank score.
>
> The cure used here: test `ISBLANK` **first** (calculated column `delivery_status`), count late orders with `is_late = TRUE ()` (a blank is not TRUE), and use `review_score IN { 1, 2 }`. Where a blank must be told apart from a real value, test it explicitly.

Try it: create a table visual with `vw_orders[delivery_status]` and [Delivered Orders]. You should see Late 7,826, On time 88,644, Unknown 8. If "Unknown" is missing and On time shows 88,652, the blank test is in the wrong place.

## V.E.2 Calculated columns and calculated tables

Create each calculated column with **Table tools → New column** (Data view or Report view) after selecting the right table, and each calculated table with **Table tools → New table**. Write the formula exactly as shown.

| Calculated column · vw_orders[delivery_status] | |
|---|---|
| **Formula** | `delivery_status = IF ( ISBLANK ( vw_orders[is_late] ), "Unknown (no delivery date)", IF ( vw_orders[is_late] = TRUE (), "Late", "On time" ) )` |
| **Purpose** | Label each delivered order Late, On time or Unknown so charts can compare them. |
| **Line by line** | • For every row, `ISBLANK` asks whether lateness is missing.<br>• If it is, the label is "Unknown (no delivery date)".<br>• Otherwise a second `IF` tests `is_late = TRUE ()` and returns "Late" or "On time". |
| **Row context** | Evaluated once for every row of the table when the model refreshes; result stored. |
| **Example check** | Late 7,826 · On time 88,644 · Unknown 8 (total 96,478). |
| **Watch for** | The order of the tests matters (see the BLANK edge case below): the blank test must come first. |

| Calculated column · vw_orders[delivery_days_capped] | |
|---|---|
| **Formula** | `delivery_days_capped = MIN ( vw_orders[delivery_days], 60 )` |
| **Purpose** | Cap delivery time at 60 so one extreme order does not stretch the histogram axis. |
| **Line by line** | • Scalar `MIN` of two numbers returns the smaller: 60 for any order that took 60 days or more. |
| **Row context** | Evaluated once for every row of the table when the model refreshes; result stored. |
| **Example check** | 306 orders sit in the 60 bucket. The histogram measure excludes orders with no delivery date. |
| **Watch for** | The cap is for display only; [Median Delivery Days] uses the uncapped `delivery_days`. |

| Calculated column · vw_sales[seller_label] | |
|---|---|
| **Formula** | `seller_label = "Seller " & LEFT ( vw_sales[seller_id], 8 )` |
| **Purpose** | A short anonymised label for the seller chart. |
| **Line by line** | • `LEFT ( text, 8 )` takes the first eight characters of the hashed id; `&` joins text. |
| **Row context** | Evaluated once for every row of the table when the model refreshes; result stored. |
| **Example check** | First seller shows as "Seller 4869f7a5". 2,970 distinct labels for 2,970 sellers. |
| **Watch for** | Check that labels are unique if the data changes. |

| Calculated column · dim_date[month_start] | |
|---|---|
| **Formula** | `month_start = DATE ( dim_date[year], dim_date[month], 1 )` |
| **Purpose** | First day of each month: the axis for monthly charts and the column of the category matrix. |
| **Line by line** | • `DATE ( y, m, d )` builds a real date from the year and month columns and day 1. |
| **Row context** | Evaluated once for every row of the table when the model refreshes; result stored. |
| **Example check** | 36 distinct values (January 2016 to December 2018). |
| **Watch for** | Format `mmm yy`. Use this column, not the day-level `date`, on monthly axes. |

| Calculated column · order_payments[installments_bucket] | |
|---|---|
| **Formula** | `installments_bucket = IF ( order_payments[payment_installments] <= 0, "0", IF ( order_payments[payment_installments] >= 11, "11+", FORMAT ( order_payments[payment_installments], "0" ) ) )` |
| **Purpose** | Group the 24 instalment counts into 12 readable buckets. |
| **Line by line** | • Zero or fewer → "0"; eleven or more → "11+".<br>• `FORMAT ( n, "0" )` turns a number such as 3 into the text "3". |
| **Row context** | Evaluated once for every row of the table when the model refreshes; result stored. |
| **Example check** | 12 buckets: "0" 2 records; "1" 52,546 (50.6%); "2" 12,413; "3" 10,461; … "10" 5,328; "11+" 341. |
| **Watch for** | Text sorts alphabetically, so it needs `installments_sort` as its sort key. |

| Calculated column · order_payments[installments_sort] | |
|---|---|
| **Formula** | `installments_sort = MAX ( 0, MIN ( order_payments[payment_installments], 11 ) )` |
| **Purpose** | Numeric key (0 to 11) that puts the buckets in logical order. |
| **Line by line** | • `MIN ( n, 11 )` caps at 11; `MAX ( 0, … )` raises negatives to 0. |
| **Row context** | Evaluated once for every row of the table when the model refreshes; result stored. |
| **Example check** | 12 values 0 to 11. Hidden column. |
| **Watch for** | Use it only in Sort by column. |

| Calculated column · rfm_grid[segment] | |
|---|---|
| **Formula** | `segment = SWITCH ( TRUE (), rfm_grid[f_bucket] >= 2 && rfm_grid[r_score] >= 4, "Champions (repeat, recent)", rfm_grid[f_bucket] >= 2 && rfm_grid[r_score] <= 3, "At-Risk Repeat (repeat, not recent)", rfm_grid[f_bucket] = 1 && rfm_grid[r_score] >= 4, "Recent One-Time (acquisition pool)", "Lapsed One-Time" )` |
| **Purpose** | Name each of the 15 score cells. |
| **Line by line** | • `SWITCH ( TRUE (), test1, result1, test2, result2, …, default )` returns the result of the first test that is true.<br>• `&&` means "and".<br>• The final argument catches everything else: one-time buyers with a recency score of 1 to 3. |
| **Row context** | Evaluated once for every row of the table when the model refreshes; result stored. |
| **Example check** | Champions 1,204 · At-Risk Repeat 1,597 · Recent One-Time 36,138 · Lapsed One-Time 54,419. |
| **Watch for** | Names are a project definition, not a standard; the scoring method is explained in IV.8.2. |

| Calculated column · vw_customers[customer_type] | |
|---|---|
| **Formula** | `customer_type = IF ( vw_customers[is_repeat], "Repeat (2+ delivered orders)", "One-time (1 delivered order)" )` |
| **Purpose** | Text label for the one-time vs repeat bar chart. |
| **Line by line** | • A true/false column can be used directly as the `IF` condition. |
| **Row context** | Evaluated once for every row of the table when the model refreshes; result stored. |
| **Example check** | One-time 90,557 people; Repeat 2,801. |
| **Watch for** | Two values only. |

| Calculated table · forecast_chart | |
|---|---|
| **Formula** | `forecast_chart = VAR _fstart = MIN ( forecast_28d[date] ) VAR _actual = SELECTCOLUMNS ( FILTER ( vw_daily_orders, vw_daily_orders[order_date] < _fstart && vw_daily_orders[order_date] >= _fstart - 90 ), "date", vw_daily_orders[order_date], "actual", CONVERT ( vw_daily_orders[orders], DOUBLE ), "forecast", CONVERT ( BLANK (), DOUBLE ), "lower", CONVERT ( BLANK (), DOUBLE ), "upper", CONVERT ( BLANK (), DOUBLE ) ) VAR _fc = SELECTCOLUMNS ( forecast_28d, "date", forecast_28d[date], "actual", CONVERT ( BLANK (), DOUBLE ), "forecast", forecast_28d[forecast], "lower", forecast_28d[lower], "upper", forecast_28d[upper] ) RETURN UNION ( _actual, _fc )` |
| **Purpose** | One table holding the last 90 actual days and the 28 forecast days on a single date column, so one line chart can draw actuals, forecast and bounds on one axis. |
| **Line by line** | • `_fstart` is the first forecast day (2018-08-23).<br>• `_actual` keeps daily orders from the 90 days before it (2018-05-25 to 2018-08-22), with the forecast columns left blank.<br>• `_fc` copies the 28 forecast rows with the actual column left blank.<br>• `CONVERT ( BLANK (), DOUBLE )` gives the blank columns a numeric type so `UNION` accepts them.<br>• `UNION` stacks the two blocks: 90 + 28 = **118 rows**. |
| **Example check** | Row count 118; sum of `actual` = 19,324; sum of `forecast` = 6,660. |
| **Watch for** | It is calculated at refresh from `vw_daily_orders` and `forecast_28d`. Refresh the report after notebook 04 re-writes the forecast so the window moves. There is no relationship to `dim_date`. |

| Calculated table · _Measures | |
|---|---|
| **Formula** | `_Measures = ROW ( "Dummy", 1 )` |
| **Purpose** | An empty home for all measures so they sit together in one tidy table, in five display folders: Revenue · Payments (all payment records) · Customers & Retention · Operations & Satisfaction · Forecast (all placed orders). |
| **How to use** | Create the table, then select it and add each measure with **Home → New measure**. To put a measure in a folder, select it → Measure tools → **Display folder** (Properties pane). |
| **Watch for** | Measures can live in any table; this one is a convention. The report's visuals refer to measures by `_Measures[Name]`, so a rebuild should use the same table name. |

## V.E.3 The 43 measures

Each entry has the same ten elements, in the same order: formula · purpose · line by line · tables and columns · numerator/denominator/population · filter context · relationship and date behaviour · format · example check · common mistakes. The formulas are verbatim from the final report; the "Used in" line lists the visuals (IDs from V.F) and the measures that call it. To create a measure: select `_Measures` → **Home → New measure** → paste the formula → set the format on the Measure tools ribbon → set the display folder.

### Display folder: Revenue

| Measure 1 of 43 · Total Revenue | |
|---|---|
| 1 · Formula | `Total Revenue = SUM ( vw_sales[line_revenue] )` |
| 2 · Purpose | The headline money figure: delivered item revenue, price plus freight, for whatever is currently selected. |
| 3 · Line by line | • `SUM ( … )` adds a column over every row left after filtering.<br>• `vw_sales[line_revenue]` is `price + freight_value` for one delivered order line (hidden column). |
| 4 · Tables and columns | `vw_sales[line_revenue]`. |
| 5 · Numerator, denominator, population | No denominator. Population: the 110,197 delivered order lines of `vw_sales` that survive the current filters. |
| 6 · Filter context | Each cell of a visual supplies a filter context (a month, a category, a state, a seller …); the sum is taken over the lines that match. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| 8 · Format | `\R\$#,0.00` · display folder: Revenue |
| 9 · Example check | No slicers → **R$15,419,773.75**. State = SP → R$5,769,703.15 (37.42%). SQL: `SELECT round(sum(line_revenue),2) FROM analytics.vw_sales;` |
| 10 · Common mistakes | • Dragging the hidden `line_revenue` column onto a card instead of using the measure.<br>• Expecting R$15,843,553.24: that figure is item revenue for **all** order statuses; the report counts delivered orders only.<br>• Using `price` alone and forgetting freight. |
| Used in | visuals P1-V04, P1-V08, P1-V11; measures AOV, Revenue MoM %, Latest Month Revenue MoM %, Category Rank, Top Category Revenue, Category Cumulative Share %, Top 5 Category Share %, Seller Rank, Top 10 Seller Revenue, Top 10 Seller Share % |

| Measure 2 of 43 · Total Orders | |
|---|---|
| 1 · Formula | `Total Orders = DISTINCTCOUNT ( vw_sales[order_id] )` |
| 2 · Purpose | Counts delivered orders (not order lines). |
| 3 · Line by line | • `DISTINCTCOUNT` counts how many different values a column has.<br>• An order with three items occupies three rows of `vw_sales` but one distinct `order_id`. |
| 4 · Tables and columns | `vw_sales[order_id]` (hidden). |
| 5 · Numerator, denominator, population | Population: delivered orders that have at least one item line surviving the filters. |

| | |
|---|---|
| **6 · Filter context** | Same as Total Revenue. A category slicer can keep only some lines of an order, and the order still counts once. |
| **7 · Relationship and date behaviour** | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| **8 · Format** | `#,0` · display folder: Revenue |
| **9 · Example check** | No slicers → **96,478**. `COUNTROWS ( vw_sales )` would give 110,197, which is the number of lines. |
| **10 · Common mistakes** | • Using `COUNTROWS` and reporting lines as orders.<br>• Summing the measure across categories: an order with items in two categories counts in both, so category totals can exceed 96,478. |
| **Used in** | visuals P1-V05; measures AOV |

## Measure 3 of 43 · AOV

| | |
|---|---|
| **1 · Formula** | `AOV =`<br>`DIVIDE ( [Total Revenue], [Total Orders] )` |
| **2 · Purpose** | Average order value: revenue per delivered order. |
| **3 · Line by line** | • `DIVIDE ( a, b )` returns a ÷ b, or blank if b is zero or blank, instead of an error.<br>• [Total Revenue] and [Total Orders] are measures, so each is re-evaluated in the current filter context. |
| **4 · Tables and columns** | No columns directly; built from [Total Revenue] and [Total Orders]. |
| **5 · Numerator, denominator, population** | Numerator: delivered revenue. Denominator: distinct delivered orders. Both over the same selection. |
| **6 · Filter context** | A ratio of two sums is computed per cell, so a month's AOV is that month's revenue ÷ that month's orders, never an average of daily AOVs. |
| **7 · Relationship and date behaviour** | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| **8 · Format** | `\R\$#,0.00` · display folder: Revenue |
| **9 · Example check** | No slicers → 15,419,773.75 ÷ 96,478 = **R$159.83**. State = SP → 5,769,703.15 ÷ 40,501 = R$142.46. |
| **10 · Common mistakes** | • Averaging `line_revenue` instead (gives R$139.93, a per-item value).<br>• Adding AOV across months or categories; ratios must be recomputed, not summed.<br>• Reading the early-2016 months as meaningful: September 2016 has one order (R$143.46). |
| **Used in** | visuals P1-V06, P2-V07 |

## Measure 4 of 43 · Revenue MoM %

| | |
|---|---|
| **1 · Formula** | `Revenue MoM % =`<br>`VAR CurrentRevenue = [Total Revenue]`<br>`VAR PriorRevenue =`<br>`    CALCULATE ( [Total Revenue], DATEADD ( dim_date[date], -1, MONTH ) )`<br>`RETURN`<br>`    DIVIDE ( CurrentRevenue - PriorRevenue, PriorRevenue )` |
| **2 · Purpose** | Month-over-month revenue growth for each month on an axis. |
| **3 · Line by line** | • `VAR CurrentRevenue` stores this month's revenue.<br>• `DATEADD ( dim_date[date], -1, MONTH )` shifts the dates in the current filter context back one month; `CALCULATE` re-evaluates [Total Revenue] for that shifted period.<br>• `DIVIDE ( current – prior, prior )` is the growth; it is blank when prior is blank or zero. |

| | |
|---|---|
| **4 · Tables and columns** | `dim_date[date]`; indirectly `vw_sales[line_revenue]`. |
| **5 · Numerator, denominator, population** | Numerator: current − prior revenue. Denominator: prior-month revenue. Same state/category selection for both. |
| **6 · Filter context** | With a month on the axis the context holds that month's dates. `DATEADD` needs a continuous, marked date table; it moves the whole set of dates back by one calendar month. |
| **7 · Relationship and date behaviour** | Relies on the marked `dim_date` and the `order_date` relationship. The prior month is the **calendar** prior month: there is no November 2016 sale, so December 2016 has a blank prior and shows nothing, whereas the SQL query 01 compares December with October. |
| **8 · Format** | `0.0%;-0.0%;0.0%` · display folder: Revenue |
| **9 · Example check** | August 2018: (985,491.64 − 1,027,807.28) ÷ 1,027,807.28 = **−4.12%**. October 2016 and January 2017 show extreme values (+32,307% and +649,657%) because their prior months are tiny; the chart axis is capped at −50% / +100% so the rest stays readable. |
| **10 · Common mistakes** | • Using `SAMEPERIODLASTYEAR` or `PREVIOUSMONTH` without a marked date table.<br>• Putting `vw_sales[order_date]` instead of `dim_date[date]` inside `DATEADD`.<br>• Expecting the SQL query 01 numbers to match for December 2016 (different rule). |
| **Used in** | visuals P1-V09 |

## Measure 5 of 43 · Latest Month Revenue MoM %

**1 · Formula**

```
Latest Month Revenue MoM % =
VAR _d = MAX ( vw_sales[order_date] )
VAR _ms = DATE ( YEAR ( _d ), MONTH ( _d ), 1 )
VAR _cur = CALCULATE ( [Total Revenue], dim_date[month_start] = _ms )
VAR _prev = CALCULATE ( [Total Revenue], REMOVEFILTERS ( dim_date ),
dim_date[month_start] = EDATE ( _ms, -1 ) )
RETURN
    DIVIDE ( _cur - _prev, _prev )
```

| | |
|---|---|
| **2 · Purpose** | One KPI figure: growth of the most recent month in the selection compared with the calendar month before it. |
| **3 · Line by line** | • `_d = MAX ( vw_sales[order_date] )` finds the latest order date that survives the filters.<br>• `_ms` rebuilds that date as the first of its month.<br>• `_cur` is revenue for that month; adding `dim_date[month_start] = _ms` narrows the existing context to the month.<br>• `_prev` is revenue for the month before: `REMOVEFILTERS ( dim_date )` clears every date filter first, then `EDATE ( _ms, -1 )` picks the prior month start.<br>• `DIVIDE` computes the growth. |
| **4 · Tables and columns** | `vw_sales[order_date]`, `dim_date[month_start]`, [Total Revenue]. |
| **5 · Numerator, denominator, population** | Numerator: current − prior. Denominator: prior-month revenue. |
| **6 · Filter context** | Evaluated on a card (no axis), so the context is only the slicers. Clearing the date filter for the prior month is deliberate: otherwise a date slicer that ends in the latest month would hide the month before. |
| **7 · Relationship and date behaviour** | Uses `dim_date[month_start]` (a calculated column) and the `order_date` relationship. If the date slicer cuts the latest month short, the current month is partial while the prior month is complete. |

| | |
|---|---|
| **8 · Format** | `0.0%;-0.0%;0.0%` · display folder: Revenue |
| **9 · Example check** | Last order date in the data is in August 2018: August R$985,491.64 vs July R$1,027,807.28 → **−4.12%**. |
| **10 · Common mistakes** | • Forgetting `REMOVEFILTERS` so the prior month disappears when a date range is chosen.<br>• Comparing a partial month with a full month (slicer range ends mid-month). |
| **Used in** | visuals P1-V07 |

## Measure 6 of 43 · Category Rank

**1 · Formula**

```
Category Rank =
RANKX (
    ALL ( vw_sales[product_category_name_english] ),
    CALCULATE ( [Total Revenue], REMOVEFILTERS ( dim_date[month_start] ) ),
    ,
    DESC,
    DENSE
)
```

| | |
|---|---|
| **2 · Purpose** | Ranks each product category by revenue, 1 = biggest. It is the engine behind the top-15 matrix and the Pareto chart. |
| **3 · Line by line** | • `RANKX ( table, expression, value, order, ties )` ranks the current item among a list.<br>• `ALL ( vw_sales[product_category_name_english] )` is the list: every category, ignoring the category filter that comes from the visual's row.<br>• `CALCULATE ( [Total Revenue], REMOVEFILTERS ( dim_date[month_start] ) )` is the score: revenue with the month column filter removed, so the rank reflects the whole selected period rather than one month.<br>• The two empty arguments skip the optional value; `DESC` ranks highest first; `DENSE` gives tied categories the same rank with no gaps. |
| **4 · Tables and columns** | `vw_sales[product_category_name_english]`, `dim_date[month_start]`, [Total Revenue]. |
| **5 · Numerator, denominator, population** | Not a ratio. Population: all 73 named categories plus one blank group (1,537 lines with no category, R$197,695.69). |
| **6 · Filter context** | `ALL` on one column clears only that column's filter, so state and the date slicer still apply: ranks are "top categories in the selected state and period". |
| **7 · Relationship and date behaviour** | The date slicer (on `dim_date[date]`) still applies; only the month column filter is removed. The matrix has months as columns, which is why the removal is essential. |
| **8 · Format** | `0` · display folder: Revenue |
| **9 · Example check** | health_beauty = 1 (R$1,412,089.53), watches_gifts = 2, bed_bath_table = 3, sports_leisure = 4, computers_accessories = 5; the 15th is office_furniture (R$335,211.36). The blank group ranks below the top 15. |
| **10 · Common mistakes** | • Omitting the inner `CALCULATE`: each month column would then re-rank categories within its own month.<br>• Using `ALL ( vw_sales )`: the state slicer would no longer affect the rank. |
| **Used in** | measures Top Category Revenue, Category Cumulative Share % |

| Measure 7 of 43 · Top Category Revenue | |
|---|---|
| 1 · Formula | `Top Category Revenue =`<br>`IF ( [Category Rank] <= 15, [Total Revenue] )` |
| 2 · Purpose | Shows revenue only for the 15 biggest categories so that the matrix and the Pareto bars list exactly 15. |
| 3 · Line by line | • `IF ( condition, value )` with no else branch returns blank when the condition is false.<br>• Blank rows are not drawn in a matrix or chart, which is how categories ranked 16+ disappear. |
| 4 · Tables and columns | Uses [Category Rank] and [Total Revenue]. |
| 5 · Numerator, denominator, population | Population: categories with rank ≤ 15; everything else is blank. |
| 6 · Filter context | The rank is computed per category row; the revenue is per cell (category × month in the matrix). |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| 8 · Format | `\R\$#,0.00` · display folder: Revenue |
| 9 · Example check | In the matrix the monthly cells of the health_beauty row add up to R$1,412,089.53. Exactly 15 category rows appear. |
| 10 · Common mistakes | • Hard-coding 15 as a visual-level "Top N" filter: that would rank inside each cell, not by whole-period revenue.<br>• Expecting the "Advanced" measure filter listed in the visual's filter pane to do the work: it holds no condition in the file; blank rows vanish by themselves. |
| Used in | visuals P1-V10, P2-V04 |

| Measure 8 of 43 · Category Cumulative Share % | |
|---|---|
| 1 · Formula | `Category Cumulative Share % =`<br>`VAR _rank = [Category Rank]`<br>`VAR _tot = CALCULATE ( [Total Revenue], REMOVEFILTERS`<br>`( vw_sales[product_category_name_english] ) )`<br>`RETURN`<br>`    IF (`<br>`        _rank <= 15,`<br>`        DIVIDE (`<br>`            SUMX ( FILTER ( ALL ( vw_sales[product_category_name_english] ),`<br>`[Category Rank] <= _rank ), [Total Revenue] ),`<br>`            _tot`<br>`        )`<br>`    )` |
| 2 · Purpose | The Pareto line: share of total revenue covered by this category and all bigger ones. |
| 3 · Line by line | • `_rank` is this category's rank.<br>• `_tot` is total revenue with the category filter removed (revenue of **all** categories).<br>• `FILTER ( ALL ( category ), [Category Rank] <= _rank )` selects this category and every category that outranks it.<br>• `SUMX ( …, [Total Revenue] )` adds their revenues; `DIVIDE` by `_tot` gives the cumulative share.<br>• `IF ( _rank <= 15, … )` limits the line to the 15 plotted categories. |
| 4 · Tables and columns | `vw_sales[product_category_name_english]`; [Category Rank]; [Total Revenue]. |
| 5 · Numerator, denominator, population | Numerator: revenue of categories ranked ≤ this one. Denominator: revenue of all categories (including the blank group). |

| | |
|---|---|
| **6 · Filter context** | Inside `SUMX` each category value becomes a row context; calling a measure there triggers **context transition**, turning the row into a filter so [Total Revenue] is that category's revenue. |
| **7 · Relationship and date behaviour** | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| **8 · Format** | `0.0%;-0.0%;0.0%` · display folder: Revenue |
| **9 · Example check** | Rank 1 → 9.16%; rank 2 → 17.36%; rank 5 → **39.25%**; rank 15 → **76.47%**. |
| **10 · Common mistakes** | • Dividing by the top-15 total: the line would end at 100%.<br>• Sorting the chart by category name; it must be sorted by revenue (the chart sorts by [Top Category Revenue] descending). |
| **Used in** | visuals P1-V10 |

| **Measure 9 of 43 · Top 5 Category Share %** | |
|---|---|
| **1 · Formula** | `Top 5 Category Share % =`<br>`VAR _t = TOPN ( 5, ALL ( vw_sales[product_category_name_english] ), [Total Revenue], DESC )`<br>`VAR _tot = CALCULATE ( [Total Revenue], REMOVEFILTERS ( vw_sales[product_category_name_english] ) )`<br>`RETURN`<br>`    DIVIDE ( SUMX ( _t, [Total Revenue] ), _tot )` |
| **2 · Purpose** | Share of revenue earned by the five biggest categories (a single headline for concentration). |
| **3 · Line by line** | • `TOPN ( 5, ALL ( category ), [Total Revenue], DESC )` returns a table of the five categories with highest revenue.<br>• `_tot` is revenue of all categories.<br>• `SUMX ( _t, [Total Revenue] )` adds the five revenues; `DIVIDE` gives the share. |
| **4 · Tables and columns** | `vw_sales[product_category_name_english]`. |
| **5 · Numerator, denominator, population** | Numerator: revenue of the five biggest categories. Denominator: revenue of all categories. |
| **6 · Filter context** | Variables are evaluated once, in the filter context where they are declared; the card has no axis so they reflect the slicers only. |
| **7 · Relationship and date behaviour** | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| **8 · Format** | `0.00%;-0.00%;0.00%` · display folder: Revenue |
| **9 · Example check** | **39.25%** with no slicers (9.16 + 8.20 + 7.95 + 7.25 + 6.70, rounded individually). |
| **10 · Common mistakes** | • Using `TOPN` without `ALL`: it would pick from the already filtered category list.<br>• Ties at 5th place: `TOPN` can return more than five rows. |
| **Used in** | not placed on a visual; kept for checking and for the figures quoted in Part VI |

| Measure 10 of 43 · Seller Rank | |
|---|---|
| 1 · Formula | `Seller Rank = RANKX ( ALL ( vw_sales[seller_id], vw_sales[seller_label] ), [Total Revenue], , DESC, DENSE )` |
| 2 · Purpose | Ranks sellers by revenue, 1 = biggest. |
| 3 · Line by line | • `ALL ( vw_sales[seller_id], vw_sales[seller_label] )` lists every seller; both columns are named because the bar chart uses `seller_label` on its axis.<br>• The score is [Total Revenue]; `DESC` and `DENSE` as before. |
| 4 · Tables and columns | `vw_sales[seller_id]`, `vw_sales[seller_label]`. |
| 5 · Numerator, denominator, population | Population: 2,970 sellers. |
| 6 · Filter context | Naming both columns in `ALL` clears both filters. With `ALL ( seller_id )` alone the axis filter on `seller_label` would remain and every bar would rank 1. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| 8 · Format | `0` · display folder: Revenue |
| 9 · Example check | Seller 4869f7a5... = 1 (R$247,007.06). `seller_id` and `seller_label` both have 2,970 distinct values, so the 8-character label does not collide. |
| 10 · Common mistakes | • Ranking by `seller_id` only (every bar becomes rank 1).<br>• Labels would collide if two sellers shared their first 8 characters; they do not in this data. |
| Used in | measures Top 10 Seller Revenue |

| Measure 11 of 43 · Top 10 Seller Revenue | |
|---|---|
| 1 · Formula | `Top 10 Seller Revenue = IF ( [Seller Rank] <= 10, [Total Revenue] )` |
| 2 · Purpose | Revenue for the ten biggest sellers only. |
| 3 · Line by line | • `IF ( [Seller Rank] <= 10, [Total Revenue] )` returns blank for rank 11 and above, so only ten bars are drawn. |
| 4 · Tables and columns | Uses [Seller Rank] and [Total Revenue]. |
| 5 · Numerator, denominator, population | Population: sellers with rank ≤ 10. |
| 6 · Filter context | Per seller (the axis). |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| 8 · Format | `\R\$#,0.00` · display folder: Revenue |
| 9 · Example check | The ten bars sum to R$1,993,154.99 (12.93% of revenue). Largest bar R$247,007.06. |
| 10 · Common mistakes | • Using a visual-level Top N filter: it ignores slicer changes in a different way and does not feed the share card. |
| Used in | visuals P2-V09 |

| Measure 12 of 43 · Top 10 Seller Share % | |
|---|---|
| 1 · Formula | `Top 10 Seller Share % =`<br>`VAR _t = TOPN ( 10, ALL ( vw_sales[seller_id] ), [Total Revenue], DESC )`<br>`VAR _tot = CALCULATE ( [Total Revenue], REMOVEFILTERS ( vw_sales[seller_id] ) )`<br>`RETURN`<br>`    DIVIDE ( SUMX ( _t, [Total Revenue] ), _tot )` |
| 2 · Purpose | Share of revenue earned by the ten biggest sellers. |
| 3 · Line by line | • `TOPN ( 10, ALL ( seller_id ), [Total Revenue], DESC )` selects ten sellers.<br>• `SUMX` adds their revenue; `DIVIDE` by revenue of all sellers (`REMOVEFILTERS ( seller_id )`). |
| 4 · Tables and columns | `vw_sales[seller_id]`. |
| 5 · Numerator, denominator, population | Numerator: revenue of ten biggest sellers. Denominator: all revenue. |
| 6 · Filter context | Reflects the slicers (state, category, date) because only the seller column is cleared. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_sales` through the `order_date` relationship, so the date slicer and a month on an axis restrict this measure. Category and state filters come straight from `vw_sales` columns. |
| 8 · Format | `0.00%;-0.00%;0.00%` · display folder: Revenue |
| 9 · Example check | 1,993,154.99 ÷ 15,419,773.75 = **12.93%**. |
| 10 · Common mistakes | • Reporting it as "seller concentration is low": 2,970 sellers share the rest; the measure only reports the top-10 share. |
| Used in | visuals P2-V08 |

### Display folder: Payments (all payment records)

| Measure 13 of 43 · Payment Records | |
|---|---|
| 1 · Formula | `Payment Records =`<br>`COUNTROWS ( order_payments )` |
| 2 · Purpose | Number of payment records, not orders. |
| 3 · Line by line | • `COUNTROWS ( order_payments )` counts rows of the payments table. |
| 4 · Tables and columns | `order_payments` (table). |
| 5 · Numerator, denominator, population | Population: every payment record, all order statuses. |
| 6 · Filter context | Filtered by `order_payments` columns only (payment type, instalment bucket). |
| 7 · Relationship and date behaviour | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| 8 · Format | `#,0` · display folder: Payments (all payment records) |
| 9 · Example check | **103,886** in total; credit_card 76,795; boleto 19,784; voucher 5,775; debit_card 1,529; not_defined 3. |
| 10 · Common mistakes | • Treating a payment record as an order: one order can have several payments.<br>• Comparing with delivered-order counts; the populations differ. |
| Used in | visuals P2-V05 |

| Measure 14 of 43 · Payment Record Share % | |
|---|---|
| 1 · Formula | `Payment Record Share % = DIVIDE ( COUNTROWS ( order_payments ), CALCULATE ( COUNTROWS ( order_payments ), ALL ( order_payments ) ) )` |
| 2 · Purpose | Share of payment records for each payment type or instalment bucket. |
| 3 · Line by line | • `COUNTROWS ( order_payments )` is the numerator for the current bar.<br>• `CALCULATE ( COUNTROWS ( order_payments ), ALL ( order_payments ) )` is the grand total with every filter on the table removed.<br>• `DIVIDE` returns the share. |
| 4 · Tables and columns | `order_payments` (all columns removed by `ALL`). |
| 5 · Numerator, denominator, population | Numerator: records in the bar. Denominator: all 103,886 records. |
| 6 · Filter context | `ALL ( order_payments )` removes every filter on the table, so each bar is a share of the whole, and the shares add up to 100%. |
| 7 · Relationship and date behaviour | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| 8 · Format | `0.00%;-0.00%;0.00%` · display folder: Payments (all payment records) |
| 9 · Example check | credit_card **73.92%**, boleto 19.04%, voucher 5.56%, debit_card 1.47%. One instalment: 52,546 ÷ 103,886 = 50.58%. |
| 10 · Common mistakes | • Using `ALLSELECTED` and getting shares that change with the selection. |
| Used in | visuals P2-V05, P2-V06 |

| Measure 15 of 43 · Avg Installments | |
|---|---|
| 1 · Formula | `Avg Installments = AVERAGE ( order_payments[payment_installments] )` |
| 2 · Purpose | Average number of instalments per payment record. |
| 3 · Line by line | • `AVERAGE` adds the column and divides by the number of non-blank rows. |
| 4 · Tables and columns | `order_payments[payment_installments]`. |
| 5 · Numerator, denominator, population | Population: payment records in the current bar. |
| 6 · Filter context | Used in tooltips on the payment-mix bar. |
| 7 · Relationship and date behaviour | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| 8 · Format | `0.00` · display folder: Payments (all payment records) |
| 9 · Example check | All records **2.85**; credit_card 3.51; every other type 1.00. Two records show 0 instalments. |
| 10 · Common mistakes | • Reading it as "instalments per order"; it averages over payment records. |
| Used in | visuals P2-V05 |

| Measure 16 of 43 · Total Payment Value | |
|---|---|
| 1 · Formula | `Total Payment Value =`<br>`SUM ( order_payments[payment_value] )` |
| 2 · Purpose | Money paid, summed over payment records. |
| 3 · Line by line | • `SUM` of the payment amount column. |
| 4 · Tables and columns | `order_payments[payment_value]`. |
| 5 · Numerator, denominator, population | Population: all payment records. |
| 6 · Filter context | Filtered only by payment-table columns. |
| 7 · Relationship and date behaviour | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| 8 · Format | `\R\$#,0.00` · display folder: Payments (all payment records) |
| 9 · Example check | **R$16,008,872.12** in total; credit_card R$12,542,084.19. |
| 10 · Common mistakes | • Expecting it to equal Total Revenue: payments cover every status, and 303 orders (0.30%) have item and payment totals that differ. |
| Used in | visuals P2-V05 |

### Display folder: Customers & Retention

| Measure 17 of 43 · Customers | |
|---|---|
| 1 · Formula | `Customers =`<br>`DISTINCTCOUNT ( vw_customers[customer_unique_id] )` |
| 2 · Purpose | Number of unique people with at least one delivered order. |
| 3 · Line by line | • `DISTINCTCOUNT` counts different `customer_unique_id` values. |
| 4 · Tables and columns | `vw_customers[customer_unique_id]` (hidden). |
| 5 · Numerator, denominator, population | Population: people (not order-scoped customer ids). |
| 6 · Filter context | No relationship, so no date slicer; the figure is all-time. |
| 7 · Relationship and date behaviour | No relationship; deliberately all-time. |
| 8 · Format | `#,0` · display folder: Customers & Retention |
| 9 · Example check | **93,358**. SQL: `SELECT count(*) FROM analytics.vw_customers;`. `customer_id` would count one per order (96,478 delivered), because it is created per order. |
| 10 · Common mistakes | • Counting `customer_id`, which overstates people. |
| Used in | visuals P3-V01, P3-V06; measures One-Time Customers |

| Measure 18 of 43 · Repeat Customers | |
|---|---|
| 1 · Formula | `Repeat Customers = CALCULATE ( DISTINCTCOUNT ( vw_customers[customer_unique_id] ), vw_customers[is_repeat] = TRUE () )` |
| 2 · Purpose | People who placed two or more delivered orders. |
| 3 · Line by line | • `CALCULATE` evaluates the distinct count with an extra filter.<br>• `vw_customers[is_repeat] = TRUE ()` keeps only people flagged as repeat. |
| 4 · Tables and columns | `vw_customers[customer_unique_id]`, `vw_customers[is_repeat]`. |
| 5 · Numerator, denominator, population | Population: people with `delivered_orders` ≥ 2. |
| 6 · Filter context | The filter argument adds to whatever context exists (for example the one-time/repeat bar). |
| 7 · Relationship and date behaviour | No relationship. |
| 8 · Format | `#,0` · display folder: Customers & Retention |
| 9 · Example check | **2,801**. |
| 10 · Common mistakes | • Comparing the Boolean column with the text "TRUE" or with a number; compare it with `TRUE ()`. |
| Used in | visuals P3-V02; measures One-Time Customers |

| Measure 19 of 43 · One-Time Customers | |
|---|---|
| 1 · Formula | `One-Time Customers = [Customers] - [Repeat Customers]` |
| 2 · Purpose | People with exactly one delivered order. |
| 3 · Line by line | • Subtracts the repeat count from the total count; both are measures evaluated in the same context. |
| 4 · Tables and columns | Uses [Customers] and [Repeat Customers]. |
| 5 · Numerator, denominator, population | Population: people minus repeat people. |
| 6 · Filter context | Same as the two measures it uses. |
| 7 · Relationship and date behaviour | No relationship. |
| 8 · Format | `#,0` · display folder: Customers & Retention |
| 9 · Example check | 93,358 − 2,801 = **90,557**. |
| 10 · Common mistakes | • Counting orders instead of people. |
| Used in | visuals P3-V03 |

| Measure 20 of 43 · Repeat Cust % | |
|---|---|
| 1 · Formula | `Repeat Cust % =`<br>`DIVIDE (`<br>`  CALCULATE (`<br>`  DISTINCTCOUNT ( vw_customers[customer_unique_id] ),`<br>`  vw_customers[is_repeat] = TRUE ()`<br>`  ),`<br>`  DISTINCTCOUNT ( vw_customers[customer_unique_id] )`<br>`)` |
| 2 · Purpose | Share of customers who are repeat buyers. |
| 3 · Line by line | • The numerator is the repeat count (see Repeat Customers).<br>• The denominator is all distinct people in the same context. |
| 4 · Tables and columns | `vw_customers[customer_unique_id]`, `vw_customers[is_repeat]`. |
| 5 · Numerator, denominator, population | Numerator: repeat people. Denominator: all people. |
| 6 · Filter context | All-time: no date/state/category filter reaches `vw_customers`. |
| 7 · Relationship and date behaviour | No relationship. |
| 8 · Format | `0.00%;-0.00%;0.00%` · display folder: Customers & Retention |
| 9 · Example check | 2,801 ÷ 93,358 = **3.00%**. |
| 10 · Common mistakes | • Describing it as "retention": it counts people who ever returned, not people who returned within a period. |
| Used in | visuals P3-V04 |

| Measure 21 of 43 · Cohort Active Customers | |
|---|---|
| 1 · Formula | `Cohort Active Customers =`<br>`SUM ( cohort_retention[active_customers] )` |
| 2 · Purpose | Raw count of people active in a cohort cell. |
| 3 · Line by line | • `SUM` adds `active_customers`. |
| 4 · Tables and columns | `cohort_retention[active_customers]`. |
| 5 · Numerator, denominator, population | Population: people active in the (cohort, month offset) cell. |
| 6 · Filter context | Matrix row = cohort, column = month offset. |
| 7 · Relationship and date behaviour | No relationship. |
| 8 · Format | `#,0` · display folder: Customers & Retention |
| 9 · Example check | Nov 2017 cohort, offset 0 = 7,060; offset 1 = 40. Total of all 219 cells = 95,194 (not a count of people, since people appear in several months). |
| 10 · Common mistakes | • Summing the grand total as if it were customers. |
| Used in | not placed on a visual; kept for checking and for the figures quoted in Part VI |

| Measure 22 of 43 · Retention % | |
|---|---|
| 1 · Formula | `Retention % = DIVIDE ( SUM ( cohort_retention[active_customers] ), CALCULATE ( SUM ( cohort_retention[active_customers] ), cohort_retention[month_offset] = 0 ) )` |
| 2 · Purpose | Share of a cohort still buying in a given month after its first purchase; fills the heat-map. |
| 3 · Line by line | • The numerator is the cell's active customers.<br>• The denominator is the same cohort's month-0 active customers: `month_offset = 0` replaces the column filter of the matrix, while the row's cohort filter stays. |
| 4 · Tables and columns | `cohort_retention[active_customers]`, `[month_offset]`, `[cohort]`. |
| 5 · Numerator, denominator, population | Numerator: active in the cell. Denominator: active in month 0 (cohort size). |
| 6 · Filter context | A filter argument on a column replaces existing filters on that same column, which is exactly what is wanted. |
| 7 · Relationship and date behaviour | No relationship. |
| 8 · Format | `0.0%;-0.0%;0.0%` · display folder: Customers & Retention |
| 9 · Example check | Nov 2017: month 0 = 100%; month 1 = 40 ÷ 7,060 = **0.57%**. |
| 10 · Common mistakes | • Dividing by the grand total instead of the cohort size.<br>• Showing totals/subtotals: they would be meaningless, so they are off. |
| Used in | visuals P3-V05 |

| Measure 23 of 43 · Month-1 Retention % (weighted) | |
|---|---|
| 1 · Formula | `Month-1 Retention % (weighted) = VAR _lastCohort = MAX ( cohort_retention[cohort] ) VAR _m1 = CALCULATE ( SUM ( cohort_retention[active_customers] ), cohort_retention[month_offset] = 1, cohort_retention[cohort] < _lastCohort ) VAR _m0 = CALCULATE ( SUM ( cohort_retention[active_customers] ), cohort_retention[month_offset] = 0, cohort_retention[cohort] < _lastCohort ) RETURN DIVIDE ( _m1, _m0 )` |
| 2 · Purpose | One headline: of everyone who first bought in a month, how many bought again the next month, pooled across cohorts. |
| 3 · Line by line | • `_lastCohort` is the newest cohort (Aug 2018).<br>• `_m1` sums month-1 activity and `_m0` sums month-0 activity, both only for cohorts older than `_lastCohort`; the newest has no month 1 yet.<br>• `DIVIDE ( _m1, _m0 )`. |
| 4 · Tables and columns | `cohort_retention[cohort]`, `[month_offset]`, `[active_customers]`. |
| 5 · Numerator, denominator, population | Numerator: 421 people. Denominator: 87,214 people (cohort sizes of the 22 older cohorts). |
| 6 · Filter context | Weighted means large cohorts count more. The newest cohort is excluded from both sides. |
| 7 · Relationship and date behaviour | No relationship. |

| | |
|---|---|
| **8 · Format** | `0.00%;-0.00%;0.00%` · display folder: Customers & Retention |
| **9 · Example check** | 421 ÷ 87,214 = **0.48%**. An unweighted mean of cohort percentages would be distorted by the December 2016 cohort (1 person, 100%). |
| **10 · Common mistakes** | • Averaging the cohort percentages.<br>• Including the newest cohort in the denominator. |
| **Used in** | visuals P3-V07 |

## Measure 24 of 43 · RFM Customers

| | |
|---|---|
| **1 · Formula** | `RFM Customers =`<br>`SUM ( rfm_grid[customers] )` |
| **2 · Purpose** | People in an RFM segment. |
| **3 · Line by line** | • `SUM` of the grid's `customers` column. |
| **4 · Tables and columns** | `rfm_grid[customers]`. |
| **5 · Numerator, denominator, population** | Population: 15 score cells that roll up into 4 segments. |
| **6 · Filter context** | Row = segment. |
| **7 · Relationship and date behaviour** | No relationship. |
| **8 · Format** | `#,0` · display folder: Customers & Retention |
| **9 · Example check** | Total **93,358**; Lapsed One-Time 54,419; Recent One-Time 36,138; At-Risk Repeat 1,597; Champions 1,204. |
| **10 · Common mistakes** | • Expecting this to change with slicers; page 3 has none. |
| **Used in** | visuals P3-V09 |

## Measure 25 of 43 · RFM Revenue

| | |
|---|---|
| **1 · Formula** | `RFM Revenue =`<br>`SUM ( rfm_grid[segment_revenue] )` |
| **2 · Purpose** | Revenue of the people in a segment. |
| **3 · Line by line** | • `SUM` of `segment_revenue`. |
| **4 · Tables and columns** | `rfm_grid[segment_revenue]`. |
| **5 · Numerator, denominator, population** | Population: all delivered revenue split by segment. |
| **6 · Filter context** | Row = segment. |
| **7 · Relationship and date behaviour** | No relationship. |
| **8 · Format** | `\R\$#,0.00` · display folder: Customers & Retention |
| **9 · Example check** | Total **R$15,419,773.75**; Lapsed One-Time R$8,617,203.27; Champions R$388,361.48. |
| **10 · Common mistakes** | • Comparing with a date-filtered Total Revenue; this measure is all-time. |
| **Used in** | visuals P3-V09 |

| Measure 26 of 43 · RFM Customer Share % | |
|---|---|
| 1 · Formula | `RFM Customer Share % = DIVIDE ( SUM ( rfm_grid[customers] ), CALCULATE ( SUM ( rfm_grid[customers] ), ALL ( rfm_grid ) ) )` |
| 2 · Purpose | Each segment's share of customers. |
| 3 · Line by line | • Numerator is the segment's customers; the denominator is the sum over `ALL ( rfm_grid )`. |
| 4 · Tables and columns | `rfm_grid[customers]`. |
| 5 · Numerator, denominator, population | Numerator: segment customers. Denominator: 93,358. |
| 6 · Filter context | `ALL` removes the segment filter from the denominator. |
| 7 · Relationship and date behaviour | No relationship. |
| 8 · Format | `0.00%;-0.00%;0.00%` · display folder: Customers & Retention |
| 9 · Example check | Lapsed One-Time **58.29%**; Recent One-Time 38.71%; At-Risk Repeat 1.71%; Champions 1.29%. |
| 10 · Common mistakes | • Using `ALLSELECTED` and getting 100% when only one bar is selected. |
| Used in | visuals P3-V08 |

| Measure 27 of 43 · RFM Revenue Share % | |
|---|---|
| 1 · Formula | `RFM Revenue Share % = DIVIDE ( SUM ( rfm_grid[segment_revenue] ), CALCULATE ( SUM ( rfm_grid[segment_revenue] ), ALL ( rfm_grid ) ) )` |
| 2 · Purpose | Each segment's share of revenue. |
| 3 · Line by line | • Same structure as the customer share, using `segment_revenue`. |
| 4 · Tables and columns | `rfm_grid[segment_revenue]`. |
| 5 · Numerator, denominator, population | Numerator: segment revenue. Denominator: R$15,419,773.75. |
| 6 · Filter context | `ALL ( rfm_grid )` in the denominator. |
| 7 · Relationship and date behaviour | No relationship. |
| 8 · Format | `0.00%;-0.00%;0.00%` · display folder: Customers & Retention |
| 9 · Example check | Lapsed One-Time **55.88%**; Recent One-Time 38.51%; At-Risk Repeat 3.09%; Champions **2.52%**. |
| 10 · Common mistakes | • Quoting 2.51% / 55.89%: those came from an older notebook method that split boundary-date ties differently (IV.8.2); the dashboard and all current documents use 2.52% and 55.88%. |
| Used in | visuals P3-V09 |

### Display folder: Operations & Satisfaction

| Measure 28 of 43 · Delivered Orders | |
|---|---|
| 1 · Formula | `Delivered Orders = COUNTROWS ( vw_orders )` |
| 2 · Purpose | Count of delivered orders in `vw_orders`. |
| 3 · Line by line | • `COUNTROWS ( vw_orders )`; `vw_orders` has one row per delivered order. |
| 4 · Tables and columns | `vw_orders` (table). |
| 5 · Numerator, denominator, population | Population: delivered orders surviving the filters. |
| 6 · Filter context | Date slicer and `vw_orders[customer_state]` apply. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| 8 · Format | `#,0` · display folder: Operations & Satisfaction |
| 9 · Example check | **96,478**. Equals Total Orders when no slicer is applied. |
| 10 · Common mistakes | • Using the page-1 state slicer on page 4: it filters `vw_sales`, not `vw_orders`. |
| Used in | visuals P4-V06 |

| Measure 29 of 43 · Late Orders | |
|---|---|
| 1 · Formula | `Late Orders = CALCULATE ( COUNTROWS ( vw_orders ), vw_orders[is_late] = TRUE () )` |
| 2 · Purpose | Delivered orders that arrived after the promised date. |
| 3 · Line by line | • `CALCULATE` counts rows after adding the filter `is_late = TRUE ()`.<br>• Orders whose lateness is blank (no delivery date) are not true, so they are not counted. |
| 4 · Tables and columns | `vw_orders[is_late]`. |
| 5 · Numerator, denominator, population | Population: delivered orders flagged late. |
| 6 · Filter context | Filter adds to the page context. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| 8 · Format | `#,0` · display folder: Operations & Satisfaction |
| 9 · Example check | **7,826**. |
| 10 · Common mistakes | • Filtering `is_late = FALSE ()` to get on-time orders: a blank matches too, so the 8 Unknown orders would be counted as on time (see the BLANK edge case). |
| Used in | visuals P4-V05 |

| Measure 30 of 43 · Late Delivery % | |
|---|---|
| 1 · Formula | `Late Delivery % =`<br>`DIVIDE (`<br>`  CALCULATE ( COUNTROWS ( vw_orders ), vw_orders[is_late] = TRUE () ),`<br>`  COUNTROWS ( vw_orders )`<br>`)` |
| 2 · Purpose | Share of delivered orders that were late. |
| 3 · Line by line | • The numerator repeats Late Orders.<br>• The denominator is every delivered order, including the 8 whose lateness is unknown. |
| 4 · Tables and columns | `vw_orders[is_late]`. |
| 5 · Numerator, denominator, population | Numerator: 7,826. Denominator: 96,478. |
| 6 · Filter context | Both parts react to the same slicers. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| 8 · Format | `0.00%;-0.00%;0.00%` · display folder: Operations & Satisfaction |
| 9 · Example check | 7,826 ÷ 96,478 = **8.11%** (7,826 ÷ 96,470 would also give 8.11%). |
| 10 · Common mistakes | • Excluding Unknown orders silently; if you do, say so.<br>• Reading the 100% / 0% points of 2016 as real: September and December 2016 have one order each. |
| Used in | visuals P4-V04, P4-V08 |

| Measure 31 of 43 · Median Delivery Days | |
|---|---|
| 1 · Formula | `Median Delivery Days =`<br>`MEDIAN ( vw_orders[delivery_days] )` |
| 2 · Purpose | Typical delivery time, robust to a long tail. |
| 3 · Line by line | • `MEDIAN` sorts the values and returns the middle one; blanks are ignored. |
| 4 · Tables and columns | `vw_orders[delivery_days]`. |
| 5 · Numerator, denominator, population | Population: 96,470 orders with a delivery date. |
| 6 · Filter context | Per slicers. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| 8 · Format | `0.0` · display folder: Operations & Satisfaction |
| 9 · Example check | **10** days (shown as 10.0). The mean is 12.09 and the most common value is 7 days (7,651 orders). |
| 10 · Common mistakes | • Reporting the mean for a skewed distribution. |
| Used in | visuals P4-V03 |

| Measure 32 of 43 · Avg Review Score | |
|---|---|
| 1 · Formula | `Avg Review Score = AVERAGE ( vw_orders[review_score] )` |
| 2 · Purpose | Mean star rating of reviewed orders. |
| 3 · Line by line | • `AVERAGE` ignores blank scores automatically. |
| 4 · Tables and columns | `vw_orders[review_score]`. |
| 5 · Numerator, denominator, population | Population: 95,832 reviewed delivered orders. |
| 6 · Filter context | Per slicers or per delivery status. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| 8 · Format | `0.00` · display folder: Operations & Satisfaction |
| 9 · Example check | Overall **4.16**; Late 2.57 (2.5651); On time 4.29 (4.2941). |
| 10 · Common mistakes | • Replacing blanks with 0 before averaging. |
| Used in | visuals P4-V09, P4-V11 |

| Measure 33 of 43 · Median Review Score | |
|---|---|
| 1 · Formula | `Median Review Score = MEDIAN ( vw_orders[review_score] )` |
| 2 · Purpose | Middle star rating of reviewed orders. |
| 3 · Line by line | • `MEDIAN` over the same column. |
| 4 · Tables and columns | `vw_orders[review_score]`. |
| 5 · Numerator, denominator, population | Population: 95,832 reviewed delivered orders. |
| 6 · Filter context | Per slicers or per delivery status. |
| 7 · Relationship and date behaviour | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| 8 · Format | `0.0` · display folder: Operations & Satisfaction |
| 9 · Example check | Overall **5**; Late 2; On time 5. |
| 10 · Common mistakes | • Reading a median of 5 as "everyone gave 5": it only says at least half did. |
| Used in | visuals P4-V09, P4-V11 |

| Measure 34 of 43 · Reviewed Orders | |
|---|---|
| 1 · Formula | `Reviewed Orders = CALCULATE ( COUNTROWS ( vw_orders ), NOT ISBLANK ( vw_orders[review_score] ) )` |
| 2 · Purpose | Delivered orders that have a review. |
| 3 · Line by line | • `NOT ISBLANK ( review_score )` keeps rows with a score. |

| | |
|---|---|
| **4 · Tables and columns** | `vw_orders[review_score]`. |
| **5 · Numerator, denominator, population** | Population: delivered orders with a review. |
| **6 · Filter context** | Per slicers or delivery status. |
| **7 · Relationship and date behaviour** | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| **8 · Format** | `#,0` · display folder: Operations & Satisfaction |
| **9 · Example check** | **95,832**; 646 delivered orders have no review. By status: Late 7,661; On time 88,163; Unknown 8. |
| **10 · Common mistakes** | • Using `COUNT` of the table, which includes unreviewed orders. |
| **Used in** | visuals P4-V11 |

## Measure 35 of 43 · Pct 1-2 Star

**1 · Formula**

```
Pct 1-2 Star =
DIVIDE (
    CALCULATE ( COUNTROWS ( vw_orders ), vw_orders[review_score] IN { 1,
2 } ),
    CALCULATE ( COUNTROWS ( vw_orders ), NOT ISBLANK
( vw_orders[review_score] ) )
)
```

| | |
|---|---|
| **2 · Purpose** | Share of reviewed orders rated 1 or 2 stars. |
| **3 · Line by line** | • The numerator counts reviewed orders whose score is in the list `{ 1, 2 }`.<br>• The denominator counts orders that have any score. |
| **4 · Tables and columns** | `vw_orders[review_score]`. |
| **5 · Numerator, denominator, population** | Numerator: orders with 1–2 stars. Denominator: reviewed orders. |
| **6 · Filter context** | `IN { 1, 2 }` matches only real scores. The simpler `review_score <= 2` also matches blank, because DAX treats a blank as 0 in numeric comparisons, which would count the 646 unreviewed orders. |
| **7 · Relationship and date behaviour** | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| **8 · Format** | `0.00%;-0.00%;0.00%` · display folder: Operations & Satisfaction |
| **9 · Example check** | Overall **12.81%** (12,273 ÷ 95,832); Late 54.07%; On time 9.22%. |
| **10 · Common mistakes** | • Writing `<= 2` (the original spec formula) and inflating the share. |
| **Used in** | visuals P4-V10, P4-V11 |

## Measure 36 of 43 · Orders With Delivery Time

**1 · Formula**

```
Orders With Delivery Time =
CALCULATE ( COUNTROWS ( vw_orders ), NOT ISBLANK ( vw_orders[delivery_days] )
)
```

| | |
|---|---|
| **2 · Purpose** | Feeds the delivery-time histogram: orders that have a delivery duration. |
| **3 · Line by line** | • Counts rows where `delivery_days` is not blank. |

| | |
|---|---|
| **4 · Tables and columns** | `vw_orders[delivery_days]`. |
| **5 · Numerator, denominator, population** | Population: delivered orders with a delivery date. |
| **6 · Filter context** | Axis = `delivery_days_capped` (0 to 60). |
| **7 · Relationship and date behaviour** | Filters travel from `dim_date` to `vw_orders` through `order_date`, so the date slicer and a month on an axis restrict this measure. State comes from `vw_orders[customer_state]`. |
| **8 · Format** | `#,0` · display folder: Operations & Satisfaction |
| **9 · Example check** | Total across the axis **96,470**; the 60-day bar holds 306 orders (60 days or more); the tallest bar is 7 days with 7,651 orders. |
| **10 · Common mistakes** | • Using [Delivered Orders]: it includes 8 orders with no delivery time. |
| **Used in** | visuals P4-V07 |

### Display folder: Forecast (all placed orders)

| **Measure 37 of 43 · Forecast Total (28 days)** | |
|---|---|
| **1 · Formula** | `Forecast Total (28 days) = SUM ( forecast_28d[forecast] )` |
| **2 · Purpose** | Total forecast orders for the next 28 days. |
| **3 · Line by line** | • `SUM` of the daily forecast column. |
| **4 · Tables and columns** | `forecast_28d[forecast]`. |
| **5 · Numerator, denominator, population** | Population: 28 forecast days (2018-08-23 to 2018-09-19). |
| **6 · Filter context** | Not filtered by any slicer. |
| **7 · Relationship and date behaviour** | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| **8 · Format** | `#,0` · display folder: Forecast (all placed orders) |
| **9 · Example check** | **6,660**. |
| **10 · Common mistakes** | • Reading the total as a 28-day expected count with a tight range; see the next two measures. |
| **Used in** | visuals P4-V13 |

| **Measure 38 of 43 · Forecast Lower Sum** | |
|---|---|
| **1 · Formula** | `Forecast Lower Sum = SUM ( forecast_28d[lower] )` |
| **2 · Purpose** | Sum of the 28 daily lower bounds. |
| **3 · Line by line** | • `SUM` of `lower`. |
| **4 · Tables and columns** | `forecast_28d[lower]`. |
| **5 · Numerator, denominator, population** | Population: 28 daily bounds. |

| | |
|---|---|
| **6 · Filter context** | None. |
| **7 · Relationship and date behaviour** | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| **8 · Format** | `#,0` · display folder: Forecast (all placed orders) |
| **9 · Example check** | **3,585** (3,584.9, displayed with no decimals). |
| **10 · Common mistakes** | • Presenting it as the lower end of a 28-day interval: adding daily bounds does not give an interval for the total. |
| **Used in** | visuals P4-V14 |

| **Measure 39 of 43 · Forecast Upper Sum** | |
|---|---|
| **1 · Formula** | `Forecast Upper Sum =`<br>`SUM ( forecast_28d[upper] )` |
| **2 · Purpose** | Sum of the 28 daily upper bounds. |
| **3 · Line by line** | • `SUM` of `upper`. |
| **4 · Tables and columns** | `forecast_28d[upper]`. |
| **5 · Numerator, denominator, population** | Population: 28 daily bounds. |
| **6 · Filter context** | None. |
| **7 · Relationship and date behaviour** | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| **8 · Format** | `#,0` · display folder: Forecast (all placed orders) |
| **9 · Example check** | **9,943** (9,943.0). |
| **10 · Common mistakes** | • Same as the lower sum: not a 28-day interval. |
| **Used in** | visuals P4-V15 |

| **Measure 40 of 43 · Actual (last 90 d)** | |
|---|---|
| **1 · Formula** | `Actual (last 90 d) =`<br>`SUM ( forecast_chart[actual] )` |
| **2 · Purpose** | Daily actual orders for the 90 days before the forecast starts. |
| **3 · Line by line** | • `SUM` of `forecast_chart[actual]`; on a single date it is that day's value; on forecast days it is blank. |
| **4 · Tables and columns** | `forecast_chart[actual]`. |
| **5 · Numerator, denominator, population** | Population: 90 days (2018-05-25 to 2018-08-22), 19,324 orders in total. |
| **6 · Filter context** | Axis = `forecast_chart[date]`. |
| **7 · Relationship and date behaviour** | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |

| | |
|---|---|
| **8 · Format** | `#,0` · display folder: Forecast (all placed orders) |
| **9 · Example check** | Sum of all 90 points = **19,324**. Never show it as a card: the sum has no business meaning. |
| **10 · Common mistakes** | • Dropping it on a card and reading a total. |
| **Used in** | visuals P4-V12 |

| **Measure 41 of 43 · Forecast** | |
|---|---|
| **1 · Formula** | `Forecast =`<br>`SUM ( forecast_chart[forecast] )` |
| **2 · Purpose** | Daily forecast orders for the chart. |
| **3 · Line by line** | • `SUM` of `forecast_chart[forecast]`; blank on actual days. |
| **4 · Tables and columns** | `forecast_chart[forecast]`. |
| **5 · Numerator, denominator, population** | Population: 28 days. |
| **6 · Filter context** | Axis = `forecast_chart[date]`. |
| **7 · Relationship and date behaviour** | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| **8 · Format** | `#,0.0` · display folder: Forecast (all placed orders) |
| **9 · Example check** | Sum of 28 points = 6,660. |
| **10 · Common mistakes** | • Using `forecast_28d` for the chart: actual and forecast then sit in two tables with no shared axis. |
| **Used in** | visuals P4-V12 |

| **Measure 42 of 43 · Lower 95%** | |
|---|---|
| **1 · Formula** | `Lower 95% =`<br>`SUM ( forecast_chart[lower] )` |
| **2 · Purpose** | Daily lower bound for the chart. |
| **3 · Line by line** | • `SUM` of `forecast_chart[lower]`. |
| **4 · Tables and columns** | `forecast_chart[lower]`. |
| **5 · Numerator, denominator, population** | Population: 28 days. |
| **6 · Filter context** | Axis = `forecast_chart[date]`. |
| **7 · Relationship and date behaviour** | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| **8 · Format** | `#,0.0` · display folder: Forecast (all placed orders) |
| **9 · Example check** | Per-day values; their sum is 3,584.9 (not an interval). |
| **10 · Common mistakes** | • Reading the dashed lines as certainty bands for the 28-day total. |
| **Used in** | visuals P4-V12 |

| Measure 43 of 43 · Upper 95% | |
|---|---|
| 1 · Formula | `Upper 95% = SUM ( forecast_chart[upper] )` |
| 2 · Purpose | Daily upper bound for the chart. |
| 3 · Line by line | • `SUM` of `forecast_chart[upper]`. |
| 4 · Tables and columns | `forecast_chart[upper]`. |
| 5 · Numerator, denominator, population | Population: 28 days. |
| 6 · Filter context | Axis = `forecast_chart[date]`. |
| 7 · Relationship and date behaviour | No relationship: nothing in the model filters this table, so date, state and category slicers have no effect. |
| 8 · Format | `#,0.0` · display folder: Forecast (all placed orders) |
| 9 · Example check | Per-day values; their sum is 9,943.0 (not an interval). |
| 10 · Common mistakes | • As above. |
| Used in | visuals P4-V12 |

VERIFICATION STATUS

**File-verified.** All 43 formulas, display folders and number formats were read from the final `.pbix`. The example values were recomputed in SQL from the warehouse built by the pipeline; they are what the measure should show with no slicer applied. The values on the report's cards, bars and table were also viewed in Power BI Desktop (cached data, no refresh) and agree.

# V.F Build the four report pages

## V.F.1 The style kit and the recipe pattern

Everything on the four pages follows one style kit, so once you have built one visual you know the formatting of all of them. The values below are stored in the report file.

| Element | Value |
|---|---|
| Page size | 1280 px wide; height 800 (page 1), 960 (page 2), 900 (page 3), 800 (page 4); type Custom; view "Fit to page" |
| Page background | `#F3F5F8` (light grey-blue) |
| Theme | Base theme "Fluent2-CY26SU09" (the default theme of Power BI Desktop 2026.09); all colours below are set on the visuals themselves |
| Fonts | Segoe UI throughout; card values Segoe UI Semibold |
| Container | White fill, border `#D9DEE5`, corner radius 6, no shadow; title bold `#1F2937` 11 pt (cards 10 pt); subtitle `#6B7280` 9 pt |
| Palette | Navy `#1F3A5F` · teal `#2A9D8F` · gold `#E9C46A` · coral `#E76F51` · grey `#8D99AE`; axis text `#6B7280`; gridlines `#E5E7EB` |
| Page grid | Left and right margin 24 px; content width 1232 px; gutter 8–12 px |
| Title strip | Title text box at (24, 10), 1232 × 34, 22 pt bold navy; subtitle text box at (24, 44), 1232 × 24, 11 pt grey |

**The recipe pattern.** Every visual card below lists: type, position and size, the question it answers, the field wells, the stored formatting, sort and filters, the expected result, the alt text and the common errors. Build each visual the same way:

1. Click an empty area of the canvas, then click the visual type in the **Visualizations** pane.
2. Drag the fields listed under **Fields** into the wells named there (the well names are the Power BI names for that visual type).
3. Open **Format visual → General → Properties** and enter **Size** (width, height) and **Position** (horizontal = x, vertical = y) from the recipe.
4. Apply the **Formatting** list: General (title, subtitle, effects, header icons) and Visual (axes, labels, legend, colours). The mapping to the Format pane is in the table below.
5. Apply the sort from the visual's "…" menu if the recipe lists one.
6. Type the **Alt text** into **Format visual → General → Alt text**.
7. Compare with **Expected result** before starting the next visual.

| Stored property | Where it is in the Format pane (menu names can differ slightly between releases) |
|---|---|
| title, subtitle | Format visual → General → Title (Subtitle sits inside Title) |
| background, border, radius, shadow | General → Effects → Background / Visual border / Shadow |
| visual header icons | General → Header icons → Show/Hide |
| category and value axes, gridlines, display units | Visual → X axis / Y axis (for bar charts the horizontal axis is the value axis) |
| data labels | Visual → Data labels |
| legend | Visual → Legend |
| line width, markers, dash | Visual → Lines (and Shapes for markers) |
| card value, category label | Visual → Callout value / Category label |
| slicer style | Visual → Slicer settings → Options → Style |
| matrix fonts, padding, subtotals, colour scale | Visual → Values / Column headers / Row headers / Grid / Row subtotals / Column subtotals; Values → Background colour → fx for the gradient |
| alt text | General → Alt text (250 characters) |
| tab order | View → Selection pane → Tab order |

VERIFICATION STATUS

**Reviewed.** Position/size, alt text, titles, colours, axis limits, labels, sorts and the two colour scales are File-verified from the stored visual definitions. The Format-pane paths are from Microsoft Learn and Desktop help; menu names can move. Two differences between the stored properties and the screenshots are disclosed in V.I.

## V.F.1 Page 1 · Executive Overview

The executive summary: a reader should know in ten seconds how much revenue there is, how many orders, what an order is worth and whether the latest month grew; the charts then show growth over time, category concentration and the geography of sales.

| | |
|---|---|
| **Canvas** | 1280 × 800 px · Custom · Fit to page · background `#F3F5F8` |
| **Visuals** | 11 visuals and 4 text boxes |
| **Suggested build order** | Slicers (date, state, category) → four KPI cards → two time charts → Pareto and state chart. |

| STEP F.1.0 · Set up the page canvas | |
|---|---|
| **Where** | Power BI Desktop, Report view. For pages 2–4 first click the **+** at the bottom to add a page, then rename it (double-click its tab). |
| **Action** | Click an empty part of the page → **Format page** (paint-roller icon) → **Canvas settings** → **Type: Custom**, **Width 1280**, **Height 800**. Then **Canvas background → Colour** `#F3F5F8`, transparency 0%. **View → Page view → Fit to page**. Name the page “Executive Overview”. |
| **Purpose** | A fixed canvas makes every position in the recipes land where it should. |
| **Expected result** | A pale grey empty page with a 1280-wide canvas. |
| **Verification** | Format page → Canvas settings shows Custom, 1280 × 800. |
| **Troubleshooting** | If the page is a standard 16:9 size, the Type is still “16:9”; change it to Custom. |
| **Next step** | The blueprint, then the visuals. |

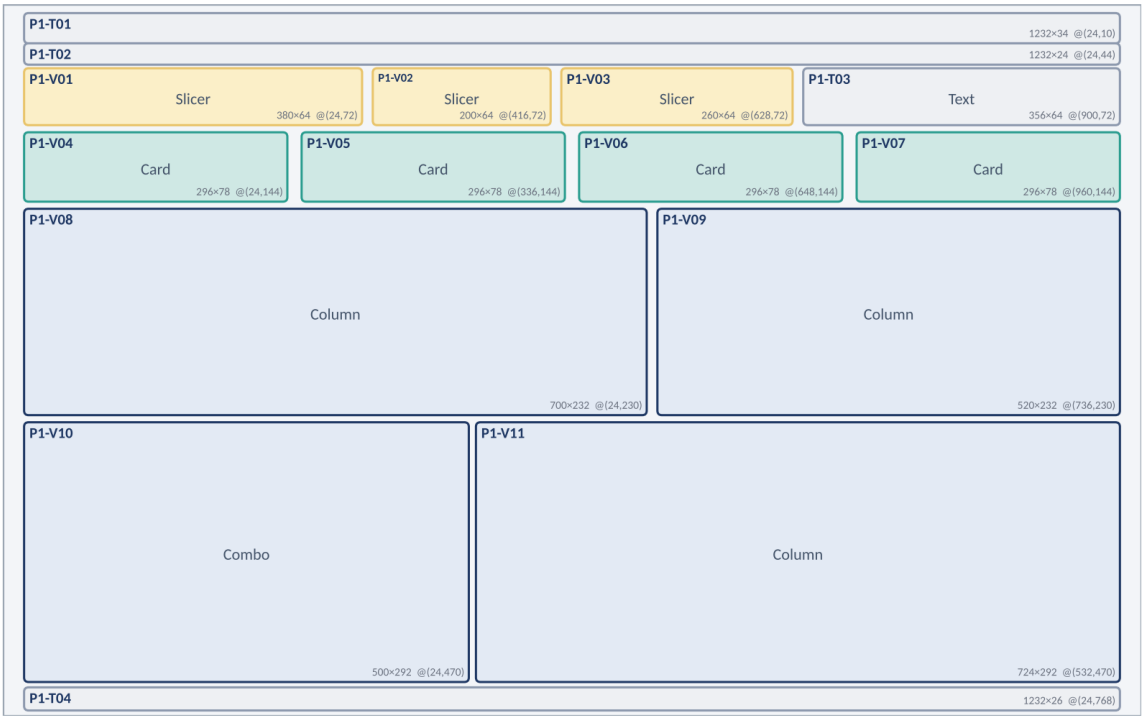



**Figure 7.** Layout blueprint of page 1 drawn from the stored positions; each box shows its ID, type, size and position (x, y).

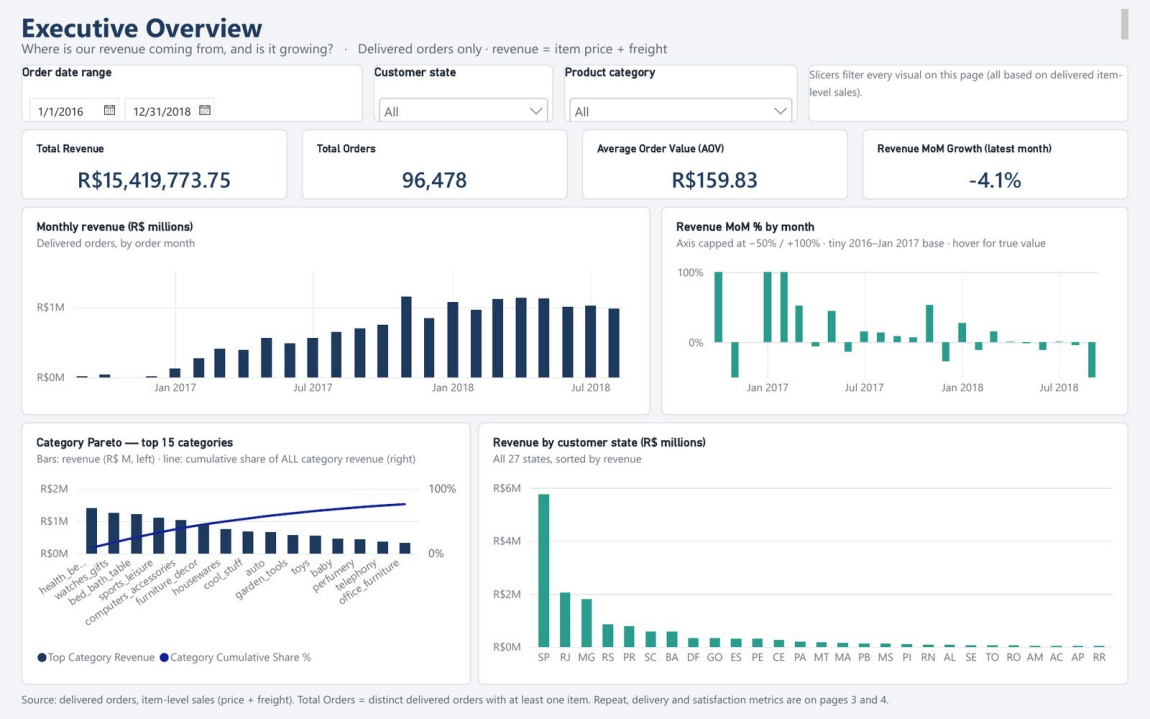



**Figure 8.** Page 1 as built. Page 1 of the saved final report, exported with File → Export → PDF (cropped to the canvas).

## Text boxes on page 1

| ID | x,y · size | Text (type it exactly) |
|---|---|---|
| P1-T01 | 24,10 · 1232×34 | Executive Overview |
| P1-T02 | 24,44 · 1232×24 | Where is our revenue coming from, and is it growing? · Delivered orders only · revenue = item price + freight |
| P1-T03 | 900,72 · 356×64 | Slicers filter every visual on this page (all based on delivered item-level sales). |
| P1-T04 | 24,768 · 1232×26 | Source: delivered orders, item-level sales (price + freight). Total Orders = distinct delivered orders with at least one item. Repeat, delivery and satisfaction metrics are on pages 3 and 4. |

Text boxes: insert with **Insert → Text box**, type the text, set Segoe UI and the sizes listed in V.F.1 (headline 22 pt bold `#1F3A5F`; subtitle 11 pt `#5B6472`; page footnotes 9 pt `#5B6472`; the boxed slicer notes 9 pt `#6B7280` on white). P3-T04 is bold 10 pt `#1F3A5F`; P4-T04 is 9 pt and P4-T05 is 8 pt in `#1F2937`. Turn the text box background and border off.

## Visual recipes · page 1

| P1-V01 · Slicer · Order date range | |
|---|---|
| **Visual type** | **Slicer** |
| **Position and size** | x 24, y 72, width 380, height 64 (pixels on the page canvas); reading/tab order position 3 of 15 on the page |
| **Question answered** | Which order dates should this page cover? |
| **Fields** | • **Field:** `dim_date[date]` |
| **Formatting (stored)** | • **Title:** “Order date range”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Slicer settings:** style **Between**; slicer header off (the container title is used); “Select all” on; multi-select on. |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | A date range control spanning 1 Jan 2016 to 31 Dec 2018. Dragging an end narrows every chart and card on the page. |
| **Alt text** | “Date range slicer. Filters every chart on this page by order date.” |
| **Common errors** | • Using a date column of a fact table (`vw_sales[order_date]`): it filters only that table. Use `dim_date[date]`.<br>• Leaving the style as a list of dates; choose **Between**. |

| **P1-V02 · Slicer · Customer state** | |
|---|---|
| **Visual type** | **Slicer** |
| **Position and size** | x 416, y 72, width 200, height 64 (pixels on the page canvas); reading/tab order position 4 of 15 on the page |
| **Question answered** | Which customer state should the page show? |
| **Fields** | • **Field:** `vw_sales[customer_state]` |
| **Formatting (stored)** | • **Title:** “Customer state”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Slicer settings:** style **Dropdown**; slicer header off (the container title is used); “Select all” on; multi-select on. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | A dropdown listing the 27 Brazilian states with a “Select all” option. Choosing SP changes Total Revenue to R$5,769,703.15. |
| **Alt text** | “Customer state slicer. Filters every chart on this page by the buyer's state.” |
| **Common errors** | • Using `vw_orders[customer_state]`: it would filter a table that no page-1 visual reads. |

| **P1-V03 · Slicer · Product category** | |
|---|---|
| **Visual type** | **Slicer** |
| **Position and size** | x 628, y 72, width 260, height 64 (pixels on the page canvas); reading/tab order position 5 of 15 on the page |
| **Question answered** | Which product category should the page show? |
| **Fields** | • **Field:** `vw_sales[product_category_name_english]` |
| **Formatting (stored)** | • **Title:** “Product category”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Slicer settings:** style **Dropdown**; slicer header off (the container title is used); “Select all” on; multi-select on. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | A dropdown of 73 category names plus “(Blank)” for the 1,537 lines without a category. |
| **Alt text** | “Product category slicer. Filters every chart on this page by product category.” |
| **Common errors** | • Using the Portuguese category column; the report uses the English name column. |

| P1-V04 · Card · Total Revenue | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 24, y 144, width 296, height 78 (pixels on the page canvas); reading/tab order position 7 of 15 on the page |
| **Question answered** | How much revenue did delivered orders generate? |
| **Fields** | • **Fields:** `_Measures[Total Revenue] (measure)` |
| **Formatting (stored)** | • **Title:** "Total Revenue", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it).<br>• **Padding (General → Properties → Padding):** bottom **8 px**; top 14, left 16, right 16 stay at their defaults. Without it the bottom of the 18 pt value is clipped in this 78 px card and its commas render like dots (V.I.1). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | R$15,419,773.75. |
| **Alt text** | "Card: total delivered-order revenue (item price plus freight) for the current selection." |
| **Common errors** | • Showing an implicit sum of `line_revenue` instead of the measure. |

| P1-V05 · Card · Total Orders | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 336, y 144, width 296, height 78 (pixels on the page canvas); reading/tab order position 8 of 15 on the page |
| **Question answered** | How many delivered orders are there? |
| **Fields** | • **Fields:** `_Measures[Total Orders] (measure)` |
| **Formatting (stored)** | • **Title:** "Total Orders", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it).<br>• **Padding (General → Properties → Padding):** bottom **8 px**; top 14, left 16, right 16 stay at their defaults. Without it the bottom of the 18 pt value is clipped in this 78 px card and its commas render like dots (V.I.1). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 96,478. |
| **Alt text** | "Card: number of delivered orders for the current selection." |
| **Common errors** | • A count of rows (110,197) instead of distinct orders. |

| P1-V06 · Card · Average Order Value (AOV) | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 648, y 144, width 296, height 78 (pixels on the page canvas); reading/tab order position 9 of 15 on the page |
| **Question answered** | How much is an average order worth? |
| **Fields** | • **Fields:** `_Measures[AOV] (measure)` |

| | |
|---|---|
| **Formatting (stored)** | • **Title:** "Average Order Value (AOV)", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it).<br>• **Padding (General → Properties → Padding):** bottom **8 px**; top 14, left 16, right 16 stay at their defaults. Without it the bottom of the 18 pt value is clipped in this 78 px card and its commas render like dots (V.I.1). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | R$159.83. |
| **Alt text** | "Card: average order value, revenue divided by delivered orders, for the current selection." |
| **Common errors** | • Averaging line values (R$139.93). |

| **P1-V07 · Card · Revenue MoM Growth (latest month)** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 960, y 144, width 296, height 78 (pixels on the page canvas); reading/tab order position 10 of 15 on the page |
| **Question answered** | Did revenue grow or fall in the latest month? |
| **Fields** | • **Fields:** `_Measures[Latest Month Revenue MoM %] (measure)` |
| **Formatting (stored)** | • **Title:** "Revenue MoM Growth (latest month)", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it).<br>• **Padding (General → Properties → Padding):** bottom **8 px**; top 14, left 16, right 16 stay at their defaults. Without it the bottom of the 18 pt value is clipped in this 78 px card and its commas render like dots (V.I.1). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | −4.1% (August 2018 R$985,491.64 against July R$1,027,807.28). |
| **Alt text** | "Card: revenue growth of the latest month compared with the month before it." |
| **Common errors** | • Using [Revenue MoM %] on a card: with no month in context it has no prior month and shows blank. |

| **P1-V08 · Clustered column chart · Monthly revenue (R$ millions)** | |
|---|---|
| **Visual type** | **Clustered column chart** |
| **Position and size** | x 24, y 230, width 700, height 232 (pixels on the page canvas); reading/tab order position 11 of 15 on the page |
| **Question answered** | How has monthly revenue developed? |
| **Fields** | • **X-axis:** `dim_date[month_start]`<br>• **Y-axis:** `_Measures[Total Revenue] (measure)` |

| | |
|---|---|
| **Formatting (stored)** | • **Title:** "Monthly revenue (R$ millions)", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "Delivered orders, by order month", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#1F3A5F`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Continuous (date/number axis).<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, display units Millions.<br>• **Data labels:** off.<br>• **Legend:** off. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 23 columns, September 2016 to August 2018 (no November 2016). Revenue climbs through 2017 to a peak in November 2017 (R$1,153,364.20), then holds near R$1.0–1.1 million a month in 2018. Display units are millions. |
| **Alt text** | "Column chart of monthly delivered revenue. Revenue grows through 2017 and then levels off in 2018; hover a column for its value." |
| **Common errors** | • Putting `dim_date[date]` (days) on the axis instead of `month_start`.<br>• Forgetting display units, so labels read 1,000,000. |

| **P1-V09 · Clustered column chart · Revenue MoM % by month** | |
|---|---|
| **Visual type** | **Clustered column chart** |
| **Position and size** | x 736, y 230, width 520, height 232 (pixels on the page canvas); reading/tab order position 12 of 15 on the page |
| **Question answered** | How fast is revenue growing or shrinking month by month? |
| **Fields** | • **X-axis:** `dim_date[month_start]`<br>• **Y-axis:** `_Measures[Revenue MoM %] (measure)` |
| **Formatting (stored)** | • **Title:** "Revenue MoM % by month", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "Axis capped at −50% / +100% · tiny 2016–Jan 2017 base · hover for true value", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#2A9D8F`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Continuous (date/number axis).<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start -0.5, end 1, decimal places 0.<br>• **Data labels:** off.<br>• **Legend:** off. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | Mostly small bars between about −10% and +55%. The first months are off the scale (October 2016 +32,307%, January 2017 +649,657%) and are clipped by the fixed axis (−50% to +100%); hover shows the true value. |
| **Alt text** | "Column chart of month-over-month revenue growth. Early 2016 to January 2017 values are extreme because the base was tiny; the axis is capped, so hover for the true value." |
| **Common errors** | • Removing the axis limits: two tiny-base months flatten the rest.<br>• Expecting a bar for December 2016: its prior month (November) has no sales, so the measure is blank. |

| P1-V10 · Line and clustered column chart · Category Pareto — top 15 categories | |
|---|---|
| **Visual type** | **Line and clustered column chart** |
| **Position and size** | x 24, y 470, width 500, height 292 (pixels on the page canvas); reading/tab order position 13 of 15 on the page |
| **Question answered** | Which categories matter most, and how concentrated is revenue? |
| **Fields** | • **X-axis:** `vw_sales[product_category_name_english]`<br>• **Column y-axis:** `_Measures[Top Category Revenue] (measure)`<br>• **Line y-axis:** `_Measures[Category Cumulative Share %] (measure)` |
| **Formatting (stored)** | • **Title:** "Category Pareto — top 15 categories", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "Bars: revenue (R$ M, left) · line: cumulative share of ALL category revenue (right)", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#1F3A5F`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical.<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, display units Millions.<br>• **Line series (cumulative share):** no colour is stored for it, so it takes the next theme colour; width 2.5 pt.<br>• **Data labels:** off.<br>• **Legend:** on, position Bottom, 9 pt `#6B7280`.<br>• **Line:** width 2.5 pt. |
| **Sort and filters** | • **Sort:** _Measures[Top Category Revenue] descending (visual's "…" menu → Sort axis).<br>• **Visual filters:** none. |
| **Expected result** | Fifteen navy bars from health_beauty (R$1.41 M) down to office_furniture (R$0.34 M) and a line rising from 9.2% to 76.5% (cumulative share of all category revenue). Five categories reach 39.25%. |
| **Alt text** | "Pareto chart: bars show revenue for the top 15 product categories, the line shows their cumulative share of all category revenue. A few categories account for a large share of sales." |
| **Common errors** | • Sorting by category name instead of revenue.<br>• Putting the share measure in the column well rather than the line well.<br>• Dividing by the top-15 total, which ends the line at 100%. |

| P1-V11 · Clustered column chart · Revenue by customer state (R$ millions) | |
|---|---|
| **Visual type** | **Clustered column chart** |
| **Position and size** | x 532, y 470, width 724, height 292 (pixels on the page canvas); reading/tab order position 14 of 15 on the page |
| **Question answered** | Where do customers buy from? |
| **Fields** | • **X-axis:** `vw_sales[customer_state]`<br>• **Y-axis:** `_Measures[Total Revenue] (measure)` |
| **Formatting (stored)** | • **Title:** "Revenue by customer state (R$ millions)", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "All 27 states, sorted by revenue", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#2A9D8F`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical, preferred category width 8.<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, display units Millions.<br>• **Data labels:** off.<br>• **Legend:** off. |

| | |
|---|---|
| **Sort and filters** | • **Sort:** _Measures[Total Revenue] descending (visual's "…" menu → Sort axis).<br>• **Visual filters:** the filter pane lists the fields but no condition is stored; leave them untouched. |
| **Expected result** | Twenty-seven columns sorted by revenue; SP leads with R$5.77 M (37.4%), then RJ R$2.06 M (13.3%) and MG R$1.82 M (11.8%). |
| **Alt text** | "Column chart of revenue for all customer states, sorted from highest to lowest. Sao Paulo leads by a wide margin." |
| **Common errors** | • Using `vw_orders[customer_state]`: no relationship to the sales measure. |

## V.F.2 Page 2 · Sales Deep-Dive

The sales deep-dive: which categories sell when (heat-map), how customers pay and instalment, order value over time and seller concentration. Payment visuals deliberately ignore the slicers.

| | |
|---|---|
| **Canvas** | 1280 × 960 px · Custom · Fit to page · background `#F3F5F8` |
| **Visuals** | 9 visuals and 4 text boxes |
| **Suggested build order** | Slicers → matrix → payment charts → AOV line → seller card and chart. |

| **STEP F.2.0 · Set up the page canvas** | |
|---|---|
| **Where** | Power BI Desktop, Report view. For pages 2–4 first click the **+** at the bottom to add a page, then rename it (double-click its tab). |
| **Action** | Click an empty part of the page → **Format page** (paint-roller icon) → **Canvas settings** → **Type: Custom**, **Width 1280**, **Height 960**. Then **Canvas background → Colour** `#F3F5F8`, transparency 0%. **View → Page view → Fit to page**. Name the page "Sales Deep-Dive". |
| **Purpose** | A fixed canvas makes every position in the recipes land where it should. |
| **Expected result** | A pale grey empty page with a 1280-wide canvas. |
| **Verification** | Format page → Canvas settings shows Custom, 1280 × 960. |
| **Troubleshooting** | If the page is a standard 16:9 size, the Type is still "16:9"; change it to Custom. |
| **Next step** | The blueprint, then the visuals. |

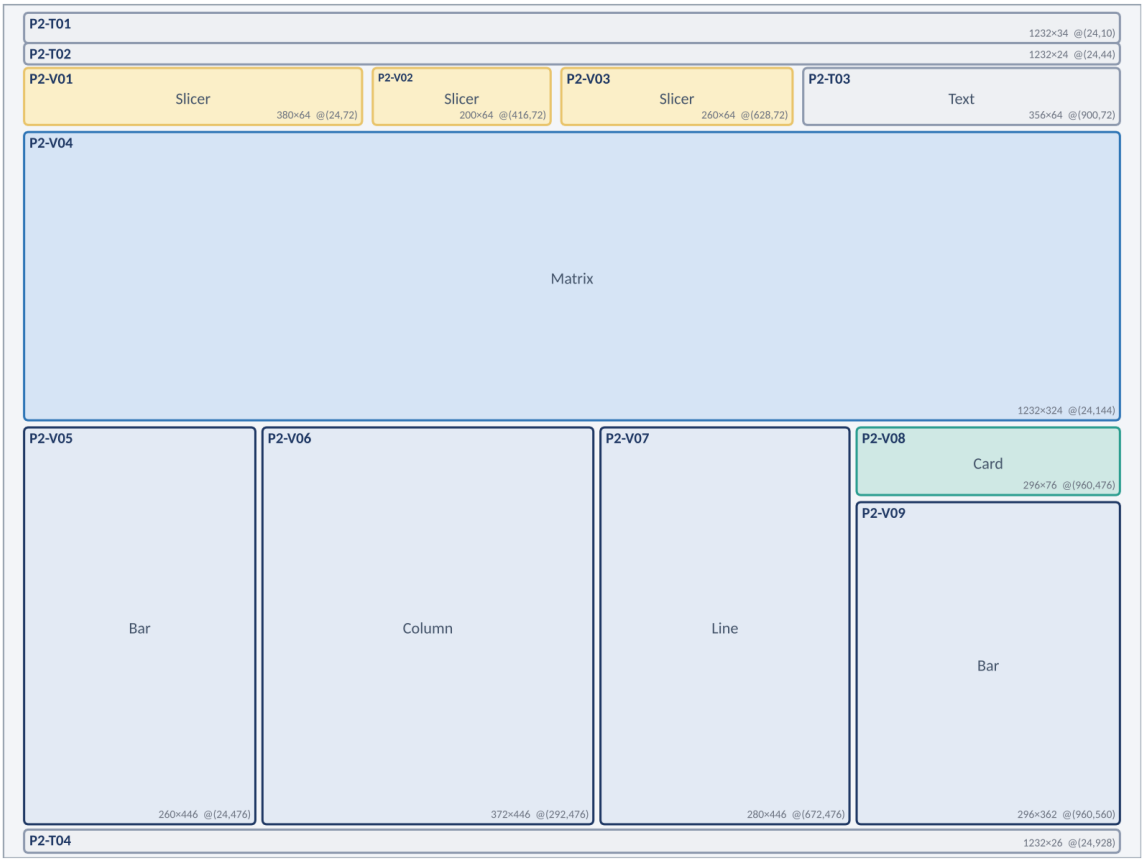



**Figure 9.** Layout blueprint of page 2 drawn from the stored positions; each box shows its ID, type, size and position (x, y).

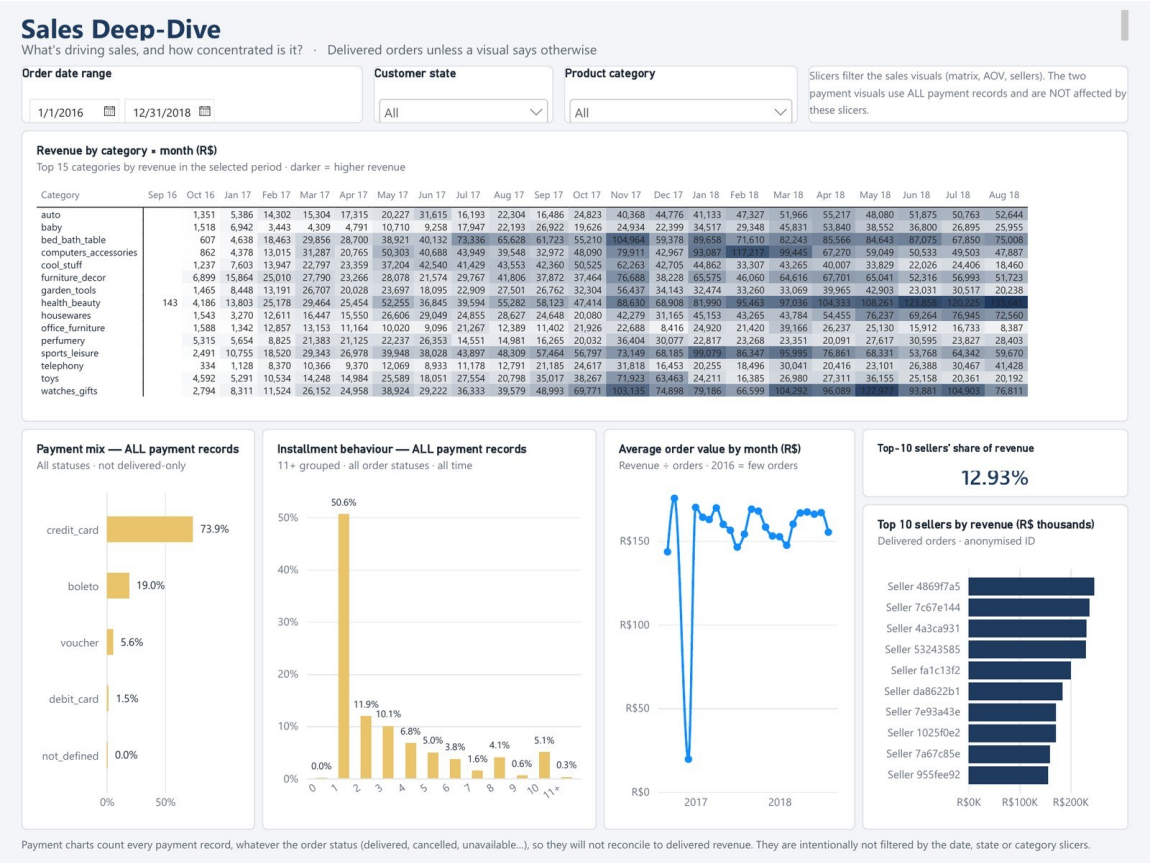



**Figure 10.** Page 2 as built. Page 2 of the saved final report, exported with File → Export → PDF (cropped to the canvas).

### Text boxes on page 2

| ID | x,y · size | Text (type it exactly) |
|---|---|---|
| P2-T01 | 24,10 · 1232×34 | Sales Deep-Dive |
| P2-T02 | 24,44 · 1232×24 | What's driving sales, and how concentrated is it? · Delivered orders unless a visual says otherwise |
| P2-T03 | 900,72 · 356×64 | Slicers filter the sales visuals (matrix, AOV, sellers). The two payment visuals use ALL payment records and are NOT affected by these slicers. |
| P2-T04 | 24,928 · 1232×26 | Payment charts count every payment record, whatever the order status (delivered, cancelled, unavailable…), so they will not reconcile to delivered revenue. They are intentionally not filtered by the date, state or category slicers. |

Text boxes: insert with **Insert → Text box**, type the text, set Segoe UI and the sizes listed in V.F.1 (headline 22 pt bold `#1F3A5F`; subtitle 11 pt `#5B6472`; page footnotes 9 pt `#5B6472`; the boxed slicer notes 9 pt `#6B7280` on white). P3-T04 is bold 10 pt `#1F3A5F`; P4-T04 is 9 pt and P4-T05 is 8 pt in `#1F2937`. Turn the text box background and border off.

### Visual recipes · page 2

| P2-V01 · Slicer · Order date range | |
|---|---|
| **Visual type** | **Slicer** |
| **Position and size** | x 24, y 72, width 380, height 64 (pixels on the page canvas); reading/tab order position 3 of 13 on the page |
| **Question answered** | Which order dates should the sales page cover? |
| **Fields** | • **Field:** `dim_date[date]` |
| **Formatting (stored)** | • **Title:** "Order date range", bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Slicer settings:** style **Between**; slicer header off (the container title is used); "Select all" on; multi-select on. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | Same date range control as page 1; it filters the matrix, AOV line and seller visuals but not the payment visuals. |
| **Alt text** | "Date range slicer. Filters the matrix, average order value and seller visuals; payment visuals are not affected." |
| **Common errors** | • Expecting the payment charts to respond; they have no relationship to the date table. |

| P2-V02 · Slicer · Customer state | |
|---|---|
| **Visual type** | **Slicer** |
| **Position and size** | x 416, y 72, width 200, height 64 (pixels on the page canvas); reading/tab order position 4 of 13 on the page |
| **Question answered** | Which state should the sales page show? |
| **Fields** | • **Field:** `vw_sales[customer_state]` |
| **Formatting (stored)** | • **Title:** "Customer state", bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Slicer settings:** style **Dropdown**; slicer header off (the container title is used); "Select all" on; multi-select on. |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | Dropdown of 27 states; filters matrix, AOV and sellers, not payments. |
| **Alt text** | “Customer state slicer. Filters the matrix, average order value and seller visuals; payment visuals are not affected.” |
| **Common errors** | • As P1-V02. |

| **P2-V03 · Slicer · Product category** | |
|---|---|
| **Visual type** | **Slicer** |
| **Position and size** | x 628, y 72, width 260, height 64 (pixels on the page canvas); reading/tab order position 5 of 13 on the page |
| **Question answered** | Which category should the sales page show? |
| **Fields** | • **Field:** `vw_sales[product_category_name_english]` |
| **Formatting (stored)** | • **Title:** “Product category”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Slicer settings:** style **Dropdown**; slicer header off (the container title is used); “Select all” on; multi-select on. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | Dropdown of the categories; filters matrix, AOV and sellers, not payments. |
| **Alt text** | “Product category slicer. Filters the matrix, average order value and seller visuals; payment visuals are not affected.” |
| **Common errors** | • As P1-V03. |

| **P2-V04 · Matrix · Revenue by category × month (R$)** | |
|---|---|
| **Visual type** | **Matrix** |
| **Position and size** | x 24, y 144, width 1232, height 324 (pixels on the page canvas); reading/tab order position 7 of 13 on the page |
| **Question answered** | Which categories sell in which months? |
| **Fields** | • **Columns:** `dim_date[month_start]`<br>• **Rows:** `vw_sales[product_category_name_english] as "Category"`<br>• **Values:** `_Measures[Top Category Revenue] (measure)` |
| **Formatting (stored)** | • **Title:** “Revenue by category × month (R$)”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** “Top 15 categories by revenue in the selected period · darker = higher revenue”, 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Fonts:** values 8 pt `#1F2937`; column headers 8 pt `#6B7280`; row headers 8 pt `#1F2937`; row padding 0; row and column subtotals off.<br>• **Cell colour (Values → Background colour → fx → Gradient):** based on `Top Category Revenue`; minimum white `#FFFFFF`, maximum navy `#1F3A5F`, automatic minimum/maximum; empty cells get no colour. |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** the filter pane lists the fields but no condition is stored; leave them untouched. |
| **Expected result** | Fifteen category rows by 23 month columns; darker cells mean more revenue (health_beauty, watches_gifts and bed_bath_table are darkest in 2017–2018); early 2016 columns are almost white. |
| **Alt text** | "Heat-map table of revenue for the top 15 categories by month. Darker cells mean higher revenue and the numbers are shown in each cell." |
| **Common errors** | • Using `Total Revenue` in Values: all 74 categories appear, not 15.<br>• Leaving subtotals on: totals disturb the colour scale.<br>• Forgetting the colour scale rule "no colour for empty cells". |

| **P2-V05 · Clustered bar chart · Payment mix — ALL payment records** | |
|---|---|
| **Visual type** | **Clustered bar chart** |
| **Position and size** | x 24, y 476, width 260, height 446 (pixels on the page canvas); reading/tab order position 8 of 13 on the page |
| **Question answered** | How do customers pay? |
| **Fields** | • **Y-axis:** `order_payments[payment_type]`<br>• **Tooltips:** `_Measures[Payment Records] (measure)`<br>• **Tooltips:** `_Measures[Avg Installments] (measure)`<br>• **Tooltips:** `_Measures[Total Payment Value] (measure)`<br>• **X-axis:** `_Measures[Payment Record Share %] (measure)` |
| **Formatting (stored)** | • **Title:** "Payment mix — ALL payment records", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "All statuses · not delivered-only", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#E9C46A`.<br>• **Y axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical.<br>• **X axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** on, 9 pt `#1F2937`, 1 decimal place(s).<br>• **Legend:** off. |
| **Sort and filters** | • **Sort:** _Measures[Payment Record Share %] descending (visual's "…" menu → Sort axis).<br>• **Visual filters:** none. |
| **Expected result** | Five bars: credit_card 73.9%, boleto 19.0%, voucher 5.6%, debit_card 1.5%, not_defined 0.0% (3 records). Hover shows records, average instalments and value. |
| **Alt text** | "Bar chart of payment methods as a share of all payment records, including every order status. Credit card is the dominant method; hover for record counts and average installments." |
| **Common errors** | • Adding the slicers' fields to the visual: payments are not related to them.<br>• Using `Payment Records` counts instead of the share measure. |

| **P2-V06 · Clustered column chart · Installment behaviour — ALL payment records** | |
|---|---|
| **Visual type** | **Clustered column chart** |
| **Position and size** | x 292, y 476, width 372, height 446 (pixels on the page canvas); reading/tab order position 9 of 13 on the page |
| **Question answered** | How many instalments do customers choose? |
| **Fields** | • **X-axis:** `order_payments[installments_bucket]`<br>• **Y-axis:** `_Measures[Payment Record Share %] (measure)` |

| | |
|---|---|
| **Formatting (stored)** | • **Title:** "Installment behaviour — ALL payment records", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "11+ grouped · all order statuses · all time", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#E9C46A`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical.<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** on, 8 pt `#1F2937`, 1 decimal place(s).<br>• **Legend:** off. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | Twelve columns in order 0, 1 … 10, 11+. One instalment is 50.6%, two 11.9%, three 10.1%, ten 5.1%. |
| **Alt text** | "Column chart of how many instalments customers choose, as a share of all payment records. Most payments are single or few instalments." |
| **Common errors** | • Alphabetical order ("1, 10, 11+, 2 …"): set Sort by column to `installments_sort`. |

| **P2-V07 · Line chart · Average order value by month (R$)** | |
|---|---|
| **Visual type** | **Line chart** |
| **Position and size** | x 672, y 476, width 280, height 446 (pixels on the page canvas); reading/tab order position 10 of 13 on the page |
| **Question answered** | Is the average order value stable? |
| **Fields** | • **X-axis:** `dim_date[month_start]`<br>• **Y-axis:** `_Measures[AOV] (measure)` |
| **Formatting (stored)** | • **Title:** "Average order value by month (R$)", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "Revenue ÷ orders · 2016 = few orders", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** leave the line at the theme default, blue `#118DFF`. (The file also stores `#2A9D8F` as a series-wide fill, but line charts do not apply it; Power BI Desktop and the screenshots show blue. See V.I.1.)<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Continuous (date/number axis).<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** off.<br>• **Legend:** off.<br>• **Line:** width 2.5 pt, markers on. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | A blue line near R$155–175 through 2017–2018 (R$155.17 in August 2018). The first 2016 months jump because each has very few orders. |
| **Alt text** | "Line chart of average order value by month for delivered orders. Early 2016 months have very few orders, so those values are volatile." |
| **Common errors** | • Putting a measure like Total Revenue in the line well.<br>• Starting the axis above 0 — the stored axis starts at 0. |

| **P2-V08 · Card · Top-10 sellers' share of revenue** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 960, y 476, width 296, height 76 (pixels on the page canvas); reading/tab order position 11 of 13 on the page |
| **Question answered** | How much do the ten biggest sellers earn? |

| | |
|---|---|
| **Fields** | • **Fields:** `_Measures[Top 10 Seller Share %] (measure)` |
| **Formatting (stored)** | • **Title:** "Top-10 sellers' share of revenue", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 12.93%. |
| **Alt text** | "Card: share of delivered revenue earned by the ten highest-revenue sellers." |
| **Common errors** | • Calling it "low concentration": 2,970 sellers share the rest and the figure is the top-10 share only. |

| **P2-V09 · Clustered bar chart · Top 10 sellers by revenue (R$ thousands)** | |
|---|---|
| **Visual type** | **Clustered bar chart** |
| **Position and size** | x 960, y 560, width 296, height 362 (pixels on the page canvas); reading/tab order position 12 of 13 on the page |
| **Question answered** | Who are the top ten sellers? |
| **Fields** | • **Y-axis:** `vw_sales[seller_label]`<br>• **X-axis:** `_Measures[Top 10 Seller Revenue] (measure)` |
| **Formatting (stored)** | • **Title:** "Top 10 sellers by revenue (R$ thousands)", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "Delivered orders · anonymised ID", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#1F3A5F`.<br>• **Y axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical, preferred category width 8, inner padding 12%.<br>• **X axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0, display units Thousands.<br>• **Data labels:** off.<br>• **Legend:** off. |
| **Sort and filters** | • **Sort:** _Measures[Top 10 Seller Revenue] descending (visual's "…" menu → Sort axis).<br>• **Visual filters:** none. |
| **Expected result** | Ten bars from "Seller 4869f7a5" (R$247.0 K) downward; the bars sum to R$1.99 M. |
| **Alt text** | "Bar chart of the ten highest-revenue sellers, shown by anonymised seller ID, with revenue in thousands of reais." |
| **Common errors** | • Putting `seller_id` on the axis: the long hashes do not fit; use the label.<br>• Setting a visual-level Top N filter instead of the rank measure. |

## V.F.3 Page 3 · Customers & Retention

Customers and retention, all-time: customer counts, the cohort heat-map, month-1 retention and the RFM segments. No slicers, because none of these tables has a date, state or category.

| | |
|---|---|
| **Canvas** | 1280 × 900 px · Custom · Fit to page · background `#F3F5F8` |
| **Visuals** | 9 visuals and 5 text boxes |
| **Suggested build order** | Cards → cohort matrix → one-time/repeat bars and month-1 card → RFM bars. |

| STEP F.3.0 · Set up the page canvas | |
|---|---|
| **Where** | Power BI Desktop, Report view. For pages 2–4 first click the **+** at the bottom to add a page, then rename it (double-click its tab). |
| **Action** | Click an empty part of the page → **Format page** (paint-roller icon) → **Canvas settings** → **Type: Custom**, **Width 1280**, **Height 900**. Then **Canvas background → Colour** `#F3F5F8`, transparency 0%. **View → Page view → Fit to page**. Name the page “Customers & Retention”. |
| **Purpose** | A fixed canvas makes every position in the recipes land where it should. |
| **Expected result** | A pale grey empty page with a 1280-wide canvas. |
| **Verification** | Format page → Canvas settings shows Custom, 1280 × 900. |
| **Troubleshooting** | If the page is a standard 16:9 size, the Type is still “16:9”; change it to Custom. |
| **Next step** | The blueprint, then the visuals. |

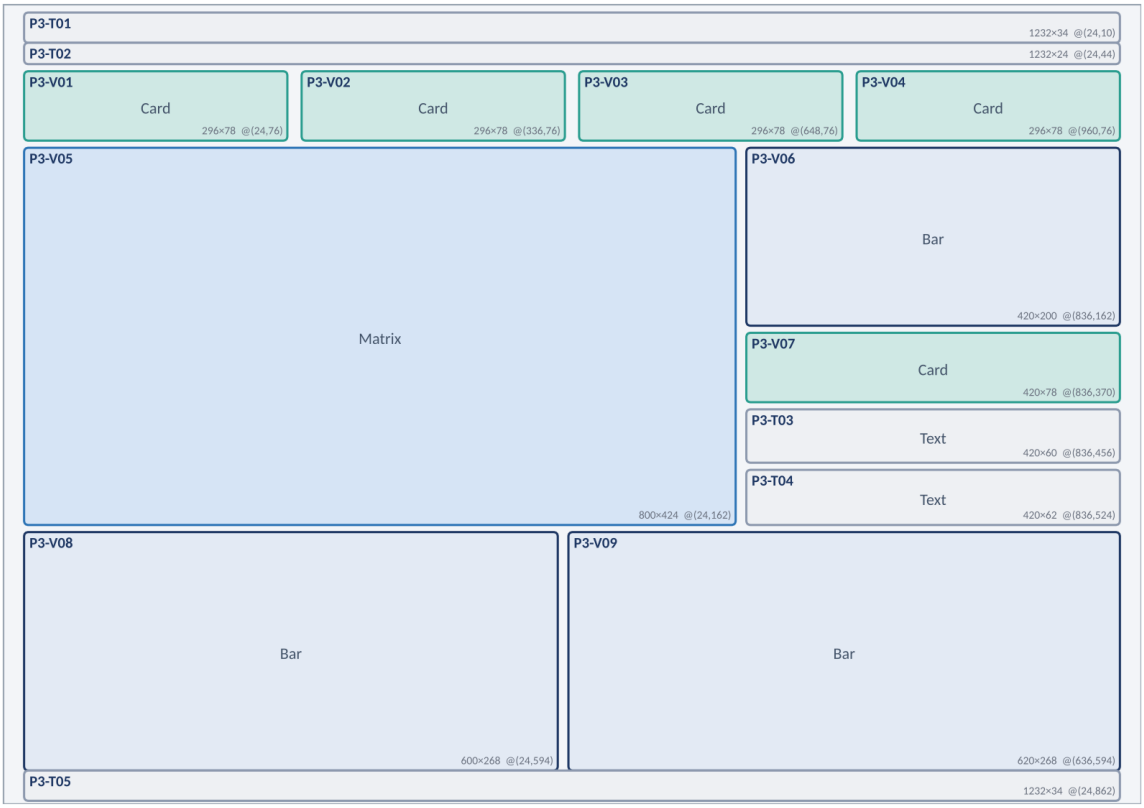



**Figure 11.** Layout blueprint of page 3 drawn from the stored positions; each box shows its ID, type, size and position (x, y).

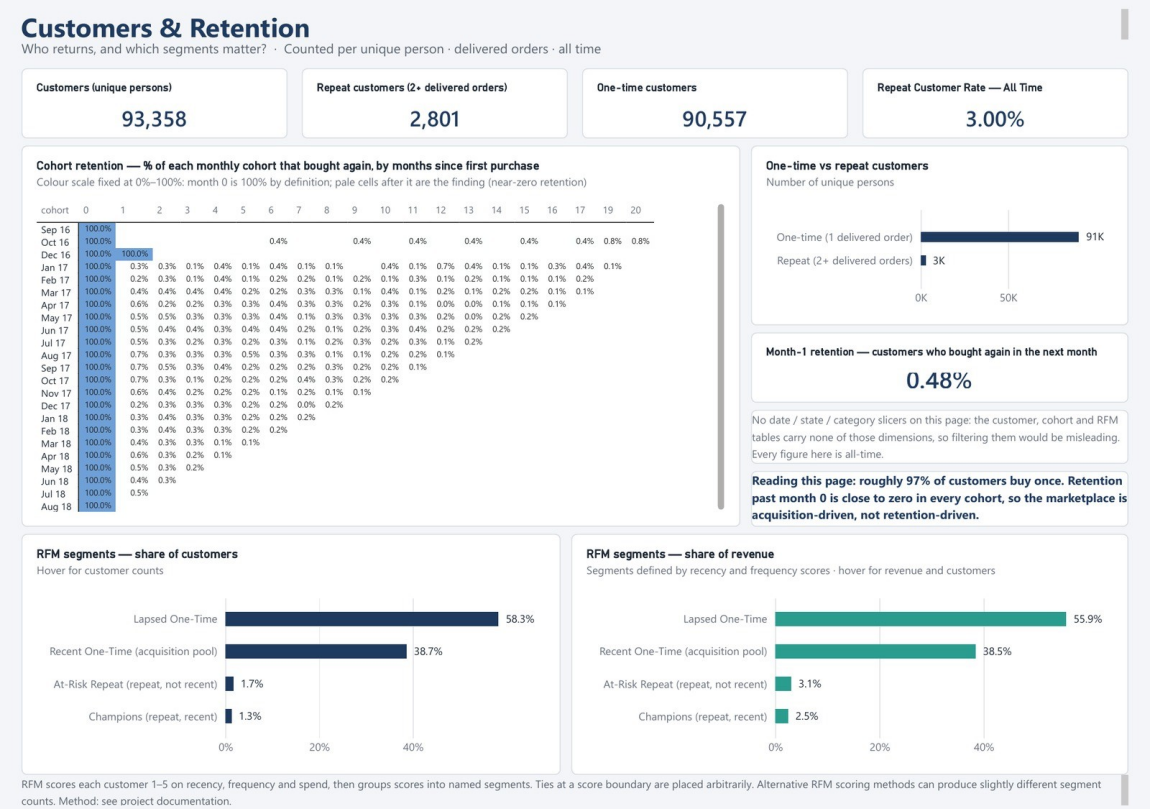



**Figure 12.** Page 3 as built. Page 3 of the saved final report, exported with File → Export → PDF (cropped to the canvas).

## Text boxes on page 3

| ID | x,y · size | Text (type it exactly) |
|---|---|---|
| P3-T01 | 24,10 · 1232×34 | Customers & Retention |
| P3-T02 | 24,44 · 1232×24 | Who returns, and which segments matter? · Counted per unique person · delivered orders · all time |
| P3-T03 | 836,456 · 420×60 | No date / state / category slicers on this page: the customer, cohort and RFM tables carry none of those dimensions, so filtering them would be misleading. Every figure here is all-time. |
| P3-T04 | 836,524 · 420×62 | Reading this page: roughly 97% of customers buy once. Retention past month 0 is close to zero in every cohort, so the marketplace is acquisition-driven, not retention-driven. |
| P3-T05 | 24,862 · 1232×34 | RFM scores each customer 1–5 on recency, frequency and spend, then groups scores into named segments. Ties at a score boundary are placed arbitrarily. Alternative RFM scoring methods can produce slightly different segment counts. Method: see project documentation. |

Text boxes: insert with **Insert → Text box**, type the text, set Segoe UI and the sizes listed in V.F.1 (headline 22 pt bold `#1F3A5F`; subtitle 11 pt `#5B6472`; page footnotes 9 pt `#5B6472`; the boxed slicer notes 9 pt `#6B7280` on white). P3-T04 is bold 10 pt `#1F3A5F`; P4-T04 is 9 pt and P4-T05 is 8 pt in `#1F2937`. Turn the text box background and border off.

## Visual recipes · page 3

| P3-V01 · Card · Customers (unique persons) | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 24, y 76, width 296, height 78 (pixels on the page canvas); reading/tab order position 3 of 14 on the page |
| **Question answered** | How many different people bought? |
| **Fields** | • **Fields:** `_Measures[Customers] (measure)` |

| | |
|---|---|
| **Formatting (stored)** | • **Title:** “Customers (unique persons)”, bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it).<br>• **Padding (General → Properties → Padding):** bottom **8 px**; top 14, left 16, right 16 stay at their defaults. Without it the bottom of the 18 pt value is clipped in this 78 px card and its commas render like dots (V.I.1). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 93,358. |
| **Alt text** | “Card: number of unique customers with at least one delivered order, counted per person.” |
| **Common errors** | • Counting `customer_id`. |

| **P3-V02 · Card · Repeat customers (2+ delivered orders)** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 336, y 76, width 296, height 78 (pixels on the page canvas); reading/tab order position 4 of 14 on the page |
| **Question answered** | How many people bought more than once? |
| **Fields** | • **Fields:** `_Measures[Repeat Customers] (measure)` |
| **Formatting (stored)** | • **Title:** “Repeat customers (2+ delivered orders)”, bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it).<br>• **Padding (General → Properties → Padding):** bottom **8 px**; top 14, left 16, right 16 stay at their defaults. Without it the bottom of the 18 pt value is clipped in this 78 px card and its commas render like dots (V.I.1). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 2,801. |
| **Alt text** | “Card: customers who placed two or more delivered orders.” |
| **Common errors** | None specific; see V.F.1. |

| **P3-V03 · Card · One-time customers** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 648, y 76, width 296, height 78 (pixels on the page canvas); reading/tab order position 5 of 14 on the page |
| **Question answered** | How many people bought only once? |
| **Fields** | • **Fields:** `_Measures[One-Time Customers] (measure)` |
| **Formatting (stored)** | • **Title:** “One-time customers”, bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it).<br>• **Padding (General → Properties → Padding):** bottom **8 px**; top 14, left 16, right 16 stay at their defaults. Without it the bottom of the 18 pt value is clipped in this 78 px card and its commas render like dots (V.I.1). |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 90,557. |
| **Alt text** | “Card: customers who placed exactly one delivered order.” |
| **Common errors** | None specific; see V.F.1. |

| **P3-V04 · Card · Repeat Customer Rate — All Time** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 960, y 76, width 296, height 78 (pixels on the page canvas); reading/tab order position 6 of 14 on the page |
| **Question answered** | What share of customers are repeat buyers? |
| **Fields** | • **Fields:** `_Measures[Repeat Cust %] (measure)` |
| **Formatting (stored)** | • **Title:** “Repeat Customer Rate — All Time”, bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it).<br>• **Padding (General → Properties → Padding):** bottom **8 px**; top 14, left 16, right 16 stay at their defaults. Without it the bottom of the 18 pt value is clipped in this 78 px card and its commas render like dots (V.I.1). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 3.00%. |
| **Alt text** | “Card: all-time share of customers who placed two or more delivered orders.” |
| **Common errors** | • Calling it “retention”. |

| **P3-V05 · Matrix · Cohort retention — % of each monthly cohort that bought again, by months since first purchase** | |
|---|---|
| **Visual type** | **Matrix** |
| **Position and size** | x 24, y 162, width 800, height 424 (pixels on the page canvas); reading/tab order position 7 of 14 on the page |
| **Question answered** | Do customers come back, cohort by cohort? |
| **Fields** | • **Columns:** `cohort_retention[month_offset]`<br>• **Rows:** `cohort_retention[cohort]`<br>• **Values:** `_Measures[Retention %] (measure)` |
| **Formatting (stored)** | • **Title:** “Cohort retention — % of each monthly cohort that bought again, by months since first purchase”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** “Colour scale fixed at 0%–100%: month 0 is 100% by definition; pale cells after it are the finding (near-zero retention)”, 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Fonts:** values 7 pt `#1F2937`; column headers 8 pt `#6B7280`; row headers 8 pt `#1F2937`; row padding 0; row and column subtotals off.<br>• **Cell colour (Values → Background colour → fx → Gradient):** based on `Retention %`; **minimum 0 → white `#FFFFFF`, maximum 1 → blue `#6FA0D8`** (fixed numbers, not automatic); empty cells get no colour. |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 23 cohort rows by 21 month columns (0–20). Column 0 is solid blue (100%); everything to its right is nearly white (about 0.2–0.7%). The triangle shape appears because newer cohorts have fewer observable months. |
| **Alt text** | “Cohort retention table. Each row is the month of a customer's first purchase, each column is months since then, and each cell is the share who bought again. Retention after month 0 is close to zero.” |
| **Common errors** | • Using automatic min/max on the colour scale: month 0 would hide everything else; the stored range is fixed 0 to 1.<br>• Putting `cohort` as text instead of a date; it must sort chronologically. |

| **P3-V06 · Clustered bar chart · One-time vs repeat customers** | |
|---|---|
| **Visual type** | **Clustered bar chart** |
| **Position and size** | x 836, y 162, width 420, height 200 (pixels on the page canvas); reading/tab order position 8 of 14 on the page |
| **Question answered** | How many customers are one-time and repeat? |
| **Fields** | • **Y-axis:** `vw_customers[customer_type]`<br>• **X-axis:** `_Measures[Customers] (measure)` |
| **Formatting (stored)** | • **Title:** “One-time vs repeat customers”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** “Number of unique persons”, 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#1F3A5F`.<br>• **Y axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical.<br>• **X axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** on, 9 pt `#1F2937`, 0 decimal place(s).<br>• **Legend:** off. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | Two bars: One-time 90,557 and Repeat 2,801, with data labels. |
| **Alt text** | “Bar chart comparing the number of one-time and repeat customers. One-time customers are the large majority.” |
| **Common errors** | • Using `customer_id` counts. |

| **P3-V07 · Card · Month-1 retention — customers who bought again in the next month** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 836, y 370, width 420, height 78 (pixels on the page canvas); reading/tab order position 9 of 14 on the page |
| **Question answered** | What share of customers buy again the very next month? |
| **Fields** | • **Fields:** `_Measures[Month-1 Retention % (weighted)] (measure)` |
| **Formatting (stored)** | • **Title:** “Month-1 retention — customers who bought again in the next month”, bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 20 pt, `#1F3A5F`, display units None; category label off (the title already names it). |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 0.48% (421 of 87,214). |
| **Alt text** | “Card: share of customers who bought again in the month after their first purchase, weighted across cohorts.” |
| **Common errors** | • Showing the unweighted mean of cohort percentages. |

| P3-V08 · Clustered bar chart · RFM segments — share of customers | |
|---|---|
| **Visual type** | **Clustered bar chart** |
| **Position and size** | x 24, y 594, width 600, height 268 (pixels on the page canvas); reading/tab order position 12 of 14 on the page |
| **Question answered** | How are customers split across RFM segments? |
| **Fields** | • **Y-axis:** `rfm_grid[segment]`<br>• **X-axis:** `_Measures[RFM Customer Share %] (measure)` |
| **Formatting (stored)** | • **Title:** “RFM segments — share of customers”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** “Hover for customer counts”, 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#1F3A5F`.<br>• **Y axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical.<br>• **X axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** on, 9 pt `#1F2937`, 1 decimal place(s).<br>• **Legend:** off. |
| **Sort and filters** | • **Sort:** _Measures[RFM Customer Share %] descending (visual’s “…” menu → Sort axis).<br>• **Visual filters:** none. |
| **Expected result** | Four bars: Lapsed One-Time 58.3%, Recent One-Time 38.7%, At-Risk Repeat 1.7%, Champions 1.3%. |
| **Alt text** | “Bar chart of the share of customers in each RFM segment; hover for customer counts.” |
| **Common errors** | • Forgetting the calculated column `segment`; the grid has only scores. |

| P3-V09 · Clustered bar chart · RFM segments — share of revenue | |
|---|---|
| **Visual type** | **Clustered bar chart** |
| **Position and size** | x 636, y 594, width 620, height 268 (pixels on the page canvas); reading/tab order position 13 of 14 on the page |
| **Question answered** | Where does revenue sit across RFM segments? |
| **Fields** | • **Y-axis:** `rfm_grid[segment]`<br>• **Tooltips:** `_Measures[RFM Revenue] (measure)`<br>• **Tooltips:** `_Measures[RFM Customers] (measure)`<br>• **X-axis:** `_Measures[RFM Revenue Share %] (measure)` |
| **Formatting (stored)** | • **Title:** “RFM segments — share of revenue”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** “Segments defined by recency and frequency scores · hover for revenue and customers”, 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#2A9D8F`.<br>• **Y axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical.<br>• **X axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** on, 9 pt `#1F2937`, 1 decimal place(s).<br>• **Legend:** off. |

| | |
|---|---|
| **Sort and filters** | • **Sort:** _Measures[RFM Revenue Share %] descending (visual's "…" menu → Sort axis).<br>• **Visual filters:** none. |
| **Expected result** | Lapsed One-Time 55.9%, Recent One-Time 38.5%, At-Risk Repeat 3.1%, Champions 2.5%. Hover shows revenue and customers. |
| **Alt text** | "Bar chart of the share of revenue contributed by each RFM segment; hover for revenue and customer counts." |
| **Common errors** | • Quoting Champions as 2.51%: that was an older notebook figure (IV.8.2); the dashboard shows 2.52%. |

## V.F.4 Page 4 · Operations & Satisfaction

Operations and satisfaction: delivery speed and lateness, the review gap between late and on-time orders, the statistical note, and the 28-day forecast.

| | |
|---|---|
| **Canvas** | 1280 × 800 px · Custom · Fit to page · background `#F3F5F8` |
| **Visuals** | 15 visuals and 5 text boxes |
| **Suggested build order** | Slicers → cards → distribution and trend → review charts and table → forecast block. |

| **STEP F.4.0 · Set up the page canvas** | |
|---|---|
| **Where** | Power BI Desktop, Report view. For pages 2–4 first click the **+** at the bottom to add a page, then rename it (double-click its tab). |
| **Action** | Click an empty part of the page → **Format page** (paint-roller icon) → **Canvas settings** → **Type: Custom**, **Width 1280**, **Height 800**. Then **Canvas background → Colour** `#F3F5F8`, transparency 0%. **View → Page view → Fit to page**. Name the page "Operations & Satisfaction". |
| **Purpose** | A fixed canvas makes every position in the recipes land where it should. |
| **Expected result** | A pale grey empty page with a 1280-wide canvas. |
| **Verification** | Format page → Canvas settings shows Custom, 1280 × 800. |
| **Troubleshooting** | If the page is a standard 16:9 size, the Type is still "16:9"; change it to Custom. |
| **Next step** | The blueprint, then the visuals. |

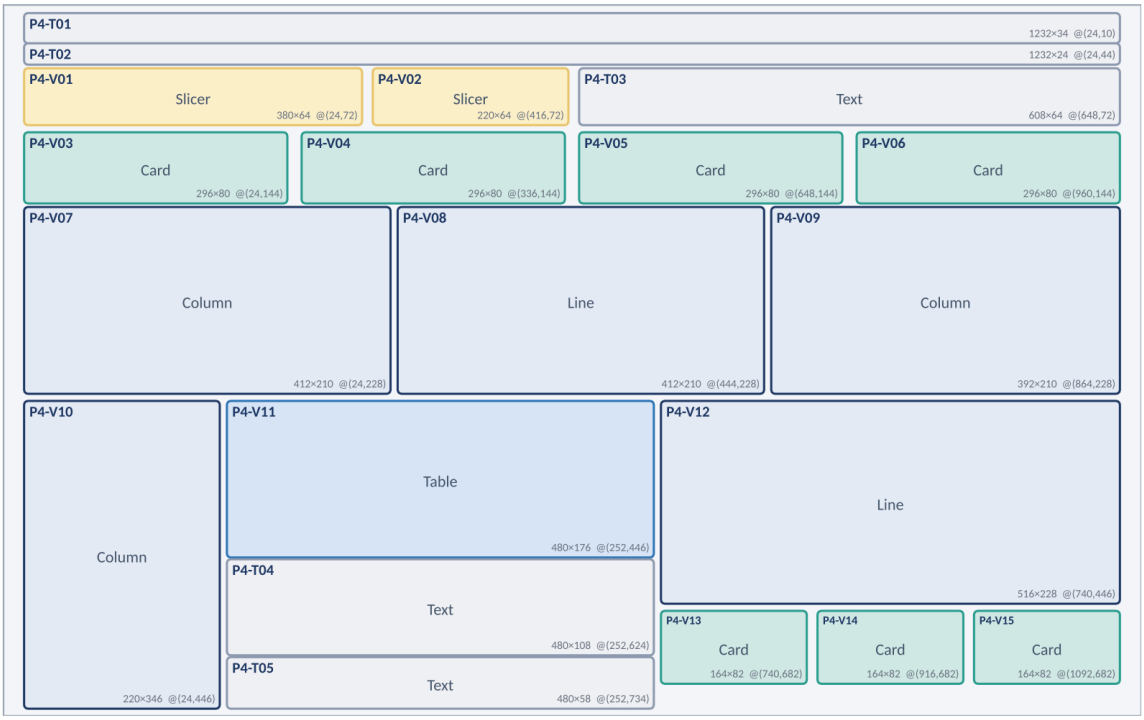



**Figure 13.** Layout blueprint of page 4 drawn from the stored positions; each box shows its ID, type, size and position (x, y).

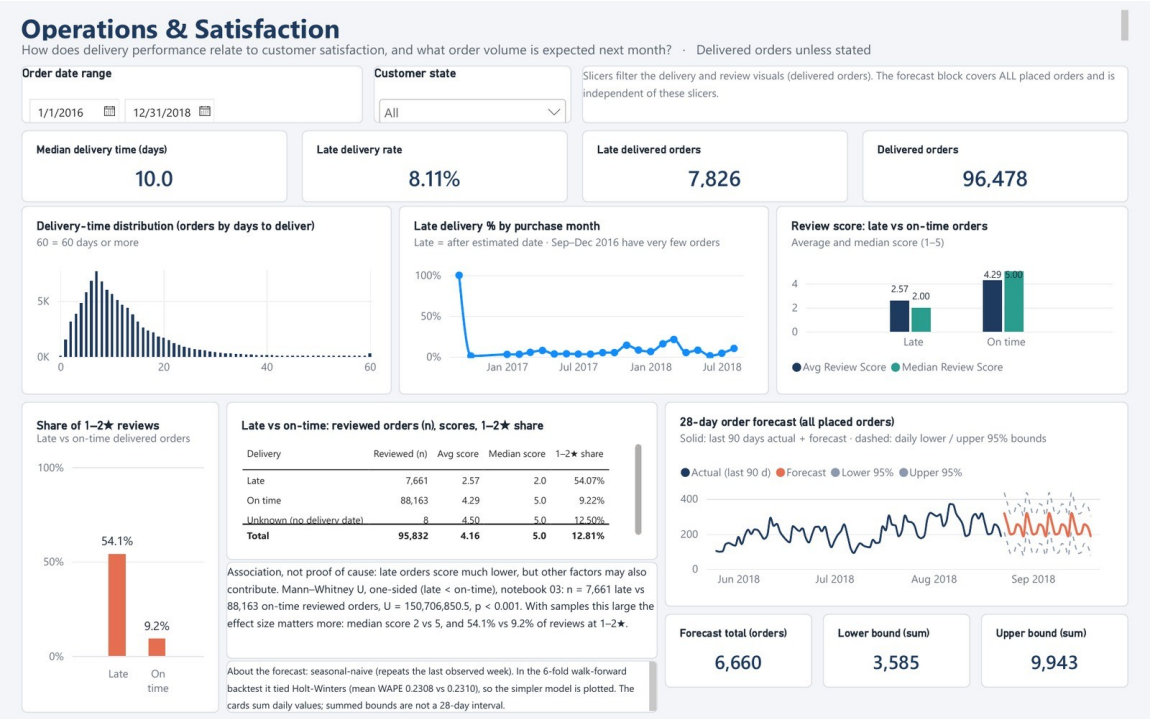



**Figure 14.** Page 4 as built. Page 4 of the saved final report, exported with File → Export → PDF (cropped to the canvas).

## Text boxes on page 4

| ID | x,y · size | Text (type it exactly) |
|---|---|---|
| P4-T01 | 24,10 · 1232×34 | Operations & Satisfaction |
| P4-T02 | 24,44 · 1232×24 | How does delivery performance relate to customer satisfaction, and what order volume is expected next month? · Delivered orders unless stated |
| P4-T03 | 648,72 · 608×64 | Slicers filter the delivery and review visuals (delivered orders). The forecast block covers ALL placed orders and is independent of these slicers. |

| ID | x,y · size | Text (type it exactly) |
|---|---|---|
| P4-T04 | 252,624 · 480×108 | Association, not proof of cause: late orders score much lower, but other factors may also contribute. Mann–Whitney U, one-sided (late < on-time), notebook 03: n = 7,661 late vs 88,163 on-time reviewed orders, U = 150,706,850.5, p < 0.001. With samples this large the effect size matters more: median score 2 vs 5, and 54.1% vs 9.2% of reviews at 1–2★. |
| P4-T05 | 252,734 · 480×58 | About the forecast: seasonal-naive (repeats the last observed week). In the 6-fold walk-forward backtest it tied Holt-Winters (mean WAPE 0.2308 vs 0.2310), so the simpler model is plotted. The cards sum daily values; summed bounds are not a 28-day interval. |

Text boxes: insert with **Insert → Text box**, type the text, set Segoe UI and the sizes listed in V.F.1 (headline 22 pt bold `#1F3A5F`; subtitle 11 pt `#5B6472`; page footnotes 9 pt `#5B6472`; the boxed slicer notes 9 pt `#6B7280` on white). P3-T04 is bold 10 pt `#1F3A5F`; P4-T04 is 9 pt and P4-T05 is 8 pt in `#1F2937`. Turn the text box background and border off.

## Visual recipes · page 4

| P4-V01 · Slicer · Order date range | |
|---|---|
| **Visual type** | **Slicer** |
| **Position and size** | x 24, y 72, width 380, height 64 (pixels on the page canvas); reading/tab order position 3 of 20 on the page |
| **Question answered** | Which order dates should the operations page cover? |
| **Fields** | • **Field:** `dim_date[date]` |
| **Formatting (stored)** | • **Title:** “Order date range”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Slicer settings:** style **Between**; slicer header off (the container title is used); “Select all” on; multi-select on. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | Same date range control; filters delivery and review visuals, not the forecast. |
| **Alt text** | “Date range slicer. Filters delivery and review visuals; the forecast is not affected.” |
| **Common errors** | None specific; see V.F.1. |

| P4-V02 · Slicer · Customer state | |
|---|---|
| **Visual type** | **Slicer** |
| **Position and size** | x 416, y 72, width 220, height 64 (pixels on the page canvas); reading/tab order position 4 of 20 on the page |
| **Question answered** | Which state should the operations page show? |
| **Fields** | • **Field:** `vw_orders[customer_state]` |
| **Formatting (stored)** | • **Title:** “Customer state”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Slicer settings:** style **Dropdown**; slicer header off (the container title is used); “Select all” on; multi-select on. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | Dropdown of 27 states from `vw_orders`; filters delivery and review visuals, not the forecast. |
| **Alt text** | “Customer state slicer. Filters delivery and review visuals; the forecast is not affected.” |
| **Common errors** | • Using `vw_sales[customer_state]`: it would not filter these visuals. |

| P4-V03 · Card · Median delivery time (days) | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 24, y 144, width 296, height 80 (pixels on the page canvas); reading/tab order position 6 of 20 on the page |
| **Question answered** | How long does delivery typically take? |
| **Fields** | • **Fields:** `_Measures[Median Delivery Days] (measure)` |
| **Formatting (stored)** | • **Title:** "Median delivery time (days)", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 10.0 days. |
| **Alt text** | "Card: median days from purchase to delivery for the current selection." |
| **Common errors** | • Reporting the mean (12.09 days). |

| P4-V04 · Card · Late delivery rate | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 336, y 144, width 296, height 80 (pixels on the page canvas); reading/tab order position 7 of 20 on the page |
| **Question answered** | What share of delivered orders is late? |
| **Fields** | • **Fields:** `_Measures[Late Delivery %] (measure)` |
| **Formatting (stored)** | • **Title:** "Late delivery rate", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 8.11%. |
| **Alt text** | "Card: share of delivered orders that arrived after the estimated delivery date." |
| **Common errors** | None specific; see V.F.1. |

| P4-V05 · Card · Late delivered orders | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 648, y 144, width 296, height 80 (pixels on the page canvas); reading/tab order position 8 of 20 on the page |
| **Question answered** | How many delivered orders were late? |
| **Fields** | • **Fields:** `_Measures[Late Orders] (measure)` |
| **Formatting (stored)** | • **Title:** "Late delivered orders", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it). |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 7,826. |
| **Alt text** | “Card: number of delivered orders that arrived after the estimated delivery date.” |
| **Common errors** | None specific; see V.F.1. |

| **P4-V06 · Card · Delivered orders** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 960, y 144, width 296, height 80 (pixels on the page canvas); reading/tab order position 9 of 20 on the page |
| **Question answered** | How many delivered orders are in the selection? |
| **Fields** | • **Fields:** `_Measures[Delivered Orders] (measure)` |
| **Formatting (stored)** | • **Title:** “Delivered orders”, bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 18 pt, `#1F3A5F`, display units None; category label off (the title already names it). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 96,478. |
| **Alt text** | “Card: number of delivered orders in the current selection.” |
| **Common errors** | None specific; see V.F.1. |

| **P4-V07 · Clustered column chart · Delivery-time distribution (orders by days to deliver)** | |
|---|---|
| **Visual type** | **Clustered column chart** |
| **Position and size** | x 24, y 228, width 412, height 210 (pixels on the page canvas); reading/tab order position 10 of 20 on the page |
| **Question answered** | How is delivery time distributed? |
| **Fields** | • **X-axis:** `vw_orders[delivery_days_capped]`<br>• **Y-axis:** `_Measures[Orders With Delivery Time] (measure)` |
| **Formatting (stored)** | • **Title:** “Delivery-time distribution (orders by days to deliver)”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** “60 = 60 days or more”, 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#1F3A5F`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Continuous (date/number axis).<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** off.<br>• **Legend:** off. |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | A right-skewed histogram of 61 bars (0–60); the tallest is 7 days (7,651 orders); the last bar (60 = 60 days or more) holds 306 orders. |
| **Alt text** | "Column chart of delivered orders by number of days taken to deliver; the last bucket groups 60 days or more." |
| **Common errors** | • Using `delivery_days` directly: the axis stretches past 200 days.<br>• Using [Delivered Orders] as the value: includes 8 orders with no delivery time. |

| P4-V08 · Line chart · Late delivery % by purchase month | |
|---|---|
| **Visual type** | **Line chart** |
| **Position and size** | x 444, y 228, width 412, height 210 (pixels on the page canvas); reading/tab order position 11 of 20 on the page |
| **Question answered** | Does lateness change over time? |
| **Fields** | • **X-axis:** `dim_date[month_start]`<br>• **Y-axis:** `_Measures[Late Delivery %] (measure)` |
| **Formatting (stored)** | • **Title:** "Late delivery % by purchase month", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "Late = after estimated date · Sep–Dec 2016 have very few orders", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** leave the line at the theme default, blue `#118DFF`. (The file also stores `#E76F51` as a series-wide fill, but line charts do not apply it; Power BI Desktop and the screenshots show blue. See V.I.1.)<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Continuous (date/number axis).<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** off.<br>• **Legend:** off.<br>• **Line:** width 2.5 pt, markers on. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | A blue line, mostly 1–10% monthly, with peaks in November 2017 (14.3%), February 2018 (16.0%) and March 2018 (21.4%). Sep–Dec 2016 points are unreliable (one to a few hundred orders). |
| **Alt text** | "Line chart of the late-delivery rate by purchase month. Months in late 2016 have very few orders, so their percentages are unreliable." |
| **Common errors** | • Reading 100% for September 2016 as real: one order. |

| P4-V09 · Clustered column chart · Review score: late vs on-time orders | |
|---|---|
| **Visual type** | **Clustered column chart** |
| **Position and size** | x 864, y 228, width 392, height 210 (pixels on the page canvas); reading/tab order position 12 of 20 on the page |
| **Question answered** | Do late orders score lower? |
| **Fields** | • **X-axis:** `vw_orders[delivery_status]`<br>• **Y-axis:** `_Measures[Avg Review Score] (measure)`<br>• **Y-axis:** `_Measures[Median Review Score] (measure)` |

| | |
|---|---|
| **Formatting (stored)** | • **Title:** “Review score: late vs on-time orders”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** “Average and median score (1–5)”, 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Series colour — Avg Review Score:** `#1F3A5F`.<br>• **Series colour — Median Review Score:** `#2A9D8F`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical.<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0, end 5.<br>• **Data labels:** on, 8 pt `#1F2937`, 2 decimal place(s).<br>• **Legend:** on, position Bottom, 9 pt `#6B7280`. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filter:** `vw_orders[delivery_status]` **is Late or On time** (Filters pane → Filters on this visual → Basic filtering → tick Late and On time). “Unknown” is left out because 8 orders cannot be compared on lateness. |
| **Expected result** | Two groups. Late: average 2.57, median 2. On time: average 4.29, median 5. Axis 0–5, labels with two decimals. |
| **Alt text** | “Column chart comparing average and median review score for late and on-time orders. Late orders score much lower.” |
| **Common errors** | • Leaving “Unknown” in the chart.<br>• Axis left on automatic: bars exaggerate differences. |

| **P4-V10 · Clustered column chart · Share of 1–2★ reviews** | |
|---|---|
| **Visual type** | **Clustered column chart** |
| **Position and size** | x 24, y 446, width 220, height 346 (pixels on the page canvas); reading/tab order position 13 of 20 on the page |
| **Question answered** | How much more often do late orders get 1–2 stars? |
| **Fields** | • **X-axis:** `vw_orders[delivery_status]`<br>• **Y-axis:** `_Measures[Pct 1-2 Star] (measure)` |
| **Formatting (stored)** | • **Title:** “Share of 1–2★ reviews”, bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** “Late vs on-time delivered orders”, 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Colour:** default series colour `#E76F51`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Categorical.<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0, end 1.<br>• **Data labels:** on, 10 pt `#1F2937`, 1 decimal place(s).<br>• **Legend:** off. |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filter:** `vw_orders[delivery_status]` **is Late or On time** (Filters pane → Filters on this visual → Basic filtering → tick Late and On time). “Unknown” is left out because 8 orders cannot be compared on lateness. |
| **Expected result** | Late 54.1% against On time 9.2%. Axis 0–100%. |
| **Alt text** | “Column chart of the share of 1 and 2 star reviews for late and on-time orders. Late orders have a far higher share.” |
| **Common errors** | • Using `<= 2` in the measure: it counts unreviewed orders. |

| P4-V11 · Table · Late vs on-time: reviewed orders (n), scores, 1–2★ share | |
|---|---|
| **Visual type** | **Table** |
| **Position and size** | x 252, y 446, width 480, height 176 (pixels on the page canvas); reading/tab order position 14 of 20 on the page |
| **Question answered** | What are the numbers behind the two charts? |
| **Fields** | • **Columns:** `vw_orders[delivery_status] as "Delivery"`<br>• **Columns:** `_Measures[Reviewed Orders] (measure) as "Reviewed (n)"`<br>• **Columns:** `_Measures[Avg Review Score] (measure) as "Avg score"`<br>• **Columns:** `_Measures[Median Review Score] (measure) as "Median score"`<br>• **Columns:** `_Measures[Pct 1-2 Star] (measure) as "1–2★ share"` |
| **Formatting (stored)** | • **Title:** "Late vs on-time: reviewed orders (n), scores, 1–2★ share", bold 11 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Fonts:** values 8 pt; column headers 8 pt. Column titles are renamed in the Columns well (see Fields). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** the filter pane lists the fields but no condition is stored; leave them untouched. |
| **Expected result** | Three rows. Late: 7,661 reviewed, 2.57, 2, 54.07%. On time: 88,163, 4.29, 5, 9.22%. Unknown: 8, 4.50, 5, 12.50%. |
| **Alt text** | "Table of late, on-time and unknown-delivery orders showing reviewed order counts, average and median score and 1 to 2 star share." |
| **Common errors** | • Applying the Late/On time filter to the table: the table is intentionally unfiltered so that Unknown stays visible. |

| P4-V12 · Line chart · 28-day order forecast (all placed orders) | |
|---|---|
| **Visual type** | **Line chart** |
| **Position and size** | x 740, y 446, width 516, height 228 (pixels on the page canvas); reading/tab order position 16 of 20 on the page |
| **Question answered** | What order volume is expected over the next 28 days? |
| **Fields** | • **X-axis:** `forecast_chart[date]`<br>• **Y-axis:** `_Measures[Actual (last 90 d)] (measure)`<br>• **Y-axis:** `_Measures[Forecast] (measure)`<br>• **Y-axis:** `_Measures[Lower 95%] (measure)`<br>• **Y-axis:** `_Measures[Upper 95%] (measure)` |
| **Formatting (stored)** | • **Title:** "28-day order forecast (all placed orders)", bold 11 pt, `#1F2937`, left-aligned.<br>• **Subtitle:** "Solid: last 90 days actual + forecast · dashed: daily lower / upper 95% bounds", 9 pt, `#6B7280`.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons shown.<br>• **Series colour — Actual (last 90 d):** `#1F3A5F`.<br>• **Series colour — Forecast:** `#E76F51`.<br>• **Series colour — Lower 95%:** `#8D99AE`.<br>• **Series colour — Upper 95%:** `#8D99AE`.<br>• **X axis (categories):** labels 9 pt `#6B7280`, axis title off, type Continuous (date/number axis).<br>• **Y axis (values):** labels 9 pt `#6B7280`, axis title off, gridlines `#E5E7EB`, start 0.<br>• **Data labels:** off.<br>• **Legend:** on, position Top, 9 pt `#6B7280`.<br>• **Line — Actual (last 90 d):** width 2 pt, solid.<br>• **Line — Forecast:** width 2.5 pt, solid.<br>• **Line — Lower 95%:** width 1.25 pt, dashed.<br>• **Line — Upper 95%:** width 1.25 pt, dashed. |

| | |
|---|---|
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | A solid navy line (90 actual days to 22 August 2018), a solid coral forecast line (23 August to 19 September) and two dashed grey bounds around the forecast. The forecast repeats the weekly pattern. |
| **Alt text** | “Line chart of daily orders: solid lines show the last 90 days of actuals and the 28-day forecast; dashed lines show the lower and upper 95 percent bounds.” |
| **Common errors** | • Using `forecast_28d[date]` on the axis: actuals are missing.<br>• Axis not starting at 0.<br>• Reading the dashed lines as a 28-day interval for the total. |

| **P4-V13 · Card · Forecast total (orders)** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 740, y 682, width 164, height 82 (pixels on the page canvas); reading/tab order position 17 of 20 on the page |
| **Question answered** | What total is forecast? |
| **Fields** | • **Fields:** `_Measures[Forecast Total (28 days)] (measure)` |
| **Formatting (stored)** | • **Title:** “Forecast total (orders)”, bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 16 pt, `#1F3A5F`, display units None; category label off (the title already names it). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 6,660 orders. |
| **Alt text** | “Card: forecast total orders over the next 28 days.” |
| **Common errors** | • Treating it as a promise: it is a point forecast. |

| **P4-V14 · Card · Lower bound (sum)** | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 916, y 682, width 164, height 82 (pixels on the page canvas); reading/tab order position 18 of 20 on the page |
| **Question answered** | What is the sum of the daily lower bounds? |
| **Fields** | • **Fields:** `_Measures[Forecast Lower Sum] (measure)` |
| **Formatting (stored)** | • **Title:** “Lower bound (sum)”, bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 16 pt, `#1F3A5F`, display units None; category label off (the title already names it). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 3,585. Not a 28-day interval. |
| **Alt text** | “Card: sum of the daily lower bounds over 28 days. Not a 28-day interval.” |
| **Common errors** | • Presenting it as the lower end for the total. |

| P4-V15 · Card · Upper bound (sum) | |
|---|---|
| **Visual type** | **Card** |
| **Position and size** | x 1092, y 682, width 164, height 82 (pixels on the page canvas); reading/tab order position 19 of 20 on the page |
| **Question answered** | What is the sum of the daily upper bounds? |
| **Fields** | • **Fields:** `_Measures[Forecast Upper Sum] (measure)` |
| **Formatting (stored)** | • **Title:** "Upper bound (sum)", bold 10 pt, `#1F2937`, left-aligned.<br>• **Container:** white background, border `#D9DEE5` radius 6, shadow off, visual-header icons hidden.<br>• **Callout value:** Segoe UI Semibold, 16 pt, `#1F3A5F`, display units None; category label off (the title already names it). |
| **Sort and filters** | • **Sort:** none stored; the axis takes its natural order.<br>• **Visual filters:** none. |
| **Expected result** | 9,943. Not a 28-day interval. |
| **Alt text** | "Card: sum of the daily upper bounds over 28 days. Not a 28-day interval." |
| **Common errors** | • As the lower bound. |

## V.G Slicers, filters and interactions

There are eight slicers in total, three each on pages 1 and 2 and two on page 4. They are **not synchronised** across pages (the file stores no sync group), so choosing a state on page 1 does not carry to page 4. The model decides what each slicer can reach.

| Page | Slicer fields | They filter | They do not filter |
|---|---|---|---|
| 1 · Executive Overview | `dim_date[date]` (Between) · `vw_sales[customer_state]` · `vw_sales[product_category_name_english]` (dropdowns) | Everything on the page: P1-V04 to P1-V11 | – |
| 2 · Sales Deep-Dive | Same three fields | P2-V04 matrix, P2-V07 AOV line, P2-V08 and P2-V09 seller visuals | P2-V05 and P2-V06 payment charts (no relationship; stated in P2-T03 and P2-T04) |
| 3 · Customers & Retention | None | – | All visuals; every figure is all-time |
| 4 · Operations & Satisfaction | `dim_date[date]` (Between) · `vw_orders[customer_state]` (dropdown) | P4-V03 to P4-V11 (delivery and review visuals) | P4-V12 to P4-V15 (forecast block) |

Why the state slicer on page 4 has no category slicer next to it: `vw_orders` has no product category, because an order may contain items of several categories.

### Interactions between visuals

The file stores **no custom visual interactions**, so Power BI's defaults apply: clicking a mark in one chart cross-filters or cross-highlights the others on the same page. Because most measures are ratios, highlighting can look odd (a share bar partly filled). To inspect or change it select a visual, then **Format → Edit interactions** and choose the filter, highlight or none icon above each other visual. The slicer behaviours above come from the model, not from interactions.

VERIFICATION STATUS

**File-verified.** Slicer fields and styles, absence of sync groups and absence of stored interaction overrides come from the report definition. Which visuals a slicer reaches follows from the three relationships and the field each visual uses.

### Slicer tests

| Test | Do | Expected |
|---|---|---|
| Date reaches all order tables | Page 4: set the date slicer to 2018 only | Cards and histogram change; the forecast line does not |
| Payments are independent | Page 2: choose state RJ | Matrix and sellers change; payment mix stays 73.92% / 19.04% … |
| Month axis keeps all months | Page 1: choose a category | The month chart still spans its months; months without sales have no bar |
| Clear/reset | Press **Clear selections** in each slicer (eraser icon) | Reference values return |

## V.H The forecast route

The forecast visuals are the most layered part of the report, so follow the chain end to end.

| Step | What happens | Where |
|---|---|---|
| 1 | Notebook 04 trains the seasonal-naive and Holt-Winters models on the stable window (2017-01-05 to 2018-08-22), back-tests both on six 28-day folds and chooses the simpler model. | IV.8.4 |
| 2 | It writes 28 rows (date, forecast, lower, upper) to `analytics.forecast_28d` and to `data/processed/forecast_28d.csv`. | PostgreSQL |
| 3 | Power BI imports `forecast_28d` as it is (V.B.4). | Power Query |
| 4 | The calculated table `forecast_chart` stacks the last 90 actual days from `vw_daily_orders` and the 28 forecast rows on one date column (V.E.2). | DAX table |
| 5 | Four measures sum its columns: `Actual (last 90 d)`, `Forecast`, `Lower 95%`, `Upper 95%` (V.E.3). | _Measures |
| 6 | The line chart P4-V12 plots them; three cards (P4-V13 to V15) read `forecast_28d` directly. | Page 4 |

> CAUTION
>
> **The summed bounds are not a 28-day interval.** The cards “Lower bound (sum)” 3,585 and “Upper bound (sum)” 9,943 add 28 *daily* bounds. The daily bounds are the 2.5th and 97.5th percentiles of the 168 pooled daily backtest errors (about −109.8 and +117.2 orders). Adding daily bounds would be right only if the daily errors were perfectly correlated, which has not been shown, so the sum is not a prediction interval for the 28-day total. In the six backtest folds the 28-day totals missed the actuals by amounts between −2,363 and +731 orders (forecast minus actual). Six folds are too few to claim a coverage level, so the report states the limitation on the cards’ alt text and in the page note instead.

Why plot seasonal naive when Holt-Winters is more sophisticated? In the backtest both scored the same mean WAPE (0.2308 versus 0.2310) and the simpler model is easier to explain and to defend. Black Friday 2017 (1,176 orders against a 197-per-day baseline) sits inside every training window, which is why the forecast should be read as a typical-week projection, not as an event forecast.

| STEP H.1 · Refresh the forecast in the report | |
|---|---|
| **Where** | PowerShell, then Power BI Desktop. |
| **Action** | Re-run notebook 04 (III.8 individual procedures), then in Power BI **Home → Refresh**. |
| **Purpose** | Moves the forecast window and re-computes the calculated table. |

| | |
|---|---|
| **Expected result** | The forecast line begins the day after the last stable day; the 90-day actual window moves with it. |
| **Verification** | `SELECT min(date), max(date), count(*) FROM analytics.forecast_28d;` shows 28 rows and the dates on the chart match. |
| **Troubleshooting** | If the chart still shows the old window, the refresh failed silently for `forecast_28d`; check Transform data → the query's Applied steps. |
| **Next step** | V.J. |

# V.I Formatting, accessibility and what is confirmed

## V.I.1 Confirmed in the file, and where the screenshots differ

| Property | Status | Notes |
|---|---|---|
| Page sizes, background `#F3F5F8`, Fit to page | File-verified | Stored in each page definition. |
| Visual positions, sizes and tab order | File-verified | Used for the blueprints and recipes. |
| Titles, subtitles, container borders, header-icon visibility | File-verified | |
| Fills, axis limits, display units, data labels, legends, line widths and dashes | File-verified | |
| Matrix colour scales | File-verified | Category matrix white → `#1F3A5F` automatic; cohort matrix 0 → white, 1 → `#6FA0D8`. |
| Alt text on all 44 non-text visuals | File-verified | Longest is 201 characters (limit 250). |
| Line colours of P2-V07 and P4-V08 | **Rendered blue `#118DFF`** | The file stores teal `#2A9D8F` and coral `#E76F51` as a series-wide fill, a setting line charts do not apply. Power BI Desktop (Format pane → Lines → Color) and the screenshots show the theme's first colour, blue `#118DFF`. A faithful rebuild leaves the line colour at its default. |
| Cumulative-share line colour on P1-V10 | Not stored | It uses the theme's next colour. |
| Thousands separator on the page-1 and page-3 cards | **Fixed and viewed** | Cause: the 78 px cards clipped the bottom of the 18 pt value, so commas rendered like dots at some zooms; the format strings (`#,0`, `\R\$#,0.00`) and the en-US locale were always correct. Fix: bottom padding 8 px on the eight 78 px KPI cards (heights unchanged). Checked in Power BI Desktop after saving, closing and reopening, at 70%, 78%/80% and 110% zoom and after changing pages: commas every time. A height change to 80 px alone was not reliable. |
| Screenshots | Exported from the final file | The four images are File → Export → PDF renders of the saved final report, cropped to the canvas; they replace earlier window captures. |
| Sort-by-column assignment, `Dummy` column visibility | Not captured | Check visually (V.D.3). |
| Menu paths in the Format pane | Reviewed | Names change between releases; use the Format-pane search box. |

## V.I.2 Accessibility review

| Check | Result |
|---|---|
| Alt text | Every one of the 44 visuals has a descriptive alt text; the two forecast-bound cards say in their alt text that the sums are not a 28-day interval. |
| Reading and tab order | Stored for each page: title strip, slicers, cards, charts, notes (View → Selection pane → Tab order). |

| Check | Result |
|---|---|
| Text contrast (WCAG AA 4.5:1) | Dark text `#1F2937` on white 14.7:1; navy `#1F3A5F` on white 11.5:1; grey `#6B7280` on white 4.83:1 (passes). The subtitles and page footnotes that sit directly on the page background `#F3F5F8` use **`#5B6472` (5.48:1, passes)**; with the earlier `#6B7280` they were 4.43:1. The rendered colour was checked in the exported report (RGB 91, 100, 114). |
| Graphics contrast (3:1) | Navy passes; coral `#E76F51` 3.09:1 and teal `#2A9D8F` 3.32:1 pass narrowly; grey `#8D99AE` 2.88:1 and gold `#E9C46A` 1.67:1 do not. The gold payment bars therefore depend on their data labels and axis, which are shown. |
| Not colour alone | The heat-maps print the numbers in the cells; forecast bounds are dashed, not only grey; bars carry data labels where precision matters. |
| Small text | The cohort matrix prints values at 7 pt so 21 columns fit; it is the smallest text in the report. |
| Language | Report locale en-US. |

# V.J Save, refresh, verify and reopen

## V.J.1 Save and refresh

| STEP J.1 · Save, refresh and reopen | |
|---|---|
| **Where** | Power BI Desktop. |
| **Action** | 1. **File → Save** (Ctrl+S) your working copy.<br>2. With Docker and the database running, **Home → Refresh**. Enter the database credentials if asked (typed by you).<br>3. Wait for every table to finish; none should show an error triangle.<br>4. Save again, close Power BI, reopen the file and look at all four pages. |
| **Purpose** | Proves the file keeps its data and can reload from the database. |
| **Expected result** | Same numbers as before the refresh; the data source has not changed, so the reference values stay identical. |
| **Verification** | Run the acceptance checks in V.J.4. |
| **Troubleshooting** | • “Couldn’t authenticate”: III.7.<br>• Error 53100: the `SET` prefix is missing from a query.<br>• “relation does not exist”: run `make views`.<br>• Save as fails because the target is locked: close any other program holding the file (an earlier build hit this, VII.7). |
| **Next step** | V.J.2. |

NOTE

**Two text boxes do not refresh.** P3-T04 (“roughly 97% of customers buy once”) and P4-T04 (the Mann–Whitney statement) are typed text, written from the data at build time. They do not change when the data refresh. If the data ever change, re-check both by hand.

CAUTION

Do not overwrite `dashboard/ecommerce_retention_dashboard_final.pbix` unless you intend to publish a new authoritative version. Verify the working copy first; then replace the file with the project closed in Git, and commit it deliberately.

## V.J.2 Reference values (what the finished report must show)

| Figure | Value | Where | Check in SQL or notebook |
|---|---|---|---|
| Total Revenue | R$15,419,773.75 | P1 card | `SELECT round(sum(line_revenue),2) FROM analytics.vw_sales;` |
| Total Orders / Delivered Orders | 96,478 | P1 card, P4 card | `SELECT count(*) FROM analytics.vw_orders;` |
| AOV | R$159.83 | P1 card | 15,419,773.75 ÷ 96,478 |
| Latest-month revenue growth | −4.1% (Aug 2018 R$985,491.64 vs Jul R$1,027,807.28) | P1 card | Query 01 monthly revenue |
| Revenue, state = SP | R$5,769,703.15 (40,501 orders, AOV R$142.46) | P1 with the state slicer | `… WHERE customer_state='SP'` |
| Top category | health_beauty R$1,412,089.53 (9.16%) | P1 Pareto, P2 matrix | Query 03 |
| Top-5 category cumulative share | 39.25%; top 15 reach 76.47% | P1 Pareto line | Query 03 |
| Top state | SP 37.42%, RJ 13.33%, MG 11.80% | P1 state chart | Query 04 |
| Payments | 103,886 records, R$16,008,872.12; credit card 73.92%, boleto 19.04%, voucher 5.56%, debit 1.47% | P2 payment mix | `SELECT count(*), sum(payment_value) FROM analytics.order_payments;` |
| One instalment | 50.6% of payment records | P2 instalments | Query 05 |
| Top-10 seller share | 12.93% (R$1,993,154.99) | P2 card | Query 06 |
| Customers · repeat · one-time · repeat rate | 93,358 · 2,801 · 90,557 · 3.00% | P3 cards | `SELECT count(*), count(*) FILTER (WHERE is_repeat) FROM analytics.vw_customers;` |
| Month-1 retention | 0.48% (421 of 87,214) | P3 card | Query 12 |
| RFM customers | Lapsed One-Time 58.29% · Recent One-Time 38.71% · At-Risk Repeat 1.71% · Champions 1.29% | P3 bars | Query 15 |
| RFM revenue | Lapsed 55.88% · Recent 38.51% · At-Risk 3.09% · Champions 2.52% | P3 bars | Query 15 |
| Median delivery time | 10.0 days | P4 card | `SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY delivery_days) FROM analytics.vw_orders;` |
| Late delivery | 7,826 orders, 8.11% | P4 cards | `SELECT count(*) FILTER (WHERE is_late) FROM analytics.vw_orders;` |
| Unknown lateness | 8 orders | P4 table | `… WHERE is_late IS NULL` |
| Reviewed orders | 95,832 (Late 7,661 · On time 88,163 · Unknown 8) | P4 table | `… WHERE review_score IS NOT NULL` |
| Review score late vs on time | avg 2.57 vs 4.29; median 2 vs 5 | P4 chart, table | Query 11 |
| 1–2 star share | Late 54.07% · On time 9.22% · all 12.81% | P4 chart, table | Query 11 |
| Forecast | 6,660 (sum of daily bounds 3,585 and 9,943) | P4 cards | `SELECT sum(forecast), sum(lower), sum(upper) FROM analytics.forecast_28d;` |

## V.J.3 Dashboards that look right but are wrong

A chart can be drawn correctly and still be wrong. These are the defects this model is most exposed to; the original build met several of them.

| Trap | Symptom | How to detect | Fix |
|---|---|---|---|
| A card shows a number but is built from the hidden column instead of the measure | Right on a page with no slicers; wrong or odd when a slicer is used (it sums the visible rows with default aggregation). | Click the visual: the field well should show `Total Revenue` with the calculator icon, not `line_revenue` with a sigma. | Replace the field with the measure. |

| Trap | Symptom | How to detect | Fix |
|---|---|---|---|
| A relationship is inactive or points the wrong way | A slicer changes nothing, or the month axis loses months. | Model view: lines solid, arrows from `dim_date` to the order tables; Manage relationships shows 3 active rows. | Recreate as in V.D. |
| `dim_date` is not marked as the date table | `Revenue MoM %` is blank or wrong; time functions error. | Right-click `dim_date`: "Mark as date table" is ticked. | D.2. |
| `BLANK() = FALSE()` mislabelling | The delivery-status table shows no "Unknown" row and "On time" reads 88,652. | Late 7,826 · On time 88,644 · Unknown 8 must add to 96,478. | Test `ISBLANK` first (V.E.2). |
| `review_score <= 2` instead of `IN { 1, 2 }` | 1–2★ share reads about 13.5% instead of 12.81% (646 unreviewed orders counted). | Overall 12.81%; Late 54.07%; On time 9.22%. | Use `IN { 1, 2 }`. |
| Seller rank over `seller_id` only | The top-10 chart lists far more than ten sellers. | Count the bars: ten. | `ALL ( seller_id, seller_label )`. |
| Category rank without removing the month filter | The matrix shows more than 15 category rows (each month ranks on its own). | Count the rows: 15. | Restore the inner `CALCULATE` with `REMOVEFILTERS ( dim_date[month_start] )`. |
| Payments related to the date table | The date slicer starts changing the payment charts and the shares no longer match 73.92% credit card. | Model view: `order_payments` has no relationship. | Delete the relationship; payments are all statuses by design. |
| Forecast chart built on `forecast_28d` | Only the forecast days appear; no actual line. | Axis field is `forecast_chart[date]`. | Use the calculated table (V.E.2). |
| Wrong state field on page 4 | The state slicer does nothing on delivery visuals. | Slicer field is `vw_orders[customer_state]`. | Swap the field. |
| Locale shows Indian digit grouping | Revenue appears as 1,54,19,773.75. | File → Options → Regional settings, and the report locale. | Set English (United States); this occurred in the original build (VII.7). |
| Card separators look wrong | A card shows R$15.419.773.75 or 96.478 instead of R$15,419,773.75 and 96,478, often only at some zoom levels. | Compare the same card at 70%, 78% and 110% zoom; check General → Properties → Size and Padding. | The format string is not the cause. The 18 pt value is clipped at the bottom of a 78 px card with the default 16 px bottom padding, which cuts off the comma tails. Set bottom padding to 8 px (V.F, page 1 and page 3 cards). |
| Views missing after `make pipeline` | Refresh fails with "relation does not exist", or old numbers remain. | V.A check 3. | Run `make views`, then refresh. |
| Averages of ratios | AOV or retention totals that disagree with the cards. | Use the measures; never average a percentage column. | V.E. |
| Early-2016 values read as findings | A 100% late rate for September 2016, +649,657% growth. | Check the order count behind the month. | Leave the caveats in the subtitles. |
| Cohort heat-map looks "empty" | Only column 0 is coloured. | That is the finding: retention after month 0 is near 0.2–0.7%. Check the colour scale is fixed 0 to 1. | Do not rescale to make it prettier. |

## V.J.4 Final acceptance test

1. With no slicers on page 1: revenue R$15,419,773.75, orders 96,478, AOV R$159.83, latest-month growth −4.1%.
2. Choose state SP on page 1: revenue R$5,769,703.15, orders 40,501. Clear it.
3. Set the date slicer to 2017 only: the monthly chart shows twelve columns and the cards fall. Reset it.
4. On page 2 apply a state: the matrix, AOV and seller visuals change; **the two payment charts do not**. Reset.
5. Page 3: 93,358 · 2,801 · 90,557 · 3.00% and 0.48% month-1. No slicers exist.
6. Page 4: 10.0 days · 8.11% · 7,826 · 96,478. Apply a state: the forecast block does not change.

7. Review table: Late 7,661 / 2.57 / 2 / 54.07%; On time 88,163 / 4.29 / 5 / 9.22%; Unknown 8.
8. Forecast cards 6,660 · 3,585 · 9,943; the forecast line starts on 23 August 2018.
9. Hover three different marks (a column, a bar with a tooltip, a matrix cell): tooltips show values with the stored formats.
10. Press Refresh with the database running: the same numbers return.
11. Save, close Power BI, reopen the file: four pages, same numbers, no credential prompt until the next refresh.

VERIFICATION STATUS

**Reviewed.** The reference values were recomputed from the warehouse; the acceptance steps are the procedure to confirm that Power BI shows them. The no-slicer values (steps 1 and 5–8) were viewed in Power BI Desktop on the cached data and agree; one slicer check was executed (state AL on page 1: R$94,172.49 and AOV R$237.21, matching the model data); the other slicer steps and the refresh (step 10) were not executed.

# Part VI — Findings, limitations and how to explain them

A project is only as good as your ability to say, accurately and without overclaiming, what it found. This part traces every headline number to its source, states the findings in the form you can defend, and lists what the analysis does not show.

## VI.1 KPI trace: from raw data to dashboard to wording

| KPI | Value | Population | SQL / notebook | Dashboard | Doc |
|---|---|---|---|---|---|
| Delivered revenue | R$15,419,773.75 | Delivered order lines (110,197) | Query 01; `analytics.vw_sales` | P1 `Total Revenue` | Memo, findings 4 |
| Delivered orders | 96,478 of 99,441 placed (97.02%) | Order-status funnel | Query 07 | P1 `Total Orders`, P4 `Delivered Orders` | Memo |
| AOV | R$159.83 | Delivered orders | Query 02 | P1 `AOV` | Memo |
| Top-5 category share | 39.25% of 74 groups | Delivered lines | Query 03 | P1 Pareto; `Top 5 Category Share %` | Finding 4 |
| Top state | SP 37.42% | Delivered lines | Query 04 | P1 state chart | Memo |
| Top-10 seller share | 12.93% | Delivered lines | Query 06 | P2 card | Memo |
| Payment mix | credit card 73.92% | **All** 103,886 payment records | Query 05 | P2 payment mix | Memo |
| Repeat rate | 3.00% (2,801 / 93,358) | People with a delivered order | Query 13; notebook 02 | P3 `Repeat Cust %` | Finding 1 |
| Month-1 retention | 0.48% (421 / 87,214) | Cohorts older than the newest | Query 12 | P3 card | Finding 1 |
| RFM | Lapsed 58.29% · Recent 38.71% · At-Risk 1.71% · Champions 1.29% of customers | People | Query 15; notebook 02 | P3 bars | Finding 6 |
| Late rate | 8.11% (7,826 / 96,478) | Delivered orders | Query 09; `vw_orders` | P4 cards | Finding 3 |
| Review gap | 2.57 vs 4.29; 54.07% vs 9.22% 1–2★ | Reviewed, delivered, known lateness | Query 11; notebook 03 | P4 charts and table | Finding 2 |
| Mann–Whitney | n = 7,661 vs 88,163; U = 150,706,850.5; p < 0.001 | As above | Notebook 03 | P4 note P4-T04 | Finding 2 |
| Reconciliation | 303 of 99,441 orders (0.30%) differ by more than R$0.01 | All orders with items and payments | Query 16 | – | Finding 5 |
| Forecast | 6,660 orders, 2018-08-23 to 2018-09-19 | All placed orders (daily) | Notebook 04 | P4 forecast block | Finding 7 |

NOTE

**One RFM calculation everywhere.** Earlier versions of notebook 02 scored recency with `pandas.qcut` and reported Champions at 2.51% of revenue and Lapsed One-Time at 55.89%; the dashboard query (query 15, `NTILE`) gives 2.52% and 55.88%. The gap comes from customers whose last order falls on a quintile-boundary date, which the two methods split differently (IV.8.2). The notebook now reads query 15, and the findings, memo, README and guides quote 2.52% and 55.88%. No conclusion changes.

## VI.2 The seven findings

| 1 · Repeat purchase is rare (3.0%) | |
|---|---|
| **What the data shows** | Only 2,801 of 93,358 people placed a second delivered order. Cohort retention after month 0 is near zero for every cohort of meaningful size (month 1 pooled: 0.48%). |
| **What it does not show** | The marketplace in this data is acquisition-driven. It does **not** show that customers are unhappy or that retention is "broken": the dataset covers about two years and one marketplace, and customers may buy elsewhere or later. |
| **How to say it** | Say: "about 97% of customers buy once in this data; retention past the first month is under one percent." |

| 2 · Late deliveries go with lower reviews | |
|---|---|
| **What the data shows** | Late: median 2, 54.07% rated 1–2★ (n = 7,661). On time: median 5, 9.22% (n = 88,163). Mann–Whitney, one-sided, $p < 0.001$. |
| **What it does not show** | An association between lateness and score. It does not prove that lateness **causes** low scores (VI.4). |
| **How to say it** | Say: "late orders score much lower; the test shows the gap is not chance; this is an association." |

| 3 · Lateness affects 8.11% of delivered orders | |
|---|---|
| **What the data shows** | 7,826 of 96,478 delivered orders arrived after the estimated date; 8 more have an unknown status because there is no delivery date. |
| **What it does not show** | Roughly one delivered order in twelve. It is measured against the estimate Olist gave, not against customer expectations. |
| **How to say it** | Say: "1 in 12 delivered orders was late against its own estimate." |

| 4 · Revenue is concentrated by category | |
|---|---|
| **What the data shows** | The top 5 of 74 category groups (including the blank group) earn 39.25% of delivered revenue; the top 15 reach 76.47%. |
| **What it does not show** | Opportunity and concentration risk. It says nothing about margin because the data has no costs. |
| **How to say it** | Say: "five categories earn almost 40% of revenue"; avoid "profit". |

| 5 · Revenue reconciles | |
|---|---|
| **What the data shows** | 303 orders (0.30%) have item totals that differ from payment totals; item revenue ties exactly to the raw data (R$15,843,553.24 for all statuses). |
| **What it does not show** | The revenue definition (price + freight, delivered) is consistent and defensible. The 0.30% gap is a property of the source (vouchers, split payments), not a pipeline error. |
| **How to say it** | Say: "revenue ties to the source; 0.30% of orders have payment totals that differ from item totals." |

| 6 · RFM: small Champions group | |
|---|---|
| **What the data shows** | Champions: 1.29% of customers, 2.52% of revenue. Recent One-Time: 38.71% of customers, 38.51% of revenue. |
| **What it does not show** | The largest reachable pool is the Recent One-Time group, because it is large and recent. The segment names and thresholds are a project definition (IV.8.2). |
| **How to say it** | Say: “the biggest opportunity is converting recent one-time buyers”, and add that thresholds are a project choice. |

| 7 · The simple forecast was good enough | |
|---|---|
| **What the data shows** | Seasonal naive and Holt-Winters tied (mean WAPE 0.2308 vs 0.2310). Forecast: 6,660 orders over 28 days. |
| **What it does not show** | Choose the simpler model. The daily bounds are not a 28-day interval (VI.5). |
| **How to say it** | Say: “I compared models by walk-forward backtest and chose the simpler one because it tied.” |

## VI.3 The Mann–Whitney test, worded correctly

**Question.** Do late orders tend to receive lower review scores than on-time orders? Review scores are whole numbers 1–5 with many ties and a strongly skewed shape, so a rank-based test is chosen instead of a t-test on means.

**Method.** Mann–Whitney U compares the ranks of the two groups. One-sided alternative: late scores are stochastically *less* than on-time scores. Groups: 7,661 late and 88,163 on-time reviewed orders (the 8 unknown-lateness orders are excluded).

| Quantity | Value | Meaning |
|---|---|---|
| U statistic | 150,706,850.5 | Out of 7,661 × 88,163 = 675,416,743 late-vs-on-time pairs, U counts pairs where the late order scored higher (ties count half). |
| U ÷ number of pairs | 22.3% | In 22.3% of random late/on-time pairs the late order scores higher. |
| Probability of superiority | 77.7% | In 77.7% of pairs the on-time order scores at least as high (ties half). |
| Rank-biserial correlation | about 0.55 | A large effect on the −1 to +1 scale. |
| p-value | **p < 0.001** | The printed `0.000e+00` is floating-point underflow, not a probability of exactly zero. |

> WHY THIS MATTERS
>
> **Wording to use:** “A one-sided Mann–Whitney U test found that late orders scored significantly lower than on-time orders (U = 150,706,850.5, n = 7,661 and 88,163, p < 0.001; median 2 vs 5).” **Wording to avoid:** “p = 0”, “proves lateness causes bad reviews”, “the effect is significant, therefore large”. With samples this large almost any difference is significant; the effect size (medians 2 vs 5; 54% vs 9% at 1–2★) is what matters.

## VI.4 Association is not causation

The data are observational: nobody assigned deliveries to be late at random. Anything that makes an order late may also make the customer unhappy. The repository names three such factors: **distance, seller and freight cost**. Product category, order size and delivery estimates are further plausible ones, but they were not investigated.

| Question | Answer from this project |
|---|---|
| Do late orders have lower reviews? | Yes, by a large margin, and not by chance. |
| Would fixing lateness raise reviews by that margin? | Not shown. This project did not control for distance, seller, freight or category, and used no experiment. |
| What analysis would be needed? | A model or matched comparison that holds those factors fixed (for example, compare late and on-time orders within the same state, seller and category), or a controlled experiment. Not performed. |
| What can the business safely do? | Treat delivery reliability as a promising lever worth testing, and measure the effect of an intervention rather than assuming it. |

## VI.5 Forecast evaluation

- **Metric.** WAPE (weighted absolute percentage error) = total absolute error ÷ total actual orders. Lower is better. It is stable when some days are small, unlike MAPE. **Bias** is the mean signed error (forecast minus actual) in orders per day.
- **Method.** Expanding-window walk-forward backtest: train on the past, forecast the next 28 days, score, then extend the training window and repeat; six folds from 2018-03-08. This mirrors real use better than a single random split.
- **Result.** Seasonal naive (repeat the last observed week) mean WAPE 0.2308, bias −10.14. Holt-Winters 0.2310, bias +2.44. A 0.1% difference is noise, so the simpler model is the recommendation. Holt-Winters has the smaller bias, which WAPE does not capture; it is a point in its favour but not enough to overturn the tie.
- **Black Friday.** 24 November 2017 had 1,176 orders against a baseline near 197 a day (about 6×). It is inside every training window, so the forecast is a typical-week projection; it does not anticipate promotions.
- **Bounds.** The daily lower and upper bounds are the 2.5th and 97.5th percentiles of 168 pooled backtest errors (about −109.8 and +117.2), with the lower bound floored at 0. Their sums (3,585 and 9,943) are not a 28-day interval (V.H). In the six folds the 28-day totals missed by −2,363 to +731 orders.
- **Monitoring.** The monitoring policy (`docs/monitoring_policy.md`) is a **written policy only**; nothing is automated. It proposes refitting every 28 days and investigating when rolling WAPE exceeds 0.34 or absolute bias exceeds 70 orders a day.
- **Cross-platform note.** A fresh Linux run reproduced the same verdict and a byte-identical forecast file; fold-level Holt-Winters numbers differ in the third or fourth decimal because the optimiser's numerics vary by platform (IV.9).

## VI.6 Which population is each number about?

Most "wrong number" arguments are population arguments. Before comparing two figures, check that they describe the same rows.

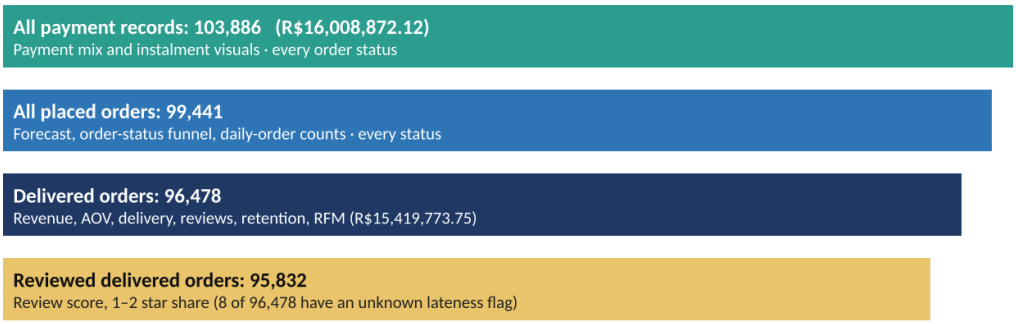



**Figure 15.** Populations used in the project, from all placed orders down to reviewed late orders.

| Population | Size | Used for |
|---|---|---|
| All placed orders | 99,441 | Order-status funnel, reconciliation, daily orders and forecast (634 days with orders) |
| Delivered orders | 96,478 | Revenue, AOV, categories, states, sellers, delivery and review views |
| Delivered orders with a delivery date | 96,470 | Delivery-time median and histogram; late rate denominator in the notebooks |
| Delivered order lines | 110,197 | `vw_sales`; revenue and item-level analysis |
| People with a delivered order | 93,358 | Customers, repeat rate, RFM |
| Reviewed delivered orders | 95,832 | Review scores; Late 7,661 · On time 88,163 · Unknown 8 |
| All payment records | 103,886 | Payment mix and instalments; not delivered-only |

## VI.7 Limitations to state up front

- **One marketplace, about two years.** The data cover 2016–2018 (revenue only from September 2016, and the latest month is August 2018). Results may not transfer to other periods or markets.
- **Delivered orders only for revenue.** Cancelled, unavailable and in-transit orders are excluded from revenue measures by design.
- **No cost or margin data.** “Revenue concentration” is not “profit concentration”.
- **Reviews are optional.** 88.34% of review titles and 58.70% of review comment messages are empty, 646 delivered orders have no review, and reviewers may differ from non-reviewers.
- **Retention has a short window.** Customers who return after the dataset ends cannot be seen; the newest cohorts have few observable months.
- **RFM definitions are a project choice.** Segment names and thresholds are not a standard, and ties at score boundaries are placed arbitrarily, so counts can differ slightly between tools.
- **The forecast is a baseline.** Six folds, no external drivers, no event model, and bounds that are not a total-period interval.
- **Association, not causation** for the lateness–review link.
- **Early-2016 months are tiny** (a handful of orders), so growth and late-rate percentages there are not findings.
- **What was not run here.** Windows commands, Docker Desktop, the Kaggle download, GitHub Actions and a live Power BI refresh were not run (Appendix B).

# VI.8 Explaining the project aloud

## The 90-second version

*"I built an end-to-end analytics project on the Olist Brazilian e-commerce dataset. A Python pipeline loads nine CSVs into PostgreSQL running in Docker, validates them with automated checks, and builds a typed star schema. Eighteen SQL queries and four notebooks answer three questions: where revenue comes from, whether customers return, and whether delivery affects satisfaction. The answers: five categories earn about 39% of delivered revenue; only 3% of customers buy twice; and late orders get much lower reviews — median 2 against 5, a significant difference, though it is an association, not proof of cause. I also forecast the next 28 days of orders, compared two models by walk-forward backtest and kept the simpler one because they tied. Everything is shown in a four-page Power BI report."*

## The 3-minute version: add these points

- **Data quality:** the pipeline stops on blocking checks (row counts, keys, foreign keys, domain values) and only logs soft issues (for example 8 delivered orders with no delivery date).
- **Definitions:** revenue is price plus freight on delivered orders; people are counted by `customer_unique_id`, not `customer_id`.
- **A bug I found:** a DAX test treated blank as false and counted unreviewed orders as low scores; testing blanks explicitly fixed it.
- **Honesty about limits:** the summed forecast bounds are not an interval; the reviews result is an association; there is no margin data.

## Questions you should expect

| Question | A defensible answer |
|---|---|
| Why PostgreSQL in Docker? | It reproduces a realistic warehouse locally with one command, and anyone can rebuild it from the repository. |
| Why Import mode in Power BI? | The data are static and small (about 35 MB in the model), so Import is fast and the report works without the database running; DirectQuery would add nothing. |
| Why Mann–Whitney and not a t-test? | Scores are ordinal, discrete and heavily skewed with many ties; a rank test makes no normality assumption. |
| Is $p < 0.001$ the finding? | No. With 95,000 reviews almost any difference is significant; the finding is the size of the gap (median 2 vs 5; 54% vs 9% at 1–2 stars). |
| Why keep the simpler forecast? | It tied the more complex model on WAPE and is easier to audit and explain. |
| How do you know the revenue is right? | Item revenue ties exactly to the raw data, and 303 orders (0.30%) with different payment totals are explained by vouchers and split payments. |
| What would you do next? | Control for distance, seller and category in the lateness analysis, add margin data, and extend the retention window. |

# Part VII — Troubleshooting and maintenance

Use this part when something fails. Sections VII.2 to VII.7 are **general guidance**, organised by the layer that is failing. Section VII.8 is separate: it records **incidents that actually happened** during the original build, so you can recognise them. Section VII.9 covers routine maintenance and VII.10 covers extending the project safely.

VERIFICATION STATUS

**Verification basis.** Error texts for the Python, SQL, PostgreSQL and Power BI items come from the project's own logs, the QA report and the tool documentation. Docker Desktop, Windows, GNU Make and the Kaggle download were not run, so their rows are Reviewed guidance, not reproduced failures.

## VII.1 A method that works for every problem

1. **Read the first error line, not the last.** Most tools print a long trace; the first line names the cause (for example `password authentication failed`).
2. **Find the layer.** Windows and Docker → PostgreSQL → Python environment → SQL → notebooks → tests → Power BI. A problem in a lower layer shows up as an error in a higher one (a stopped container appears as "connection refused" in Python and as "cannot connect" in Power BI).
3. **Test the layer directly.** `docker compose ps`, then `pg_isready`, then a one-line Python connection, then the SQL, then Power BI.
4. **Change one thing, re-test, write down what you changed.**
5. **Do not reach for `make clean`.** It deletes the database volume. Almost nothing in this project needs it; a rebuild from CSVs takes minutes anyway, but you lose any state you have not recorded (III.6).
6. **Never edit a test or a SQL check to make a failure go away.** A blocking check that fails is telling you the data are not what the pipeline expects.

## VII.2 Docker and PostgreSQL

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| `error during connect … dockerDesktopLinuxEngine: The system cannot find the file specified` | `docker version` shows a Client section but no Server. | Docker Desktop is not running. | Start Docker Desktop and wait until it reports the engine is running (whale icon steady). | `docker info` prints a Server section. |
| `docker` is not recognised | `where docker` | Not installed, or the terminal was opened before the install. | Reopen the terminal; if still missing, reinstall Docker Desktop (II.6). | `docker --version` |
| Docker Desktop says WSL 2 or virtualisation is missing | Docker's start-up message; Task Manager → Performance → CPU shows Virtualization: Enabled? | Hardware virtualisation off in BIOS/UEFI, or WSL 2 not installed. | Enable virtualisation in BIOS; install WSL 2 as Docker's installer instructs. | Docker Desktop starts. |
| `port is already allocated` or `Only one usage of each socket address` | `netstat -ano \| findstr :5432` | Another PostgreSQL (or this project's old container) uses port 5432. | Stop the other service, or set `DB_PORT=5433` in `.env`, run `docker compose down`, start again, and use the same port in Power BI. | `docker compose ps` shows the new port mapping. |
| Container `unhealthy` or restarting | `docker logs olist_postgres` | Corrupted volume, a bad environment value, or a full disk. | Read the last log lines. If the volume is damaged and you accept losing the database, follow III.6 to rebuild. | `pg_isready` accepts connections. |
| `password authentication failed for user …` | `.env` password versus the password used when the volume was first created. | Docker applies `DB_PASSWORD` **only when it creates the volume**. | Put the original password back in `.env`, or change it inside the database as shown in III.7. | Python connects; the one-line test in step 10.1 works. |

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| `DB_USER`/`DB_PASSWORD` variable is not set, defaulting to a blank string | The warning appears when you run `docker compose`. | Compose was run outside the project folder, or `.env` is missing. | `cd` to the repository root; confirm `.env` exists (do not print it). | No warning; container starts. |
| Python: `could not connect to server: Connection refused` | `docker compose ps`; `DB_HOST` and `DB_PORT` in `.env`. | Container stopped, or the port in `.env` differs from the mapped port. | Start the container; align the port. | `pg_isready` and Python both succeed. |
| Data “disappeared” after a restart | `docker volume ls`; `docker compose ps` | Container was removed with `down -v` (deletes the volume) or a different compose project name created a new volume. | Restore by re-running the pipeline (III.6). Use `docker compose down` without `-v`. | Counts match the table in III.2 step 8. |

## VII.3 SQL, the pipeline and the Makefile

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| `relation "stg.orders" does not exist` | `docker exec … psql -c "\dt stg.*"` | Ingestion has not run or failed. | Run the ingest step (PLAYBOOK step 5, III.2), then continue. | Nine `stg` tables with the row counts of step 6. |
| Input checks stop the pipeline (blocking failure) | The log names the check: row count, primary key, foreign key or domain value. | A CSV is missing, truncated, edited, or replaced by the synthetic fixtures. | Compare file sizes with the table in II.8; restore the original CSVs; re-run. | `make test` input tests pass. |
| Row counts are 100 orders instead of 99,441 | `docker exec … psql -c "SELECT count(*) FROM stg.orders;"` | `make fixtures` wrote synthetic CSVs over `data/raw`. | Re-obtain the real CSVs (II.8) and re-run the pipeline. | Staging counts 99,441 / 112,650 / … |
| Output checks fail after the transform | The log names the check. | A transform step was interrupted or the staging data changed. | Re-run `make pipeline`; do not edit the SQL. | Output tests pass. |
| Pipeline succeeds but Power BI or Excel fail: relation `analytics.vw_*` missing | `SELECT count(*) FROM information_schema.views WHERE table_schema='analytics';` | The pipeline drops the views with `DROP … CASCADE`. | `make views`. | Four views, counts in V.A. |
| `make: command not found` or `make is not recognized` | `where make` | GNU Make is not installed on Windows. | Install with Scoop or Chocolatey (II.7) or use the no-make commands in the equivalents table. | `make --version`. |
| `make clean` fails on `rm` or `find` | The Makefile uses Unix commands in that target. | Windows has no `rm`/`find` in the default shell (not tested here). | Do not use `make clean` on Windows; remove generated files by hand. | – |
| `UnicodeDecodeError` while loading a CSV | Open the file size and compare with II.8. | The CSV was opened and re-saved in Excel. | Restore the original file; never save raw CSVs from Excel. | Ingestion completes. |
| SQL result differs by a few rows from this guide | Compare the counts in III.2 and IV.5. | Different dataset version. | Check the file sizes in II.8; the guide’s numbers are for the files listed there. | Counts match. |

## VII.4 Windows, Python, Git and Kaggle

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| `python` opens the Microsoft Store or prints nothing | `where python`; `py -0` | Windows “app execution alias” intercepts `python`. | Use the `py` launcher (`py -3.12`) or turn the alias off (Settings → Apps → Advanced app settings → App execution aliases). | `python --version` prints 3.12.x. |

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| `Activate.ps1 cannot be loaded because running scripts is disabled` | Error text in PowerShell. | Execution policy blocks the venv activation script. | Run once: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`, or skip activation and call `.\.venv\Scripts\python.exe` directly. | Prompt shows `(.venv)` or the direct call runs. |
| `No matching distribution found for …` during `pip install` | `python --version` | Python too new or too old for a pinned package. | Use Python 3.12 (the version the project's CI uses). | `pip install -r requirements.txt` completes. |
| `ModuleNotFoundError: No module named 'src'` | `pwd` | Running from the wrong folder or as a script. | `cd` to the repository root and run `python -m src.<module>`. | Module starts. |
| `401 Unauthorized` / "Could not find kaggle.json" | `dir $env:USERPROFILE\.kaggle` | Kaggle credentials missing, or the dataset terms not accepted. | Create credentials as in II.8, or download the dataset by hand and unzip into `data/raw` (the ingest skips the download when all nine CSVs exist). | Nine CSVs in `data/raw`. |
| `fatal: … Filename too long` on clone | Error text. | Windows path-length limit. | `git config --global core.longpaths true`, clone again. | Clone finishes. |
| `LF will be replaced by CRLF` warnings | Appear on `git add`. | Line-ending conversion. | Harmless; ignore. | – |
| `.git/index.lock exists` | `dir .git\*.lock` | A previous Git process was interrupted. | Confirm no Git process is running, then delete the lock file by hand. | `git status` works. |

## VII.5 Notebooks

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| Kernel error or `No module named …` in a notebook | Which Python started the kernel? | Notebooks launched outside the venv. | Use `make notebooks` or `jupyter nbconvert --execute` from the venv (III.8). | Notebook runs to the end. |
| Notebook cannot connect to the database | Same one-line connection test. | Container stopped or `.env` mismatch. | VII.2. | Connection test passes. |
| Notebook 04 asserts about missing days | Message names the window. | The raw data differ from the dataset used in this guide. | Check the CSV sizes (II.8) and row counts. | Assertion passes. |
| Forecast fold numbers differ in the third decimal | Compare `forecast_28d.csv`. | Optimiser numerics differ by platform. | No action; the verdict and the forecast file should still match (IV.9). | Seasonal naive 0.2308 / Holt-Winters 0.2310. |
| `git status` shows notebooks modified | `git diff --stat` | Executing a notebook rewrites its saved outputs. | Keep the new outputs if you intend to, otherwise `git checkout -- notebooks/<name>.ipynb`. | Clean `git status`. |
| Notebook runs but the `forecast_28d` table is stale | Query its dates. | Notebook 04 was not re-run. | Re-run notebook 04, then refresh Power BI. | 28 rows, latest dates. |

## VII.6 Tests, linting and compilation

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| `pytest` is not recognised | `where pytest` | The venv is not active. | `.\.venv\Scripts\python.exe -m pytest -q`. | Tests are collected. |
| Tests error on connection | VII.2 checks. | Database not running. | Start it, then re-run. | 10 tests pass. |
| Tests fail right after `make fixtures` | Row counts are 100 orders. | Fixtures replaced the data. | Restore the real CSVs and re-run the pipeline. | Real-data counts return. |

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| “Is it safe to run the tests?” | – | – | Yes. The 10 tests are SELECT-only: they read the database and never modify it. | Counts unchanged afterwards. |
| “Which linter should I run?” | Look for `ruff/flake8` config. | None is configured. | None is required. Optional safe check: `python -m compileall src tests`. | No syntax errors. |

## VII.7 Power BI

| Symptom | Diagnostic | Cause | Solution | Verification |
|---|---|---|---|---|
| Refresh fails with PostgreSQL error `53100` (“could not resize shared memory segment”) | Error text in the refresh dialog. | Parallel query workers need more shared memory than Docker’s 64 MB `/dev/shm`. | Keep `SET max_parallel_workers_per_gather = 0;` at the start of each native query, or add `shm_size: 1gb` to the PostgreSQL service and recreate the container. | Refresh completes; row counts V.B.5. |
| “Couldn’t authenticate with the credentials provided” | Does the same user and password work in `psql` / Python? | Wrong password, or the volume was created with a different one. Power BI’s driver once rejected the default scram-sha-256 method; the container therefore starts with md5 (II.10). | Re-enter credentials (Data source settings → Edit Permissions → Database tab). If the volume predates the md5 setting, rebuild the volume (III.6). | Refresh succeeds. |
| `relation "analytics.vw_sales" does not exist` | V.A check 3. | Views dropped by the pipeline. | `make views`, refresh. | Four views. |
| Native-query approval dialog appears | Dialog titled about a native database query. | Power Query asks before running native SQL. | Check the text is `SET …; SELECT * FROM analytics.<name>` and click **Run**. | Query loads. |
| A slicer does nothing | Model view. | Relationship missing, inactive, or the slicer field is in an unrelated table. | V.D and V.G. | Cards change. |
| `Revenue MoM %` is blank everywhere | Marked date table? | `dim_date` not marked or the month is not on the axis. | D.2; place `month_start` on the axis. | Aug 2018 −4.12%. |
| “A table of multiple values was supplied where a single value was expected” | Which measure? | A formula returns a column or table where one value is needed. | Compare with the verbatim formula in V.E.3. | Measure returns a number. |
| Numbers show Indian digit grouping | Regional settings. | The machine locale was applied. | English (United States) under Options → Current file → Regional settings. | R$15,419,773.75. |
| Totals differ from this guide | V.J.2 and V.J.3. | A trap in V.J.3, or the database is not the real one. | Follow the traps table. | Reference values match. |
| “Save as” fails or the file is locked | Is it open elsewhere? | Another process holds the file. | Close the other program, retry; save under a new name. | File saved. |
| Forecast chart shows old dates | Date range in the chart. | `forecast_28d` or the report is stale. | Re-run notebook 04, refresh. | Chart begins 2018-08-23. |

## VII.8 Incidents from the original build (historical record)

These really happened while the dashboard was built; they are kept as evidence and as a list of things to look for. They are recorded in `dashboard/BUILD_QA_REPORT.md`. Each was fixed in the final report.

| Incident | What happened | Cause | Resolution | Lesson |
|---|---|---|---|---|
| Shared-memory error on refresh | Loading from the Docker database failed with error 53100. | Docker's 64 MB `/dev/shm` is too small for parallel query workers. | The `SET max_parallel_workers_per_gather = 0;` prefix in every native query; parallel loading set to One. | A container's defaults differ from a server's. |
| On-time count 88,171 instead of 88,163 | The dashboard disagreed with notebook 03 by 8 orders. | Calculated column tested `is_late = FALSE`; in DAX a blank equals FALSE, so the 8 undelivered orders became "On time". | Return "Unknown (no delivery date)" when `is_late` is blank (V.E). | Reconcile every dashboard number with an independent source. |
| `Pct 1-2 Star` too high | Share of low reviews was inflated. | `review_score <= 2` also matches blank scores. | Use `IN { 1, 2 }` over reviewed orders. | Blank is not "nothing" in DAX. |
| `Seller Rank` returned 1 for every seller | The top-10 chart was wrong. | `ALL ( seller_id )` left the axis filter on `seller_label` in place. | `ALL ( seller_id, seller_label )`. | Remove every filter the visual adds. |
| Revenue shown as 1,54,19,773.75 | Indian digit grouping on the revenue card. | The author's machine locale (en-IN) was applied. | Report locale set to English (United States). | Set locale explicitly; verified on the author's machine only. |
| A native query was rejected | Power BI's query box refused a statement that dropped views. | Native-query boxes accept read statements. | The four views were created from the repository with `make views`. | Create database objects outside Power BI. |
| Power BI rejected valid credentials | Connection failed although the password was right. | The driver did not accept the default scram-sha-256 method. | `POSTGRES_INITDB_ARGS: "--auth-host=md5"` in the compose file (applies when the volume is created). | Authentication method is a creation-time setting. |
| Save As and Git lock files | "Save as" was blocked; Git reported stale locks. | Another process held the file; zero-byte `.git/*.lock` files were left behind. | Closed the other process; removed the stale locks. | Check for open handles before saving. |

## VII.9 Maintenance

| When | Do | Why |
|---|---|---|
| After every `make pipeline` | `make views` | The pipeline drops the four dashboard views. |
| After notebook 04 | Refresh Power BI | The forecast table and calculated table change. |
| Before relying on a result | Run `make test` | Ten read-only checks on the loaded data. |
| After a Windows restart | Start Docker Desktop, `docker compose up -d --wait`, `docker compose ps` | The container does not start itself unless Docker is set to launch with Windows (III.5). |
| After changing `.env` | Remember the password only applies at volume creation | III.7. |
| Monthly | `git status`, `git pull`; check Docker Desktop updates | Keep tools current. |
| Disk getting full | `docker system df`; remove unused images only with care | Docker images and volumes grow. |
| Dependency updates | Change `requirements.txt` in a copy, run `make test` and the notebooks, compare the verdicts | Package updates can change numerics (IV.9). |
| The database | Treat as disposable: the code and CSVs are the source of truth | A rebuild is one pipeline run. |

## VII.10 Extending the project safely

Add things; do not rewrite existing ones. After any change run the checks in the right-hand column.

| To add | How | Safe-change checklist |
|---|---|---|
| A new SQL analysis | New file in `sql/03_analysis/` with a `-- QUESTION:` line, read-only, numbered next. | Run `make analysis`; check its CSV; add the result to the findings only after recomputing it. |
| A new dashboard view | New file in `sql/04_dashboard_views/`; register it where `build_dashboard_views` lists the files. | `make views`; row count; relate or deliberately not relate it in V.D. |
| A new measure | Create in `_Measures` with a display folder and format; follow the ten-element pattern in V.E.3. | Test with no slicer and with SP; add it to the reference table. |
| A new visual | Follow V.F.1; give it alt text and a tab-order position. | Compare with SQL; run the slicer tests in V.G. |
| A new test | Add a SELECT-only check under `tests/`. | `make test`; never write to the database from a test. |
| A new notebook | Copy the structure of an existing one; read only from `analytics`. | Run top to bottom on a clean kernel. |
| A change to a definition (for example revenue) | Change it in one place and trace every consumer with the KPI table in VI.1. | Update the dashboard measure, the SQL view, the notebooks and the docs together. |

# Part VIII — Hands-on practice

Reading is not learning. These exercises make you do each skill once. Try an exercise before reading its hint, and its hint before its answer. Every numeric answer was recomputed from the warehouse.

## VIII.1 Exercises with hints and answer keys

| Exercise 1 · Prove the database is healthy | |
|---|---|
| **Task** | Without opening Power BI, show that PostgreSQL is running and that the nine staging tables hold the expected rows. |
| **Hint** | You need `docker compose ps` and a `psql` query through `docker exec`. The expected counts are in the staging table in II.8. |
| **Answer key** | `docker compose ps` shows `olist_postgres` healthy; `docker exec olist_postgres psql -U analyst -d olist -Atc "SELECT count(*) FROM stg.orders;"` prints `99441`; the nine counts are 99,441 · 1,000,163 · 112,650 · 103,886 · 99,224 · 99,441 · 32,951 · 3,095 · 71. |

| Exercise 2 · What does `make pipeline` silently destroy? | |
|---|---|
| **Task** | Name the objects it drops that the dashboard needs, and the command that restores them. |
| **Hint** | Think about `DROP … CASCADE` and which objects depend on the tables. |
| **Answer key** | The four views `analytics.vw_sales`, `vw_orders`, `vw_customers` and `vw_daily_orders`. `make views` (or `python -m src.build_dashboard_views`) recreates them. `forecast_28d` is not touched. |

| Exercise 3 · Run the pipeline without Make | |
|---|---|
| **Task** | Write the PowerShell commands to run the pipeline, the views and the tests from the repository root. |
| **Hint** | Use the equivalents table in II.7. |
| **Answer key** | `.\.venv\Scripts\python.exe -m src.run_pipeline`, then `.\.venv\Scripts\python.exe -m src.build_dashboard_views`, then `.\.venv\Scripts\python.exe -m pytest -q` (10 passed). |

| Exercise 4 · Monthly revenue for 2017 | |
|---|---|
| **Task** | Write a SQL query for delivered revenue per month in 2017 using `analytics.vw_sales`. Which month is highest and what is it? |
| **Hint** | Use `date_trunc`, `sum(line_revenue)` and a half-open date range. |
| **Answer key** | <pre>SELECT date_trunc('month', order_date)::date AS month,<br>       round(sum(line_revenue), 2) AS revenue<br>FROM analytics.vw_sales<br>WHERE order_date >= '2017-01-01' AND order_date < '2018-01-01'<br>GROUP BY 1 ORDER BY 1;</pre> Twelve rows; the highest is **November 2017, R$1,153,364.20**. January is R$127,482.37. |

| Exercise 5 · Repeat customers in SQL | |
|---|---|
| **Task** | How many people are repeat customers, out of how many? |
| **Hint** | The view `analytics.vw_customers` has one row per person and a Boolean `is_repeat`. |
| **Answer key** | `SELECT count(*) FILTER (WHERE is_repeat), count(*) FROM analytics.vw_customers;` returns **2,801** and **93,358**, so 3.00%. |

| Exercise 6 · The 1–2 star share, and why SQL and DAX can disagree | |
|---|---|
| **Task** | Compute the share of reviewed orders with 1–2 stars in SQL. Then predict what the DAX measure would show if it used `review_score <= 2` in `CALCULATE`. |
| **Hint** | In SQL a NULL never satisfies `<= 2`. In DAX a blank behaves like 0 in that comparison. There are 646 delivered orders without a review. |
| **Answer key** | SQL: `SELECT round(100.0 * count(*) FILTER (WHERE review_score IN (1,2)) / count(review_score), 2) FROM analytics.vw_orders;` → **12.81**. The DAX `<= 2` form would add the 646 unreviewed orders to the numerator: (12,273 + 646) ÷ 95,832 ≈ **13.48%**, which is wrong. The project uses `IN { 1, 2 }`. |

| Exercise 7 · Late rate with a different denominator | |
|---|---|
| **Task** | Write a measure `Late Delivery % (known only)` whose denominator counts only orders with a known lateness. What does it show and why does the card barely change? |
| **Hint** | Use `NOT ISBLANK ( vw_orders[is_late] )` inside `CALCULATE`. |
| **Answer key** | `DIVIDE ( [Late Orders], CALCULATE ( COUNTROWS ( vw_orders ), NOT ISBLANK ( vw_orders[is_late] ) ) )` → 7,826 ÷ 96,470 = 8.1124%, displayed **8.11%**, the same as the original at two decimals because only 8 orders differ. Which denominator is right is a definition choice; state it. |

| Exercise 8 · Effect size from the Mann–Whitney U | |
|---|---|
| **Task** | Given U = 150,706,850.5, $n_1$ = 7,661 late and $n_2$ = 88,163 on-time orders, compute U ÷ ($n_1$ × $n_2$), the probability of superiority and the rank-biserial correlation. |
| **Hint** | The number of pairs is $n_1 \times n_2$ = 675,416,743. Rank-biserial = 1 − 2U ÷ ($n_1$ × $n_2$). |
| **Answer key** | U ÷ pairs = 0.2231 (22.3%). Probability that an on-time order scores higher or equal = 77.7%. Rank-biserial = 1 − 2 × 0.2231 = **0.554**, a large effect. |

| Exercise 9 · Why does `np.tile(last_week, 4)` forecast 28 days? | |
|---|---|
| **Task** | Explain the seasonal-naive forecast in one sentence. |
| **Hint** | 28 = 4 × 7. |
| **Answer key** | It repeats the last observed 7-day pattern four times, so each forecast day equals the order count of the same weekday one cycle earlier. |

| Exercise 10 · Build three measures and a card | |
|---|---|
| **Task** | Create `Total Revenue`, `Total Orders` and `AOV` (V.E.3), put them on three cards, then apply the state slicer SP. |
| **Hint** | Use the formulas verbatim; check the format codes. |
| **Answer key** | No slicer: R$15,419,773.75 · 96,478 · R$159.83. SP: R$5,769,703.15 · 40,501 · R$142.46. |

| Exercise 11 · Break and repair a measure | |
|---|---|
| **Task** | Change `Pct 1-2 Star` so that its numerator uses `vw_orders[review_score] <= 2`. Compare the overall value with 12.81% and then restore the original. |
| **Hint** | Expect a value near 13.5%. |
| **Answer key** | About 13.48%. The numerator now includes blanks. Restoring `IN { 1, 2 }` returns 12.81%. |

| Exercise 12 · See the Unknown orders | |
|---|---|
| **Task** | Build a table of `vw_orders[delivery_status]` against `Delivered Orders`. What should it show? What would show instead if the `ISBLANK` test were moved after the `FALSE` test? |
| **Hint** | Unknown is the 8 delivered orders without a delivery date. |
| **Answer key** | Late 7,826 · On time 88,644 · Unknown 8. With the wrong order the Unknown row disappears and On time reads 88,652. |

| Exercise 13 · Why does the payment chart ignore the slicers? | |
|---|---|
| **Task** | Find the evidence in the model and say whether you would change it. |
| **Hint** | Open Model view and look for lines touching `order_payments`. |
| **Answer key** | `order_payments` has no relationship, so no slicer can filter it. It is deliberate: payments cover every order status, delivered revenue does not. Relating it to the date table would mix two populations, so you should not change it. |

| Exercise 14 · Read the cohort heat-map | |
|---|---|
| **Task** | Why does the heat-map form a triangle, and why is everything right of column 0 almost white? |
| **Hint** | Think about how many months each cohort has had to be observed, and about the 3% repeat rate. |
| **Answer key** | Newer cohorts have fewer observable months, so their later cells do not exist. Column 0 is 100% by definition; later cells are 0.2–0.7% because only 3% of people ever buy again. The colour scale is fixed 0 to 1, so those cells stay pale: that is the finding, not an error. |

| Exercise 15 · Fix an over-claim | |
|---|---|
| **Task** | Rewrite: “Late deliveries cause bad reviews (p = 0), so fixing delivery will raise the average score by 1.7 points.” |
| **Hint** | Three problems: causation, p = 0, and a forecast of the effect. |
| **Answer key** | “Late orders score significantly lower than on-time orders (one-sided Mann–Whitney U, p < 0.001; median 2 versus 5). This is an association: distance, seller and freight cost may contribute, so the improvement from fixing lateness has to be tested rather than assumed.” |

| Exercise 16 · Same name, different population | |
|---|---|
| **Task** | A colleague asks why `Total Payment Value` (R$16,008,872.12) is not equal to `Total Revenue` (R$15,419,773.75). Answer. |
| **Hint** | List the populations. |
| **Answer key** | Payments cover all 103,886 payment records of every order status and include vouchers and instalment payments; revenue is price plus freight on delivered orders only. 303 orders also have item and payment totals that differ. The two are different populations and are not expected to match. |

| Exercise 17 · Extend safely | |
|---|---|
| **Task** | You want a new chart of late rate by state. List the steps. |
| **Hint** | Use the extending checklist in VII.10 and the recipe pattern in V.F.1. |
| **Answer key** | Check query 10 for the answer in SQL first; add a bar chart on page 4 with `vw_orders[customer_state]` on the axis and `Late Delivery %` as value; filter out states with fewer than 30 orders (as query 10 does); give it alt text and a tab-order position; compare with query 10 (AL 23.93%, MA 19.67%, PI 15.97%); add it to the V.J reference table. |

## VIII.2 A staged learning plan

Nine stages. Move on when the right-hand column is true for you, not when the calendar says so.

| Stage | Focus | Read | You are ready when |
|---|---|---|---|
| 1 | Orientation | Part I; skim Part II | Explain the architecture in two minutes |
| 2 | Set up and run | Part II and Part III.2–III.4 | Run the pipeline and tests yourself; Exercises 1–3 |
| 3 | SQL | Part IV.4–IV.6 | Exercises 4–6; write one new query |
| 4 | Python and statistics | Part IV.7–IV.8, Part VI.3 | Exercises 8–9; explain Mann–Whitney aloud |
| 5 | Power BI model | V.A–V.D | Rebuild the model in a copy; model checklist passes |
| 6 | DAX | V.E | Exercises 10–12; write all 43 measures from the cards |
| 7 | Report pages | V.F–V.G | Rebuild one page, then all four; acceptance test in V.J.4 |
| 8 | Findings and interview | Part VI | Exercises 15–16; give the 90-second explanation without notes |
| 9 | Troubleshooting and extension | Part VII; Exercise 17 | Break something on purpose and fix it |

## VIII.3 “I can explain it unaided” checklist

Tick each line only if you can do it aloud, without notes.

| ☐ | I can... |
|---|---|
| ☐ | Draw the architecture and say what each stage reads and writes. |
| ☐ | Start the project after a Windows restart without looking anything up. |
| ☐ | Say which commands destroy data and which are safe. |
| ☐ | Explain the grain of each analytics table and why `customer_unique_id` is the person key. |
| ☐ | Explain a cohort table and read the heat-map. |
| ☐ | Explain RFM and why the segment names are a project choice. |
| ☐ | Explain Mann–Whitney, report it as $p < 0.001$ and state the effect size. |
| ☐ | Explain walk-forward backtesting, WAPE and why the simpler forecast was kept. |
| ☐ | Say why summed daily bounds are not a 28-day interval. |
| ☐ | Explain filter context, row context and context transition with one example each. |
| ☐ | Explain the `BLANK() = FALSE()` trap and how the model avoids it. |

| ☐ | I can… |
|---|---|
| ☐ | Name the three relationships and say why the other tables are not related. |
| ☐ | Rebuild any measure from its description and check it against SQL. |
| ☐ | Say which slicers affect which visuals on each page and why. |
| ☐ | Name the population behind each headline figure. |
| ☐ | Spot, by eye, a dashboard that looks right but is wrong (V.J.3). |
| ☐ | State what association does and does not prove. |
| ☐ | List what was not verified (Appendix B). |

# Appendix A — Command reference

From the repository root in PowerShell. “Make” column shows the equivalent when GNU Make is installed.

| Task | Command | Make |
|---|---|---|
| Start the database | `docker compose up -d --wait` | `make db-up` |
| Container state | `docker compose ps` | – |
| Readiness | `docker exec olist_postgres pg_isready -U analyst -d olist` | – |
| Stop (keeps data) | `docker compose down` | `make db-down` |
| Run the pipeline | `.\.venv\Scripts\python.exe -m src.run_pipeline` | `make pipeline` |
| Run the 18 queries | `.\.venv\Scripts\python.exe -m src.run_analysis` | `make analysis` |
| Create dashboard views | `.\.venv\Scripts\python.exe -m src.build_dashboard_views` | `make views` |
| Write the Excel summary | `.\.venv\Scripts\python.exe -m src.build_excel_summary` | `make excel` |
| Run the tests | `.\.venv\Scripts\python.exe -m pytest -q` | `make test` |
| Run a notebook | `.\.venv\Scripts\python.exe -m jupyter nbconvert --to notebook --execute --inplace notebooks/<name>.ipynb` | `make notebooks` (all four) |
| Syntax check (optional) | `.\.venv\Scripts\python.exe -m compileall src tests` | – |
| Count staging rows | `docker exec olist_postgres psql -U analyst -d olist -Atc "SELECT count(*) FROM stg.orders;"` | – |
| Volumes | `docker volume ls` | – |
| **Destructive:** replace raw CSVs with fixtures | `.\.venv\Scripts\python.exe tests/fixtures/generate_fixtures.py data/raw` | `make fixtures` |
| **Destructive:** delete the database volume | `docker compose down -v` | `make clean` (also deletes files) |

# Appendix B — What was verified, and how

This appendix lists what each part of the guide rests on and what was not checked. Labels: **Executed** (the command was run and its output observed), **File-verified** (read from the final report file or repository), **Reviewed** (from documentation, repository text or screenshots), **Not verified**.

| Area | Status | Basis | A safe check you can make |
|---|---|---|---|
| Python pipeline, tests, 18 queries, four views, Excel summary, four notebooks | Executed | Run on Linux against PostgreSQL 16.2 (not the Docker container) with the real CSVs: pipeline 33.7 s; tests 10 passed; staged 18-step route 43 s; notebooks 4–6 s each | Run the same commands on Windows and compare counts (III.2) |
| Fixture route and CI test route | Executed | Separate second database; real data untouched | – |
| Row counts, reference values, statistics, forecast | Executed | Recomputed from the warehouse; Mann–Whitney from notebook 03 inputs; a fresh run reproduced the verdicts | V.J.2 |
| Final Power BI model and visuals | File-verified | Read from the final `.pbix` (tables, formulas, formats, relationships, visual fields, positions, formatting) | Open the file and compare with V.C–V.F |
| DAX results | Executed (viewed) | SQL-computed equivalents; the no-slicer card, bar and table values were also viewed in Power BI Desktop on the cached data and agree | Acceptance test V.J.4 |
| Power BI refresh against the Docker database | Not verified | The final file was opened and viewed and one page-1 slicer (state AL) was checked; no refresh against the database | V.J.1 |
| Power BI menu paths | Reviewed | Microsoft Learn; paths vary by release | Use the Format-pane search |
| “Parallel loading = One” setting | Not verified | Recorded in the QA report only; not in the model file | Not needed to reproduce |
| Sort-by-column assignment and `Dummy` column visibility | Not verified | Not exposed by the extraction | Look at the instalment chart order |
| Line colours of P2-V07 and P4-V08 | Executed (viewed) | Power BI Desktop draws both lines in the theme blue `#118DFF`; the stored teal and coral are not applied | V.I.1 |
| KPI card thousands separators | Executed (viewed) | Bottom padding 8 px on the eight 78 px KPI cards; commas checked after save, close and reopen at 70%, 78%/80% and 110% zoom and after page changes, from the working copy and again from the final path | V.I.1 |
| Text contrast | Computed and viewed | WCAG ratios computed from stored colours: page-background text `#5B6472` on `#F3F5F8` is 5.48:1 (was `#6B7280`, 4.43:1); rendered colour confirmed in the exported report | V.I.2 |
| Windows PowerShell commands, GNU Make on Windows, Docker Desktop, md5 authentication in the container, Docker volume name | Not verified | Same commands executed on Linux; Windows behaviour from documentation and the compose file | Part II checklists; `docker volume ls` |
| Kaggle download | Not verified | The ingest skips the download when all nine CSVs exist, so a manual download is equivalent | Compare file sizes (II.8) |
| GitHub Actions CI | Not verified | Workflow read, not run | – |
| Timing on Windows (3–8 minutes in the PLAYBOOK) | Not verified | Linux timings only | Time your own run |
| Screen-reader walk-through, high-contrast theme | Not performed | Not part of the project’s QA | – |

# Appendix C — Index of the 43 measures and 44 visuals

## Measures

| # | Measure | Folder | Format | Page |
|---|---|---|---|---|
| 1 | `Total Revenue` | Revenue | `\R\$#,0.00` | 63 |
| 2 | `Total Orders` | Revenue | `#,0` | 63 |
| 3 | `AOV` | Revenue | `\R\$#,0.00` | 64 |
| 4 | `Revenue MoM %` | Revenue | `0.0%;-0.0%;0.0%` | 64 |
| 5 | `Latest Month Revenue MoM %` | Revenue | `0.0%;-0.0%;0.0%` | 65 |
| 6 | `Category Rank` | Revenue | `0` | 66 |
| 7 | `Top Category Revenue` | Revenue | `\R\$#,0.00` | 67 |
| 8 | `Category Cumulative Share %` | Revenue | `0.0%;-0.0%;0.0%` | 67 |
| 9 | `Top 5 Category Share %` | Revenue | `0.00%;-0.00%;0.00%` | 68 |
| 10 | `Seller Rank` | Revenue | `0` | 69 |
| 11 | `Top 10 Seller Revenue` | Revenue | `\R\$#,0.00` | 69 |
| 12 | `Top 10 Seller Share %` | Revenue | `0.00%;-0.00%;0.00%` | 70 |
| 13 | `Payment Records` | Payments (all payment records) | `#,0` | 70 |
| 14 | `Payment Record Share %` | Payments (all payment records) | `0.00%;-0.00%;0.00%` | 71 |
| 15 | `Avg Installments` | Payments (all payment records) | `0.00` | 71 |
| 16 | `Total Payment Value` | Payments (all payment records) | `\R\$#,0.00` | 72 |
| 17 | `Customers` | Customers & Retention | `#,0` | 72 |
| 18 | `Repeat Customers` | Customers & Retention | `#,0` | 73 |
| 19 | `One-Time Customers` | Customers & Retention | `#,0` | 73 |
| 20 | `Repeat Cust %` | Customers & Retention | `0.00%;-0.00%;0.00%` | 74 |
| 21 | `Cohort Active Customers` | Customers & Retention | `#,0` | 74 |
| 22 | `Retention %` | Customers & Retention | `0.0%;-0.0%;0.0%` | 75 |
| 23 | `Month-1 Retention % (weighted)` | Customers & Retention | `0.00%;-0.00%;0.00%` | 75 |
| 24 | `RFM Customers` | Customers & Retention | `#,0` | 76 |
| 25 | `RFM Revenue` | Customers & Retention | `\R\$#,0.00` | 76 |
| 26 | `RFM Customer Share %` | Customers & Retention | `0.00%;-0.00%;0.00%` | 77 |
| 27 | `RFM Revenue Share %` | Customers & Retention | `0.00%;-0.00%;0.00%` | 77 |
| 28 | `Delivered Orders` | Operations & Satisfaction | `#,0` | 78 |
| 29 | `Late Orders` | Operations & Satisfaction | `#,0` | 78 |
| 30 | `Late Delivery %` | Operations & Satisfaction | `0.00%;-0.00%;0.00%` | 79 |
| 31 | `Median Delivery Days` | Operations & Satisfaction | `0.0` | 79 |
| 32 | `Avg Review Score` | Operations & Satisfaction | `0.00` | 80 |
| 33 | `Median Review Score` | Operations & Satisfaction | `0.0` | 80 |
| 34 | `Reviewed Orders` | Operations & Satisfaction | `#,0` | 80 |
| 35 | `Pct 1-2 Star` | Operations & Satisfaction | `0.00%;-0.00%;0.00%` | 81 |
| 36 | `Orders With Delivery Time` | Operations & Satisfaction | `#,0` | 81 |
| 37 | `Forecast Total (28 days)` | Forecast (all placed orders) | `#,0` | 82 |

| # | Measure | Folder | Format | Page |
|---|---|---|---|---|
| 38 | `Forecast Lower Sum` | Forecast (all placed orders) | `#,0` | 82 |
| 39 | `Forecast Upper Sum` | Forecast (all placed orders) | `#,0` | 83 |
| 40 | `Actual (last 90 d)` | Forecast (all placed orders) | `#,0` | 83 |
| 41 | `Forecast` | Forecast (all placed orders) | `#,0.0` | 84 |
| 42 | `Lower 95%` | Forecast (all placed orders) | `#,0.0` | 84 |
| 43 | `Upper 95%` | Forecast (all placed orders) | `#,0.0` | 85 |

## Visuals

| ID | Type | Title | Page |
|---|---|---|---|
| P1-V01 | Slicer | Order date range | 88 |
| P1-V02 | Slicer | Customer state | 89 |
| P1-V03 | Slicer | Product category | 89 |
| P1-V04 | Card | Total Revenue | 90 |
| P1-V05 | Card | Total Orders | 90 |
| P1-V06 | Card | Average Order Value (AOV) | 90 |
| P1-V07 | Card | Revenue MoM Growth (latest month) | 91 |
| P1-V08 | Clustered column chart | Monthly revenue (R$ millions) | 91 |
| P1-V09 | Clustered column chart | Revenue MoM % by month | 92 |
| P1-V10 | Line and clustered column chart | Category Pareto — top 15 categories | 93 |
| P1-V11 | Clustered column chart | Revenue by customer state (R$ millions) | 93 |
| P2-V01 | Slicer | Order date range | 96 |
| P2-V02 | Slicer | Customer state | 96 |
| P2-V03 | Slicer | Product category | 97 |
| P2-V04 | Matrix | Revenue by category × month (R$) | 97 |
| P2-V05 | Clustered bar chart | Payment mix — ALL payment records | 98 |
| P2-V06 | Clustered column chart | Installment behaviour — ALL payment records | 98 |
| P2-V07 | Line chart | Average order value by month (R$) | 99 |
| P2-V08 | Card | Top-10 sellers' share of revenue | 99 |
| P2-V09 | Clustered bar chart | Top 10 sellers by revenue (R$ thousands) | 100 |
| P3-V01 | Card | Customers (unique persons) | 102 |
| P3-V02 | Card | Repeat customers (2+ delivered orders) | 103 |
| P3-V03 | Card | One-time customers | 103 |
| P3-V04 | Card | Repeat Customer Rate — All Time | 104 |
| P3-V05 | Matrix | Cohort retention — % of each monthly cohort that bought again, by mont | 104 |
| P3-V06 | Clustered bar chart | One-time vs repeat customers | 105 |
| P3-V07 | Card | Month-1 retention — customers who bought again in the next month | 105 |
| P3-V08 | Clustered bar chart | RFM segments — share of customers | 106 |
| P3-V09 | Clustered bar chart | RFM segments — share of revenue | 106 |
| P4-V01 | Slicer | Order date range | 109 |
| P4-V02 | Slicer | Customer state | 109 |
| P4-V03 | Card | Median delivery time (days) | 110 |

| ID | Type | Title | Page |
|---|---|---|---|
| P4-V04 | Card | Late delivery rate | 110 |
| P4-V05 | Card | Late delivered orders | 110 |
| P4-V06 | Card | Delivered orders | 111 |
| P4-V07 | Clustered column chart | Delivery-time distribution (orders by days to deliver) | 111 |
| P4-V08 | Line chart | Late delivery % by purchase month | 112 |
| P4-V09 | Clustered column chart | Review score: late vs on-time orders | 112 |
| P4-V10 | Clustered column chart | Share of 1–2★ reviews | 113 |
| P4-V11 | Table | Late vs on-time: reviewed orders (n), scores, 1–2★ share | 114 |
| P4-V12 | Line chart | 28-day order forecast (all placed orders) | 114 |
| P4-V13 | Card | Forecast total (orders) | 115 |
| P4-V14 | Card | Lower bound (sum) | 115 |
| P4-V15 | Card | Upper bound (sum) | 116 |

# Appendix D — Official links

| Resource | Address |
|---|---|
| Docker Desktop for Windows | https://docs.docker.com/desktop/setup/install/windows-install/ |
| Power BI Desktop (Microsoft Store) | https://aka.ms/pbidesktopstore |
| Power BI Desktop (Download Center) | https://www.microsoft.com/download/details.aspx?id=58494 |
| Python for Windows | https://www.python.org/downloads/windows/ |
| Git for Windows | https://git-scm.com/downloads/win |
| Scoop | https://scoop.sh |
| Chocolatey | https://chocolatey.org |
| Kaggle CLI | https://github.com/Kaggle/kaggle-cli |
| Olist dataset on Kaggle | https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce |
| Power Query PostgreSQL connector | https://learn.microsoft.com/en-us/power-query/connectors/postgresql |
| Set and use date tables | https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-date-tables |
| Slicers in Power BI | https://learn.microsoft.com/en-us/power-bi/visuals/power-bi-visualization-slicers |
| Page display settings | https://learn.microsoft.com/en-us/power-bi/create-reports/power-bi-report-display-settings |
| Sort one column by another | https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-sort-by-column |
| Conditional formatting | https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-conditional-table-formatting |
| Design reports for accessibility | https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-accessibility-creating-reports |

*Menu names and installers change; if a link has moved, search the vendor's site for the same title.*

# Glossary

| Term | Meaning |
|---|---|
| **Aggregation** | Combining many rows into one number, for example a sum or a count. |
| **Alt text** | A text description of a visual for people who use screen readers. |
| **AOV** | Average order value: revenue divided by the number of orders. |
| **Association** | Two things varying together; it does not by itself show that one causes the other. |
| **Calculated column** | A DAX column computed row by row when the model loads. |
| **Calculated table** | A table created by a DAX expression. |
| **Cardinality** | How rows of two tables correspond: one-to-many, many-to-one, and so on. |
| **Cohort** | A group of customers who share a starting month, here their first purchase. |
| **Container (Docker)** | A running, isolated instance of an image; here the PostgreSQL server. |
| **Context transition** | DAX turning the current row into filters when a measure is evaluated inside a row context. |
| **CTE** | Common table expression: a named temporary result inside a SQL query (`WITH … AS`). |
| **Cross-filter direction** | Which way a filter flows along a relationship; here always from date table to order tables. |
| **DAX** | Data Analysis Expressions: the formula language of Power BI. |
| **Dimension table** | A table of descriptive attributes such as customers or dates. |
| **Docker Compose** | A tool that starts containers from a YAML file; here `docker-compose.yml`. |
| **Fact table** | A table of events or transactions, such as orders or order items. |
| **Filter context** | The set of filters in force when a measure is evaluated. |
| **Foreign key** | A column that must match a key in another table. |
| **Grain** | What one row of a table represents. |
| **Healthcheck** | A command Docker runs to decide whether a container is ready (`pg_isready`). |
| **Holt-Winters** | A forecasting method with level, trend and seasonality components. |
| **Import mode** | Power BI copies the data into the report file. |
| **LAG** | A SQL window function that reads a value from a previous row. |
| **Lapsed One-Time** | Project segment: a single delivered order and low recency. |
| **Mann–Whitney U** | A rank-based test of whether one group tends to have larger values than another. |
| **Measure** | A DAX formula evaluated at report time in the current filter context. |
| **Median** | The middle value when values are sorted. |
| **Month offset** | Whole months between a customer's first purchase month and a later month. |
| **Native query** | SQL sent unchanged to the database by Power Query. |
| **NTILE** | A SQL window function that splits ordered rows into equal-sized groups. |
| **p-value** | The probability of a result at least this extreme if there were no real difference. |
| **Pareto chart** | Bars in descending order with a cumulative-share line. |
| **PBIX** | The Power BI Desktop report file. |
| **Pipeline** | The ordered stages that load, check and transform the data. |
| **Primary key** | A column (or set) that uniquely identifies each row. |
| **Power Query** | The Power BI component that loads and shapes data before it enters the model. |
| **Rank-biserial correlation** | An effect size for rank tests, from −1 to +1. |
| **RFM** | Recency, frequency and monetary value: a way of scoring customers. |

| Term | Meaning |
|---|---|
| **Relationship** | A link between two model tables that lets filters pass. |
| **Retention** | The share of a cohort that buys again in a later period. |
| **Row context** | DAX's current row while a column or iterator is calculated. |
| **Schema** | A named group of tables in PostgreSQL (`stg`, `analytics`). |
| **Seasonal naive** | A forecast that repeats the last observed cycle. |
| **Slicer** | A control that filters a page. |
| **Staging** | A raw copy of the source data before transformation. |
| **Star schema** | Fact tables surrounded by dimension tables. |
| **venv** | A Python virtual environment: an isolated set of packages. |
| **View** | A saved SQL query that behaves like a table. |
| **Volume (Docker)** | Storage that survives container removal; here the database files. |
| **Walk-forward backtest** | Evaluating a forecast by repeatedly training on the past and testing on the next window. |
| **WAPE** | Weighted absolute percentage error: total absolute error divided by total actual. |
| **Window function** | A SQL function that computes across related rows without collapsing them. |